

Collaborative Robot Lab Manual

Universal Robots Hands-On Activities – Modules 5 to 11

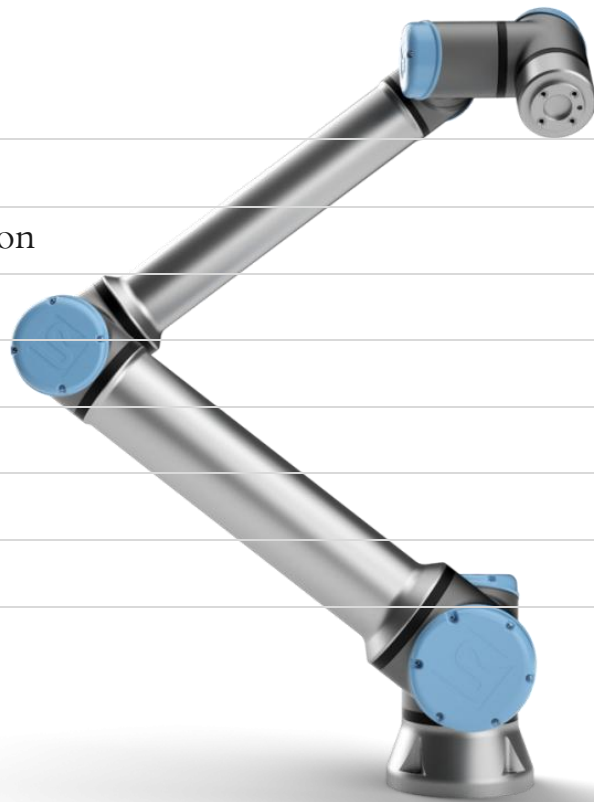
CONTENTS

Module 5	Getting Started	9 activities
Module 6	Operations and Motion	9 activities
Module 7	Pick and Place	3 activities
Module 8	Programming	4 activities
Module 9	Palletizing	3 activities
Module 10	Features/Planes	4 activities
Module 11	Safety Settings	5 activities

COMPILED BY

Dr. Wenhao Yang

Department of Industrial & Systems Engineering, Lamar University



Module 5: Getting Started

Hands-On Activities (UR Academy Student Module)

Source: Universal Robots Academy, Student Module 5 “Getting Started”. Activities and figures reproduced in original order; figure numbers follow the source.

Hands-On Activity 1: Review Robot Control Box Hardware

Below we will start our first hands-on activity. We will start by opening the control box. The power should be off. (Ensure you have approval before working on the robot.)

1. Use the provided key to open the control box.
 - This is a dedicated key for the control box. Keep secure with the equipment.
2. Find the various types of Inputs and Outputs (I/O) on the **Safety Control Board**.

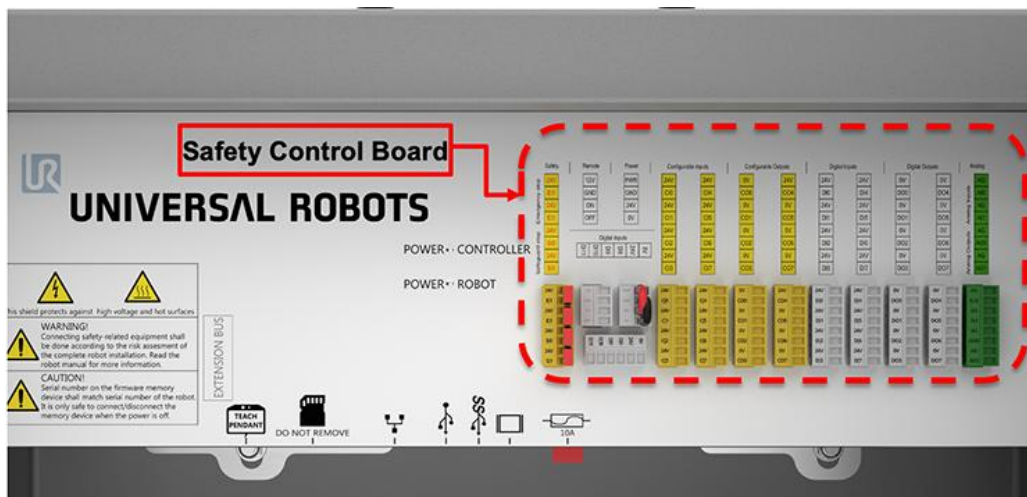


Figure 5.1A – Controller Available IO

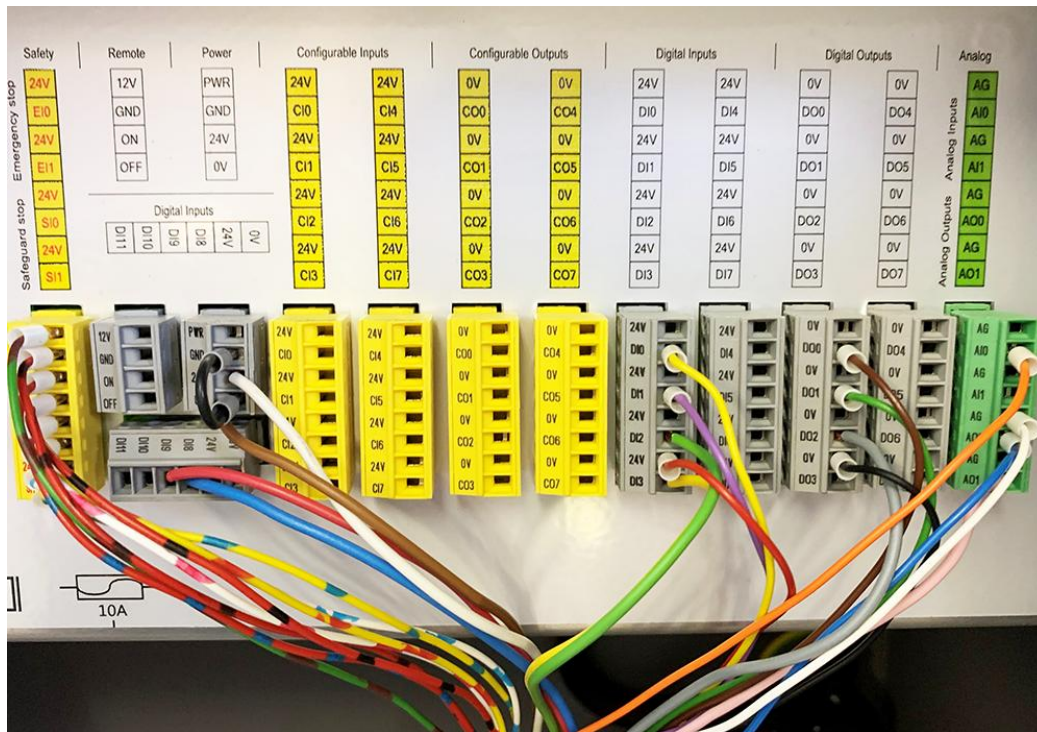


Figure 5.1B – Controller Wired IO

- **Dedicated Redundant Safety Input.**
 - **Configurable IO.**
 - **Digital IO.**
 - **Analog IO.**
3. Find the connection points for:
- **Teach Pendant.**
 - **Ethernet.**
 - **USB 2.0/3.0.**

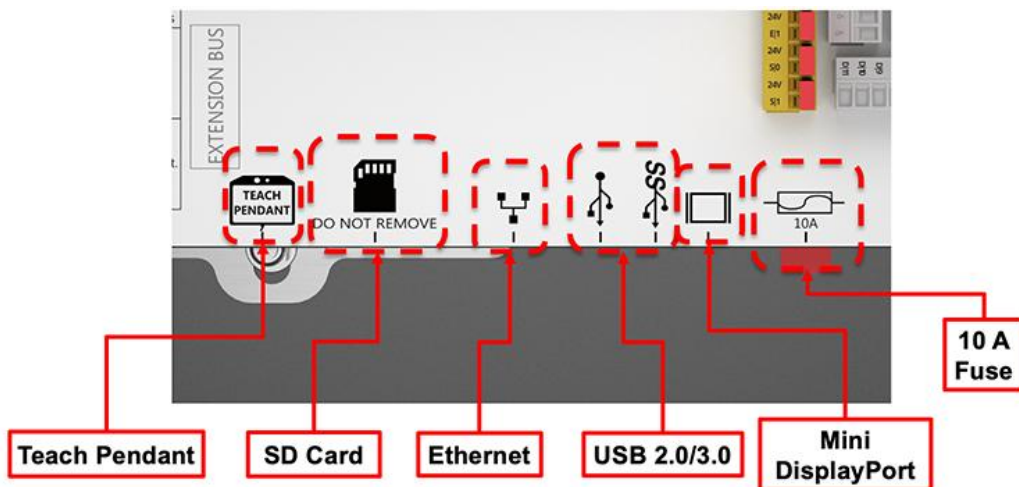


Figure 5.2 – Controller Connections

4. Close the control box and secure the Key.

Hands-On Activity 2: Initialize the Robot

Start the robot up and initialize the arm.

1. Press the **Power Button** on the teach pendant to start the system.
 - The power button will illuminate, and the **PolyScope** software will begin the boot sequence. The robot controller and teach pendant will then boot up, displaying the current software version.
 - Ensure the software version matches the latest version released by Universal Robots. Version 5.12 is the latest as of 8/18/2022.
2. Press the red button in the lower left of the screen. It is currently labeled as **Power Off**. This will open the initialize screen.
 - Verify that the **Active Payload** matches the physical robot. The end effector should not be attached, so the **Active Payload** should read 0.0 kg.

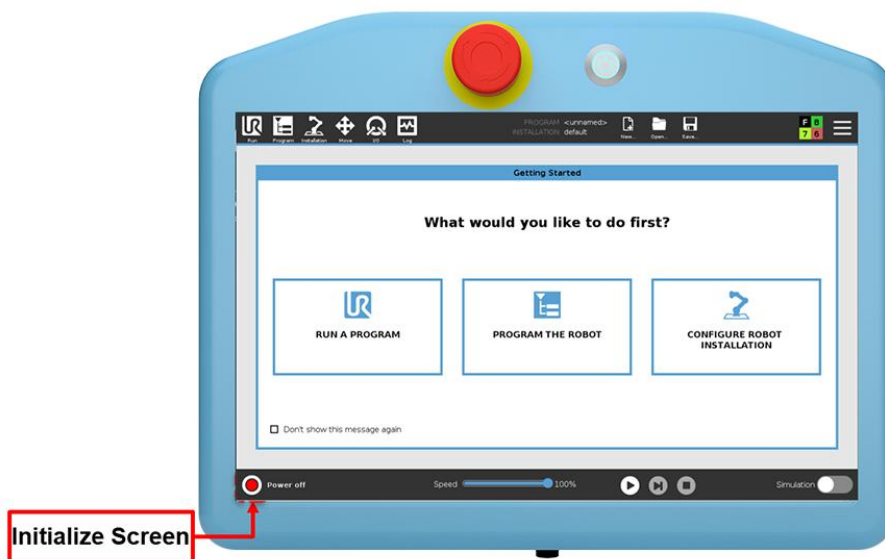


Figure 5.3 – Screenshot Startup Screen

3. Press the **ON Button** to supply power to the robot arm.
 - Note the **Robot Active** indicator is yellow. Stay clear of the robot when starting it.



Figure 5.4 – Screenshot of Idle State

4. Press the **START Button** to release the brakes and move the robot to **Robot Operational**, or Robot in **Normal Mode**.
 - Note that the robot will move slightly and make *clicking* sounds. This is normal for the robot as it releases the brakes.
5. Click on **Exit**. You have now started up the robot arm!

Hands-On Activity 3: Pendant Navigation

1. From the **Getting Started Screen**, press the **Program the Robot Button**.
 - A selection of three icons is present.
 - If this selection is not present, move to the next step.
2. Sequence through the following **General Header Icons** by pressing the icon in the upper left of the screen.

	Run – a simple means of operating the robot using pre-written programs.
	Program – creates and/or modifies robot programs.
	Installation – configures robot arm settings and external equipment.
	Move – controls and/or regulates robot movement.
	I/O – monitors and sets live Input/Output signals to and from the robot control box.
	Log – indicates robot health as well as any message, warning, or errors.

Figure 5.5 – General Header Icons

- **Run.**
- **Program.**
- **Installation.**
- **Move.**
- **I/O.**
- **Log.**

3. Review **Footer Icons.**

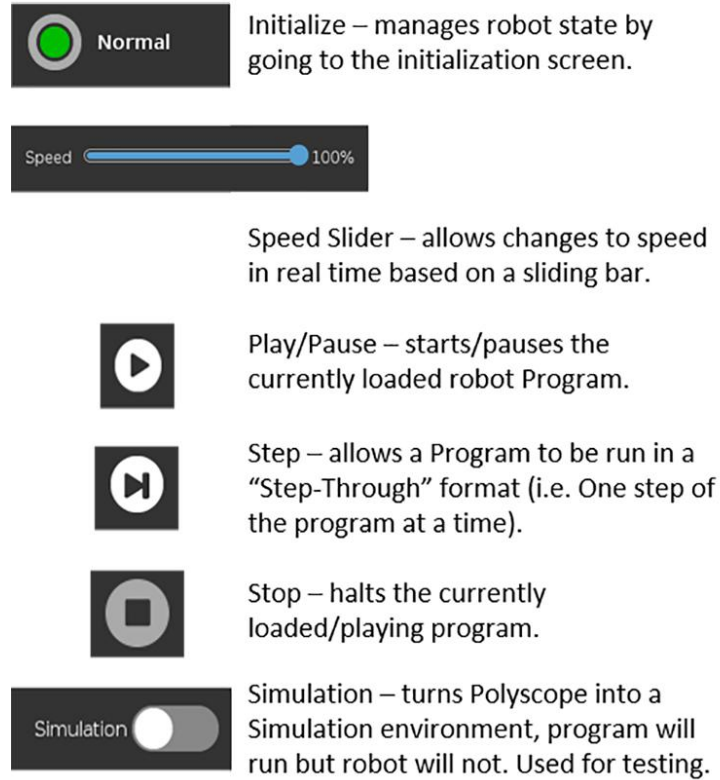


Figure 5.6 – Footer Icons

- Move the **Speed Slider Bar** from 100% to 25% by dragging the blue button to the left.
 - Note the location of the **Play Button**, **Step Button**, and **Pause Button**.
 - **Simulation** can be switched to ON to allow for testing without robot movement.
4. Press the **Program Icon** located in the header.
 5. Review the **Program Tree Toolbar** located at the bottom of the window.

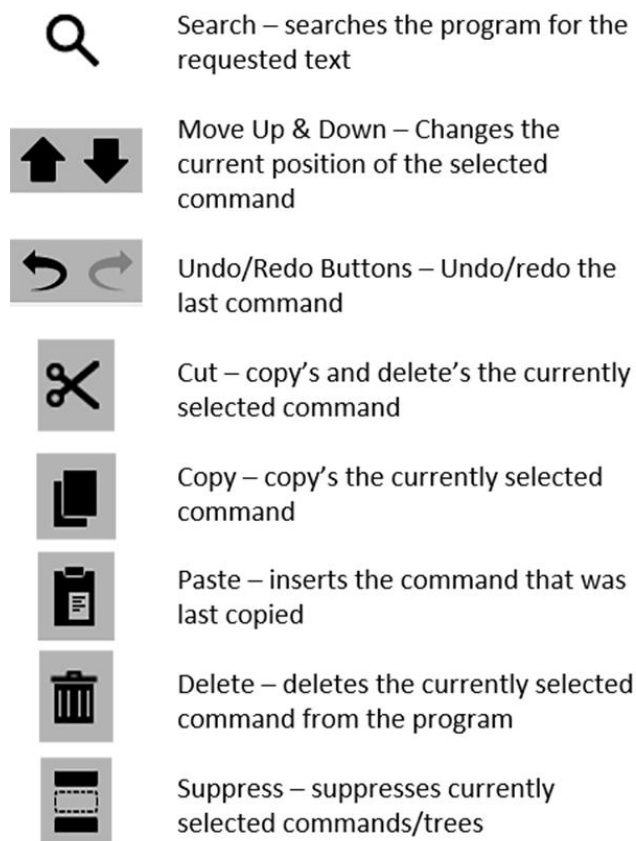


Figure 5.7 – Program Tree Toolbar

- When creating and editing your program, the **Cut and Paste** function is used frequently.
 - The Arrows allow you to move lines of code up and down the program tree as needed.
6. Sequence through the **Program Screen Tabs**.
- **Command.**
 - This is the default screen.
 - Uncheck the “**Program Loops Forever**” box.
 - **Graphics.**
 - Use the **Zoom +/- Slider Bar** to change the size of the robot image.
 - Click on the robot to adjust position on the screen.
 - **Variables.**
 - No variables will appear here at this time.

Hands-On Activity 4: Jog the Robot

1. Press the **Move Icon** in the header.
2. Jog the robot by each axis by pressing the arrows in the **Joint Position Window**. Start with the base axis.
 - Sequence through to the **Wrist 3** axis.
 - Start with the robot **Feature** in **View** and the robot in a neutral pose as shown in Figure 5.8. Note the angles of the **Joint Position**. This position will be the **Start Position**.

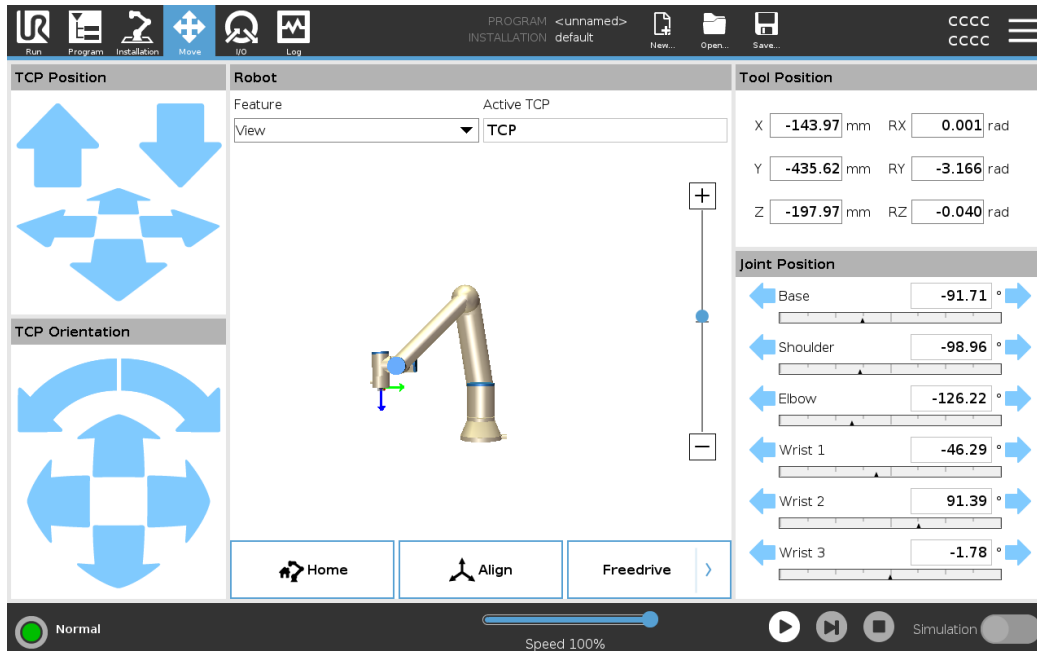


Figure 5.8 – Start Position

- While working with the live robot you may increase the speed as you become comfortable with its operation and movement.
 - When working in the simulated environment increase the Speed to 100%. Adjust the robot image position if it is difficult to view the motion.
 - Note the change in angular value as each joint is moved. There is also a reference marker identifying the location of each joint within the working envelope.
3. While in the **View Feature** move the robot with the **TCP Position Arrows** and the **TCP Orientation Arrows**.
 - Note the blue dot indicating the **Tool Center Point (TCP)** is shown at the center of the flange on **Wrist 3** since the tool data has not yet been defined.
 4. Switch to the **Base Feature**. Move the robot with the **TCP Position Arrows** and the **TCP Orientation Arrows**.
 5. Switch to the **TCP Feature**. Move the robot with the **TCP Position Arrows** and the **TCP Orientation Arrows**.

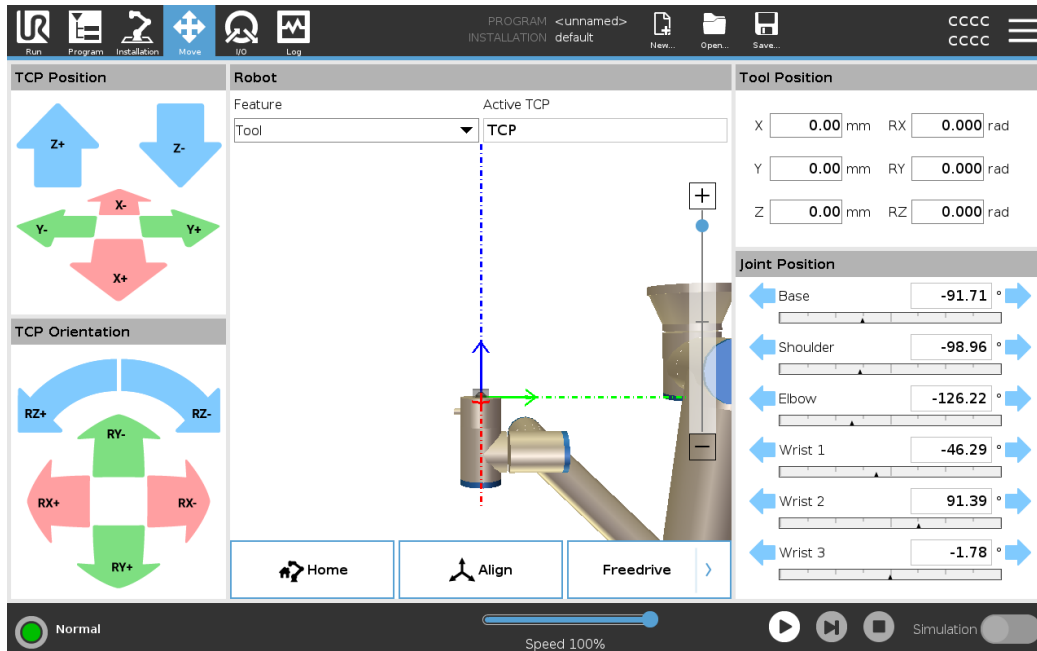


Figure 5.9 – Feature - Tool

- Note the Z+ and Z- direction of motion on the robot has switched.
6. Press the **Home Button**.
 - Note the difference between the current robot position and the **Home** position.
 - Use the “**Move robot to: New position**” to move the robot to **Home**.
 - Note the simulated motion.
 - You must hold the “**Move robot to: New position**” to continue motion. This is a safety feature to avoid collisions.
 - If you encounter an interference you can switch to **Manual** and navigate past the obstacle.
 - When the robot reaches the **Home Position** it will prompt you to continue.
 - Move the robot back to a neutral pose.
 7. Press the **Align Button**.
 - Use the “**Move Robot to: Aligned Z-Axis**” button to align the **Z-Axis** of the TCP with the **Base Z-Axis**.
 8. Press on **Freedrive** in the **User Interface (UI)**.
 - For this exercise, you will use the **Freedrive Button** on the UI to allow the robot to be moved manually. There are four modes available. Use the **Custom** settings in Figure 5.10 to move the robot manually.
 - **All axes are free.**
 - **Translation.**
 - **Plane.**
 - **Rotation.**
 - **Custom:** Various combinations of X, Y, Z, RX, RY, RZ.

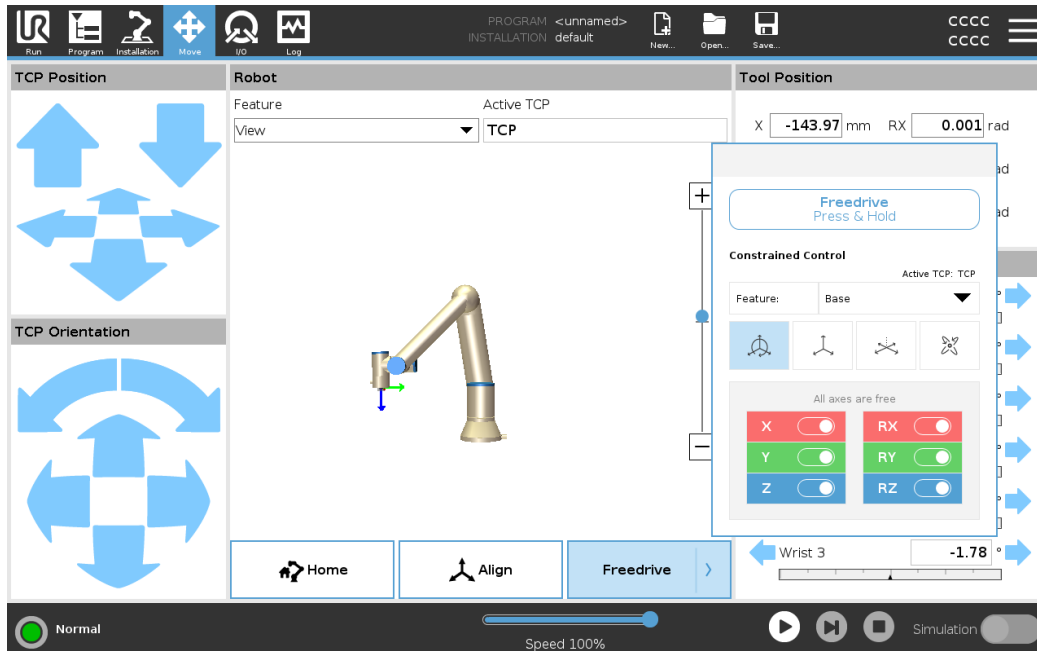


Figure 5.10 – Custom Settings

- Repeat this exercise using the **Freemove Button** on the back of the TP to allow the robot to be moved manually. Note that you still need to control the mode desired in the UI.
- When using the simulator, the **Freemove Window** will display the settings. Simulating the robot motion is not available.

Hands-On Activity 5: Setup Tool Center Point (TCP)

1. Use your **End of Arm Tooling (EOAT)** for this exercise.
 - Note that there are several manufacturers of grippers in the **UR Caps** program.
2. Press the **Installation icon** located in the **Header**.
 - From the **General** pulldown menu, located on the left, select the **TCP tab**.
 - If the payload has not been set, the 0.0 kg entry will be highlighted yellow.
3. Use a ruler to obtain the TCP dimensions from the tool flange.

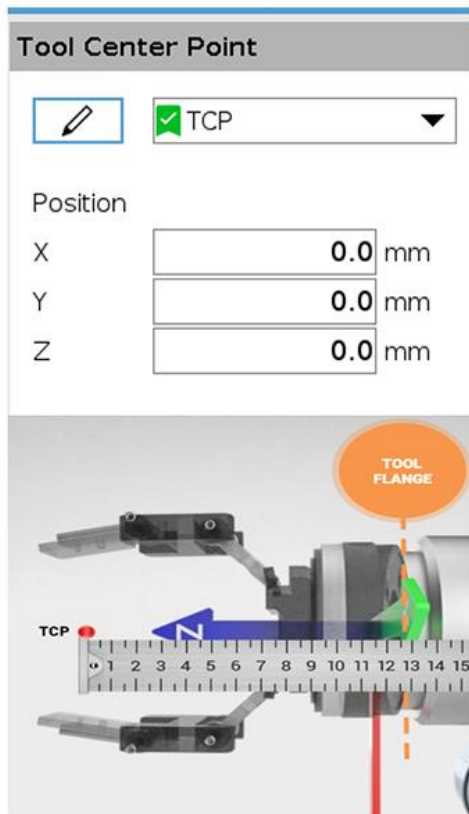


Figure 5.11 – Measuring TCP

4. Use a drawing or CAD data to obtain the TCP dimensions from the tool flange, or
5. Use the **setup Wizard** to estimate the TCP dimensions from the tool flange.

Tool Center Point

 ☒ TCP   

Position

X	0.0 mm
Y	0.0 mm
Z	0.0 mm

 **Measure**

Teach TCP Position

● Calculated position:
[-19.04, -91.06, 34.24]

Move the TCP to the same position from different angles.

☒ Modify point 1

☒ Modify point 2

☒ Modify point 3

☒ Modify point 4



Tool Center Point

 ☒ TCP 

Position

X	-19.04 mm
Y	-91.06 mm
Z	34.24 mm

Figure 5.12 – TCP Wizard

Hands-On Activity 6: Setup Payload

1. Use a scale to obtain the gripper weight which is the initial **Payload** value.

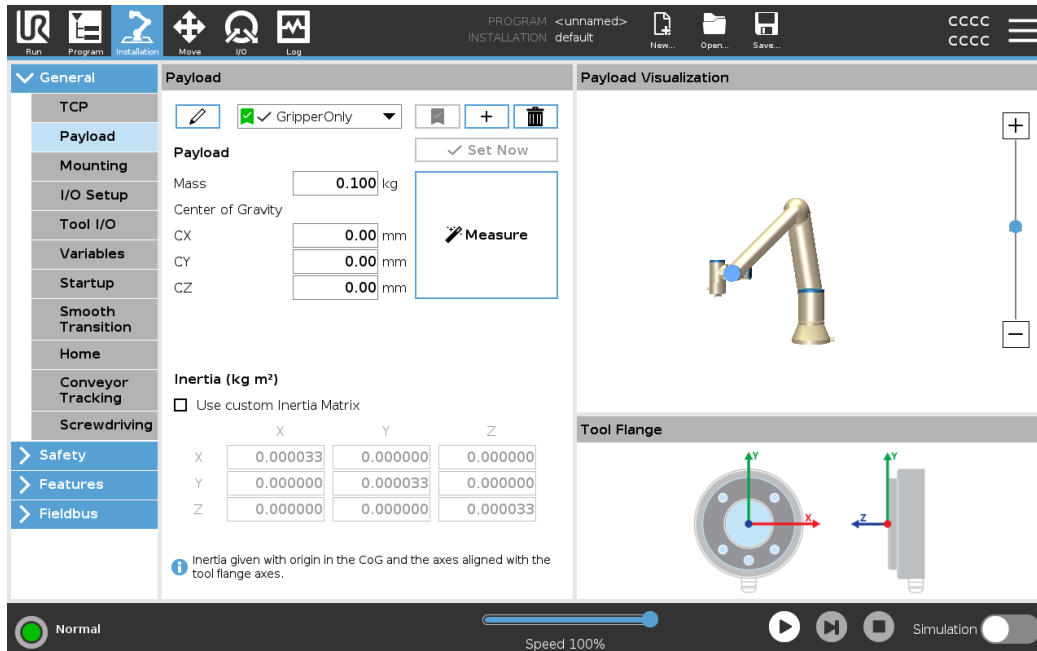


Figure 5.13 – Payload Entry

- The payload should not exceed 3.0 kg for the UR3e robot.
 - The payload should not exceed the max payload for the UR Robot. This is based on the robot's name/number:
 - 3.0 kg for UR3e,
 - 5.0 kg for UR5e,
 - 12.5 kg for UR10e, and
 - 16.0 kg for UR16e.
 - The payload will need to be modified in the program when handling a part.
2. Other options/considerations:
- Use a drawing or CAD data to obtain the gripper weight which is the **Payload** value, or
 - Use the **setup Wizard** to estimate the gripper weight which is the **Payload** value.

Hands-On Activity 7: Setup Center of Gravity (COG)

1. Use a drawing or CAD data to obtain the **COG** location for the gripper, or
2. Use the **setup Wizard**.

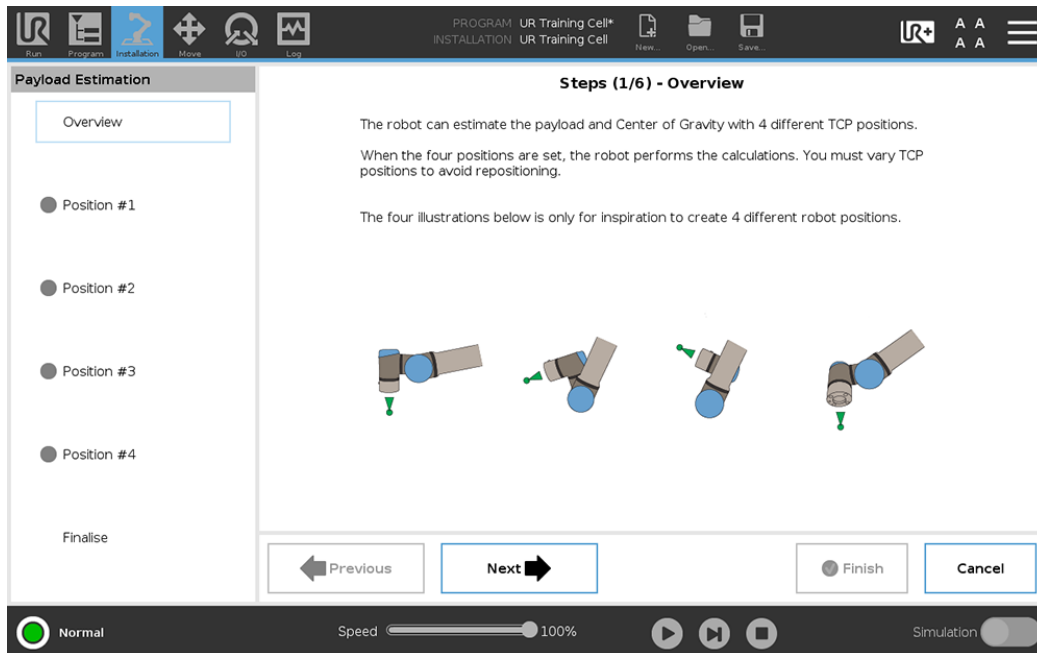


Figure 5.14 – COG Wizard

Hands-On Activity 8: Saving Files

1. Press the **Save icon** located in the **Header**.
2. You have the option of saving the program file, installation file, or both.
3. Click to save both the **Program File** and the **Installation File**. Save the program file as **UR TRAINING CELL**.

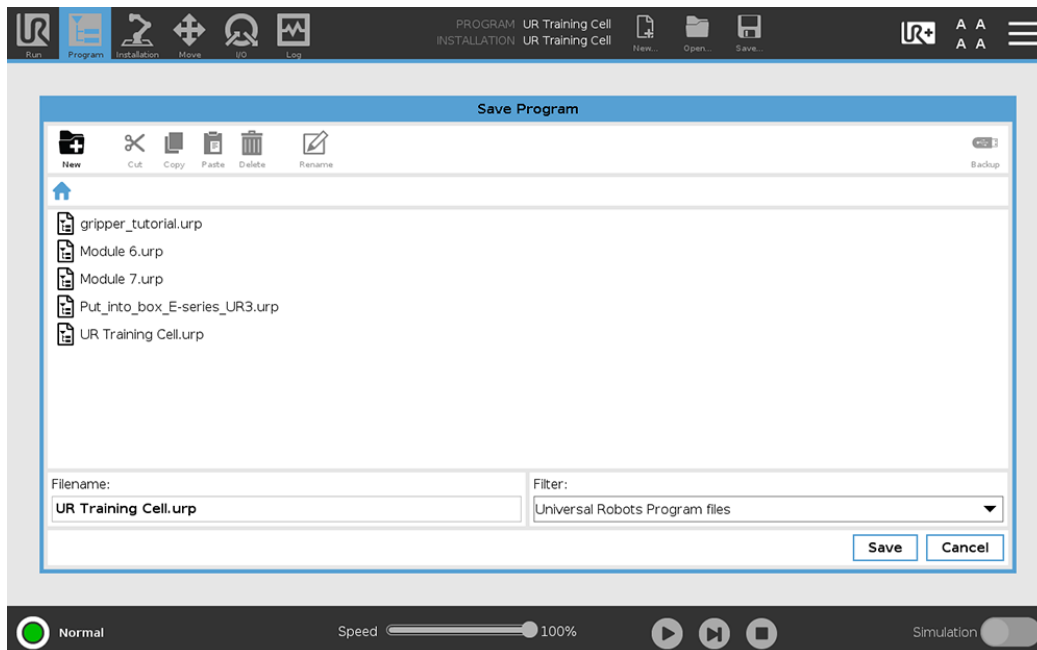


Figure 5.15 – Saving Files

Hands-On Activity 9: Trace the Conveyor Using Different Methods

1. Go to the **Move Tab**.
2. Adjust the speed using the speed slider as you approach objects. Note: Slower speed settings will provide smaller move increments each time a move button is pushed.
3. Move the robot to the **Start Position** (Figure 5.8).
4. Use **Freedrive** to move the robot slowly in free space.
 - Use the custom constrained mode selecting X, Y, Z, RZ.
 - It is easy to violate safety in the most sensitive settings. If the robot alarms you can reset and continue.
 - **Freedrive** is good for a quick reach check.
 - **Freedrive** is good for recovering from a crash condition.
5. Hover the robot gripper approximately 25 mm above the corner of the conveyor above the sensor at that location.

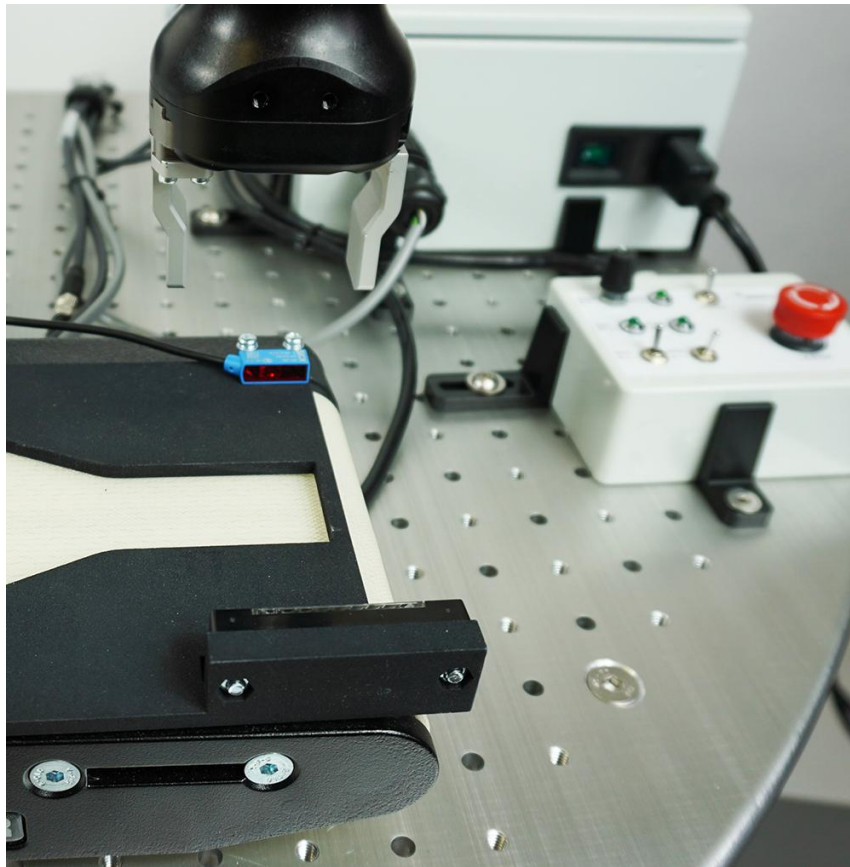


Figure 5.16 – Conveyor Corner

6. Select **View** from the **Feature dropdown menu**. Using this setting, move the robot to follow the outline of the conveyor with the gripper or EOAT.
7. Select **Base** from the **Feature dropdown menu**. Using this setting, move the robot to follow the outline of the conveyor with the gripper or EOAT.
8. Return the robot to the **Start Position**.

9. Use the **Tool Feature** to move up and down in the Z direction. Start with -Z as it is in the opposite direction of the **View** and **Base Features**.
10. Practice rolling the wrist about the **Start Position** using the arrows in the **TCP Orientation** window.
11. Return to **View Feature**. Put the robot in the pose shown in Figure 5.17. Move the robot in the Z+ and Z- directions. Switch to the **Tool Feature** and move up and down in the Z direction. The **Tool Feature** is useful for programming the loading and unloading of parts into machines at angles.

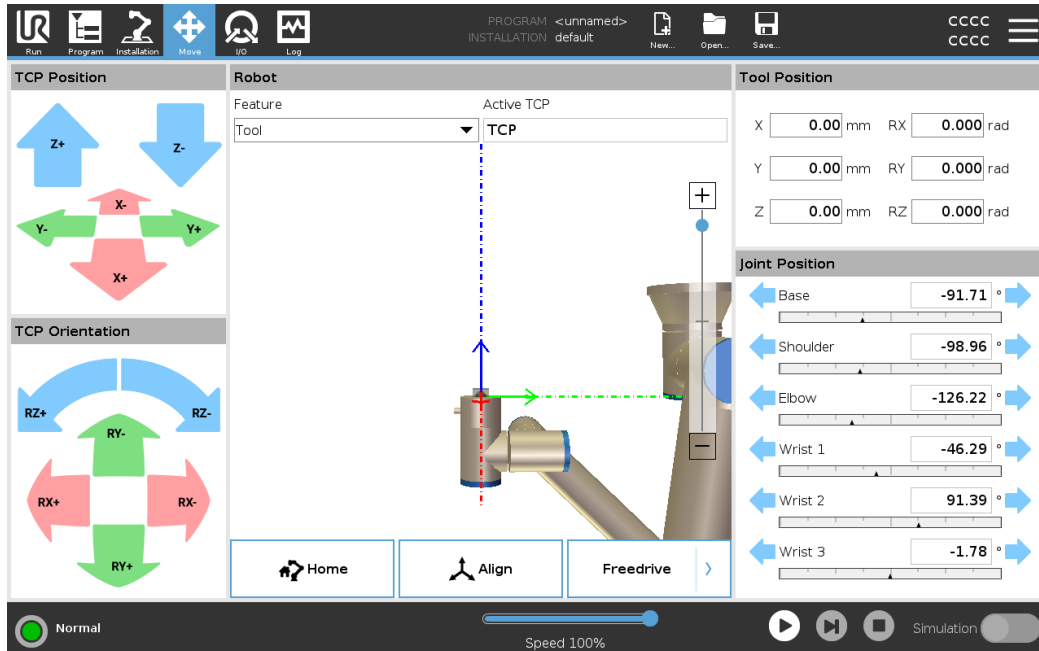


Figure 5.17 – Tool Feature Z Axis Movement

12. Return to the **View Feature** and Align the robot to the starting plane.
13. Return to **Start Position**.

Module 6: Operations and Motion

Hands-On Activities (UR Academy Student Module)

Source: Universal Robots Academy, Student Module 6 "Operations and Motion". Activities and figures reproduced in original order; figure numbers follow the source.

Hands-On Activity 1: Programming with Joint Moves

1. Make sure the robot is still in the **Start Position** in the **UR Training Cell** program.
2. Place a block in the nest on the right side of the conveyor.

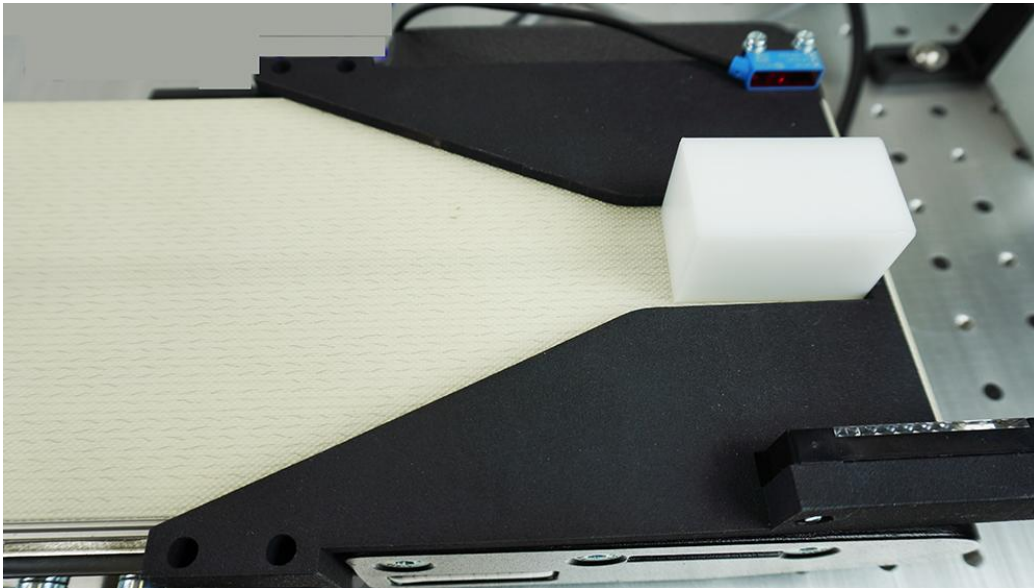


Figure 6.1 – Block Location

3. Create a new program named **JTrace** by using the **Save As command**.
4. Program with **Joint Moves** to trace the conveyor.
5. Use the following parameters.
 - Speed at 50%.
 - Default settings for the joint moves. The default speed is 60 degrees/sec.
 - No blending radius on the **Waypoints**.

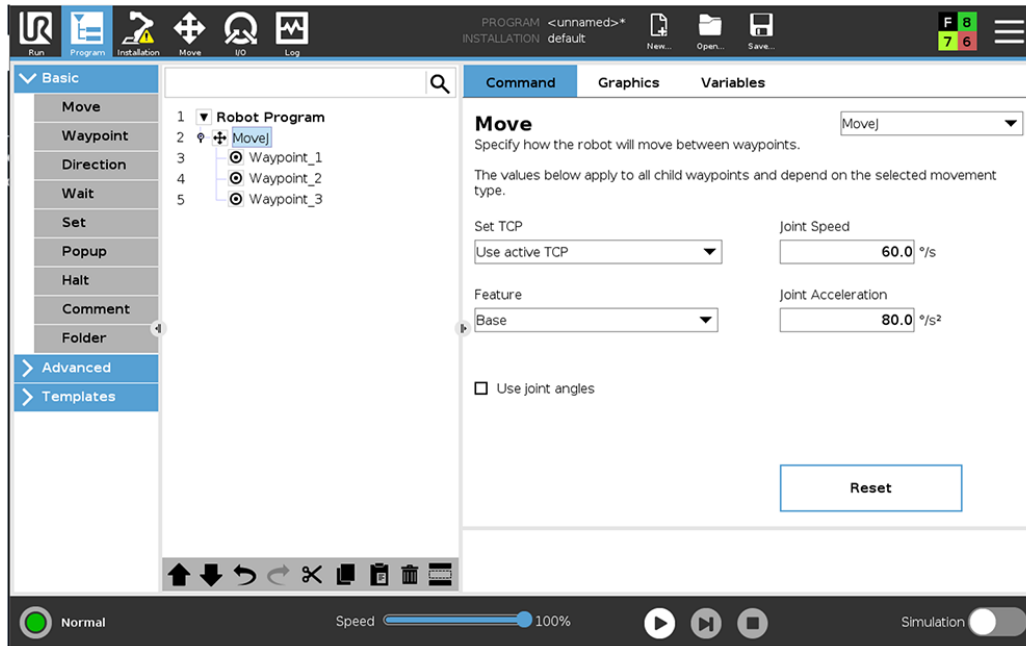


Figure 6.2 – Joint Move Screen

6. Save the **Start Position** in the program.
 - Create a **Waypoint** and name it **Start Position**. Set the point in a neutral pose over the conveyor.
 - Note: The program must be at this first point to run the program.

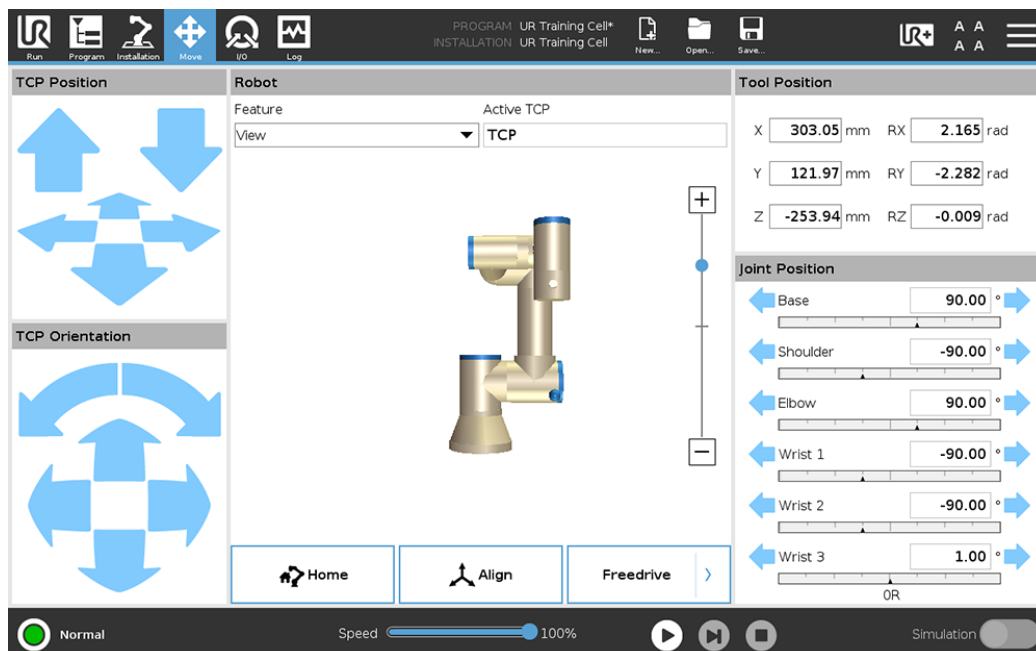


Figure 6.3 – Neutral Pose

7. Save the **Corner 1** in the program.
 - Create a **Waypoint** and name it **Corner 1**.
 - Set **Waypoint**.

- Move to the right to reach the edge of the conveyor using the right arrow in **TCP Position**.
 - Select **OK**.
8. Save the **Corner 2** in the program.
- Create a **Waypoint** and name it **Corner 2**.
 - Set **Waypoint**.
 - Move to the front edge of the conveyor using the approach arrow in **TCP Position**.
 - Select **OK**.
9. Save the **Corner 3** in the program.
- Create a **Waypoint** and name it **Corner 3**.
 - Set **Waypoint**.
 - Move to the left side of the conveyor using the left arrow in **TCP Position**.
 - Select **OK**.
10. Save the **Corner 4** in the program.
- Create a **Waypoint** and name it **Corner 4**.
 - Set **Waypoint**.
 - Move to the right to the edge of the conveyor using the right arrow in **TCP Position**.
 - Select **OK**.
11. Linking **Waypoints**.

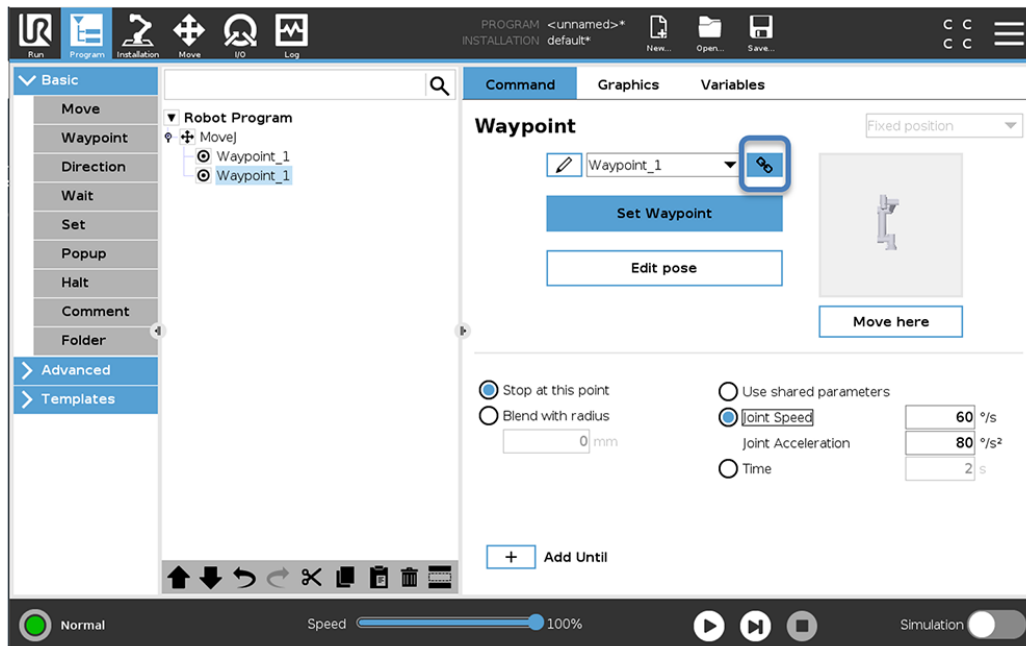


Figure 6.4 – Linking Waypoints

12. Create a **Waypoint** and link to **Corner 1**. Move to that position.
13. Create a **Waypoint** and link to **Start Position**. Move to that position.

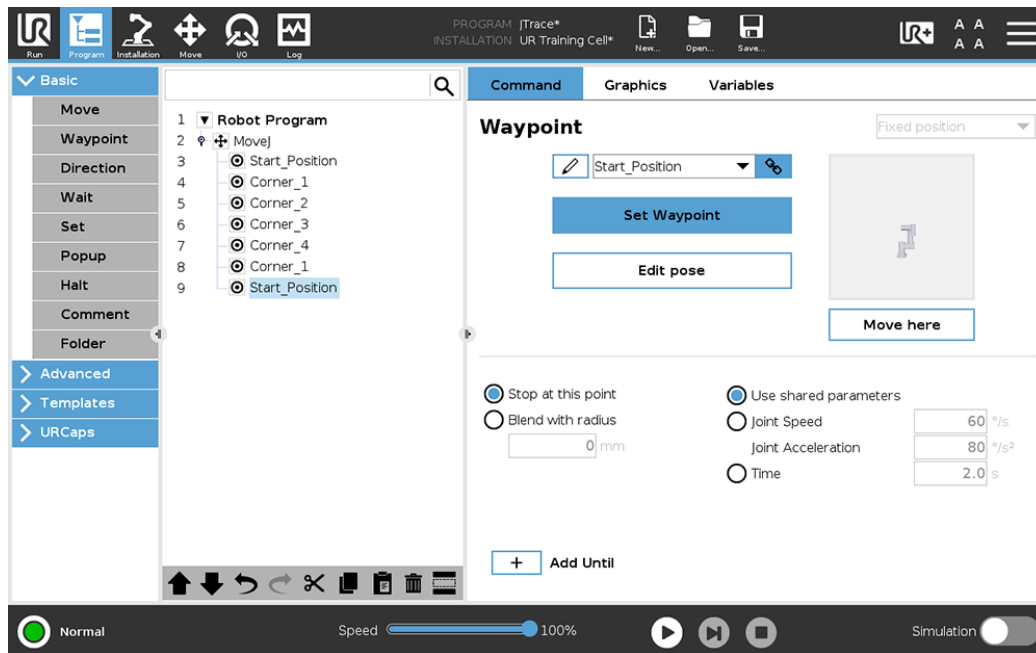


Figure 6.5 – JTrace Program

1. Run program by pressing **Play**. Press **Pause** immediately to step through the program one move at a time using **Next**.
2. Save the program.

Hands-On Activity 2: Programming with Linear Moves

1. Create a new program named **LTrace** by using the **Save As** command.
2. Program a **Linear Move** to trace the conveyor.
3. Change the **JMove** to a **LMove**.
4. Use the following parameters.
 - Set speed for linear moves at 125 mm/sec.
 - No blending radius for the **Waypoints**.

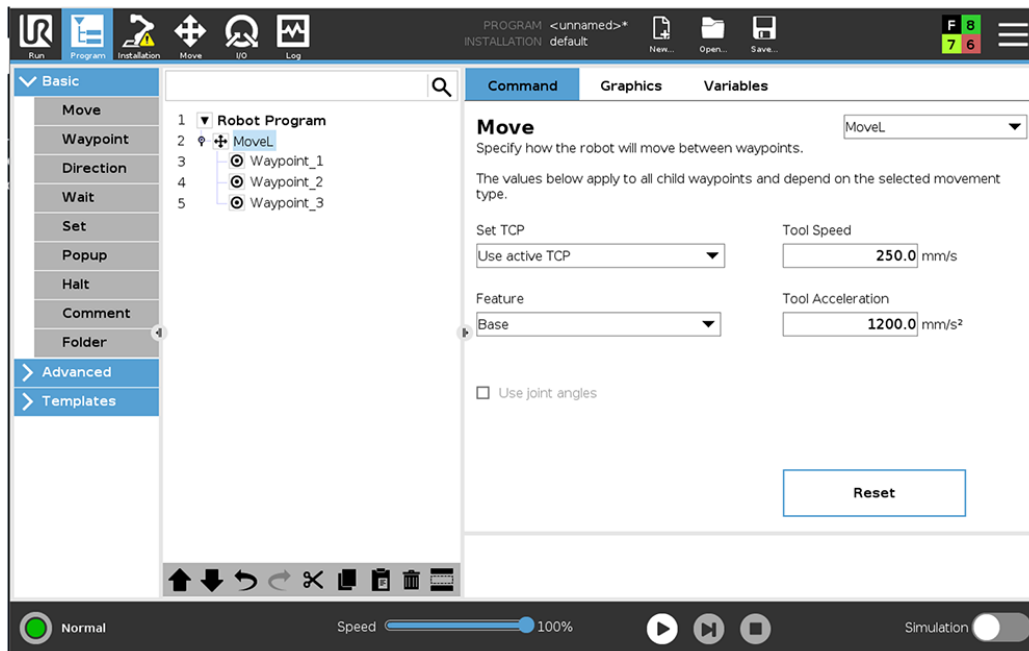


Figure 6.6 – Linear Move Screen

5. Run program.
6. Save the program.

Hands-On Activity 3: Programming with Process Moves

1. Create a new program named **PTrace** by using the **Save As** command.
2. Change the **LMove** to a **PMove**.
3. Program a **Process Move** to trace the conveyor.

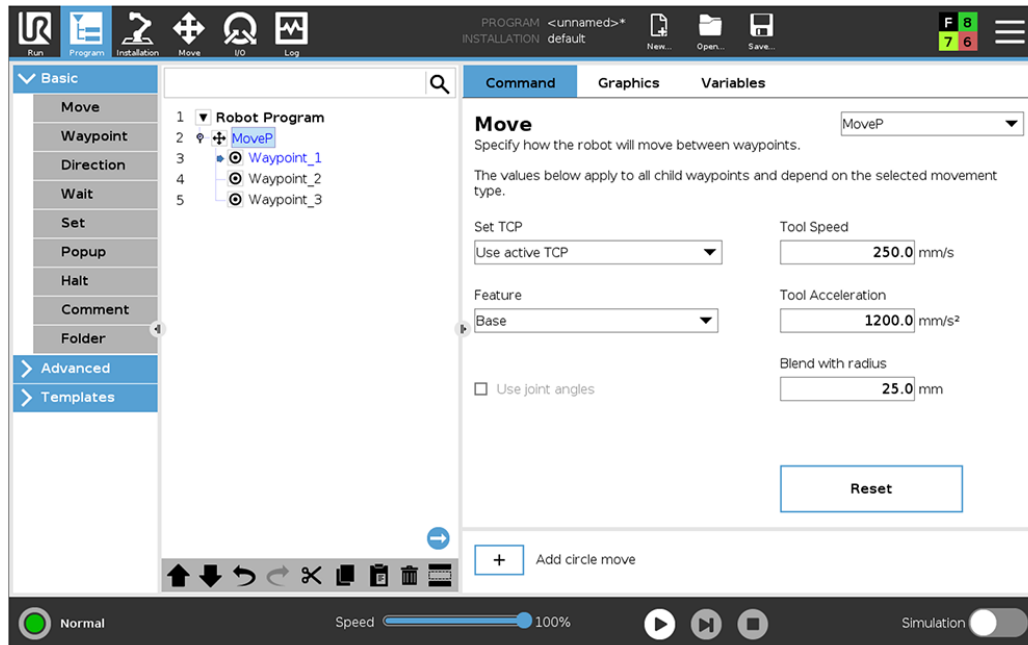


Figure 6.7 – Process Move Screen

4. Reduce Speed to 50 mm/s. **Process Moves** maintain constant TCP speed and require lower speeds.
5. Run program.
6. Save the program.

Hands-On Activity 4: Programming with Circle Moves

1. Create a new program named **CMov** by using the **Save As** command.
2. Program a **Circular Move** to make an arc from the **Start Position**.

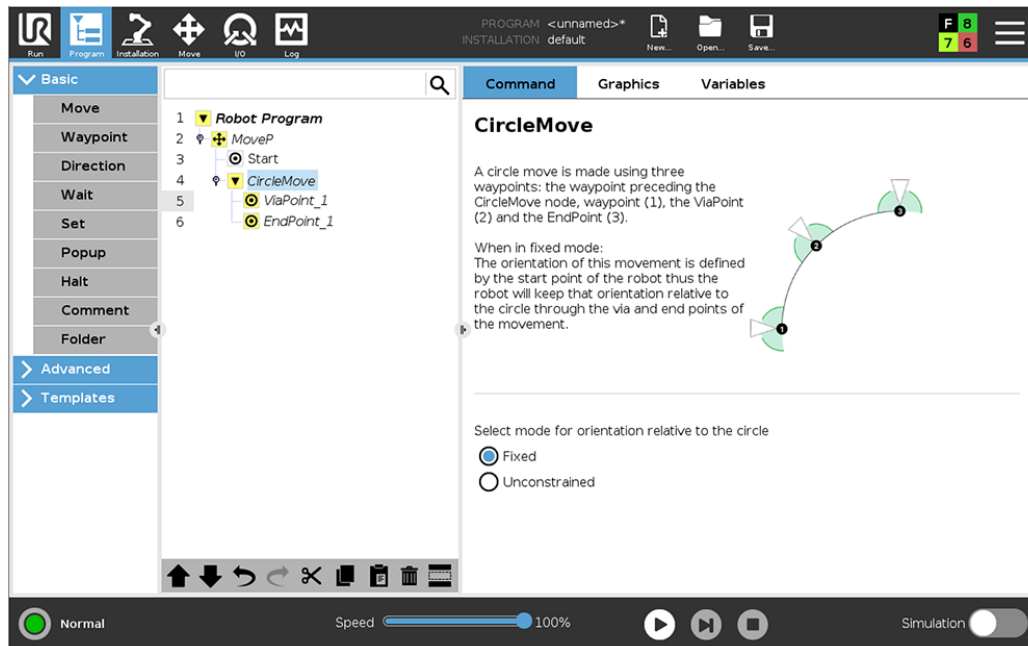


Figure 6.8 – Circular Move Screen

- Use **3-point method**.
3. Keep the **Start Position**.
 4. Delete all the Corner points in the program.
 5. Select the **Add circle move** in the **Move Command window** at the bottom.

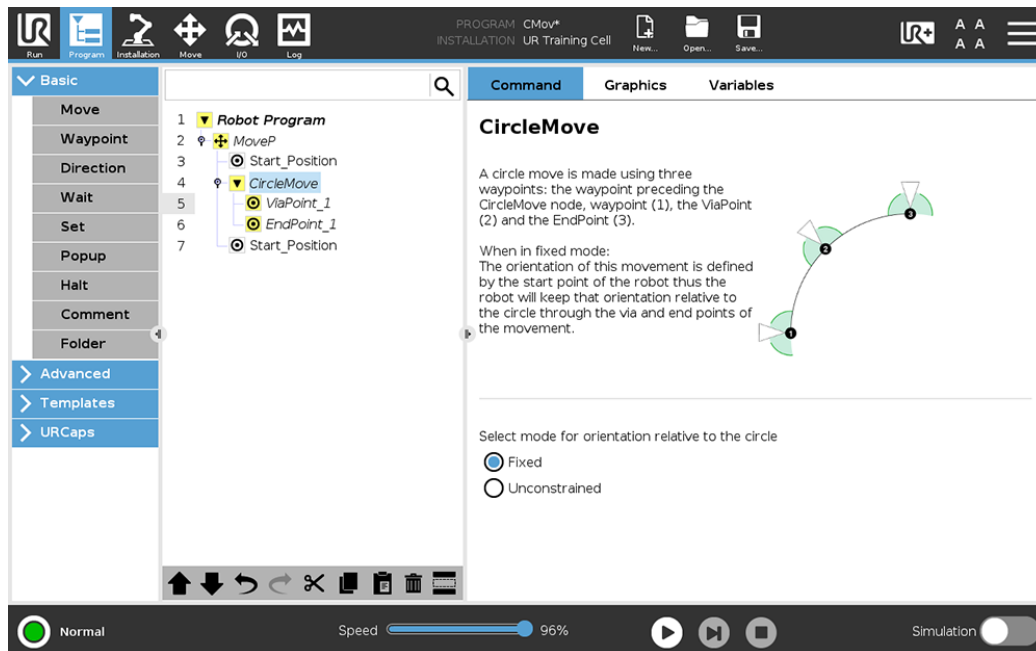


Figure 6.9 – CMov Program 1

- **Fixed method** maintains the tool orientation at the pick point for the rest of the circle points within this move.
6. Teach **Endpoint** by moving the robot to the front side of the conveyor using the approach arrow.
 7. Teach **ViaPoint** by moving back to the center of the conveyor and moving to the right 65 mm by increasing the value in the Y direction.
 8. Return to the **Start Position** and Run program.
 9. Within the **Circle Move** select the **Unconstrained method**.

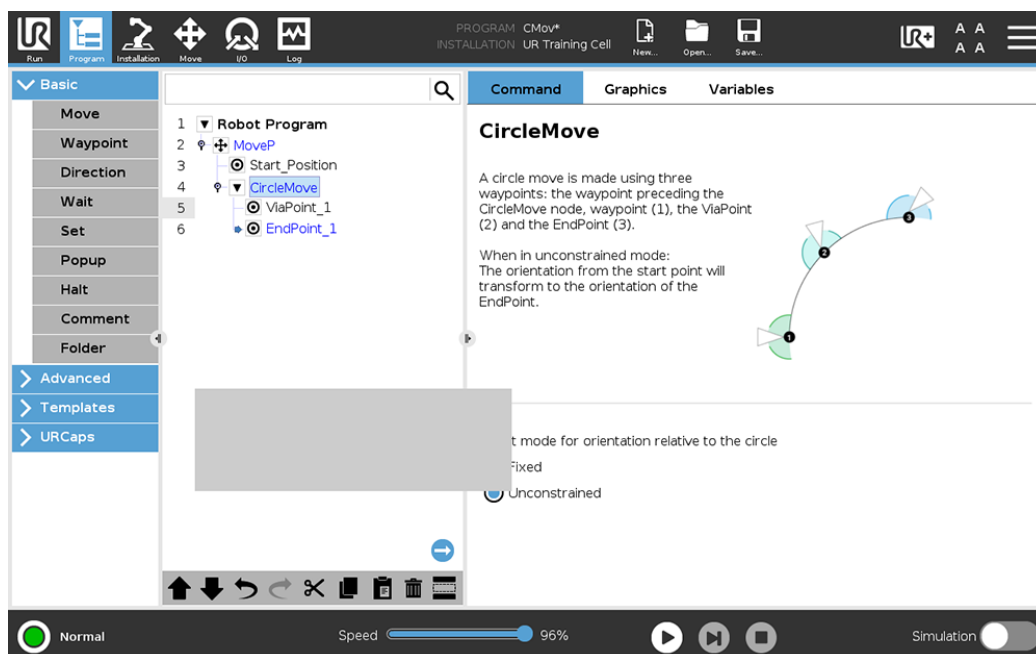


Figure 6.10 – CMov Program 2

- **Unconstrained method** will start with the orientation at the first point and will transform to the orientation at the end point.
10. Return to the **Start Position** and Run program. Note the difference in the rotation of the tool.
 11. Creating a complete circle.

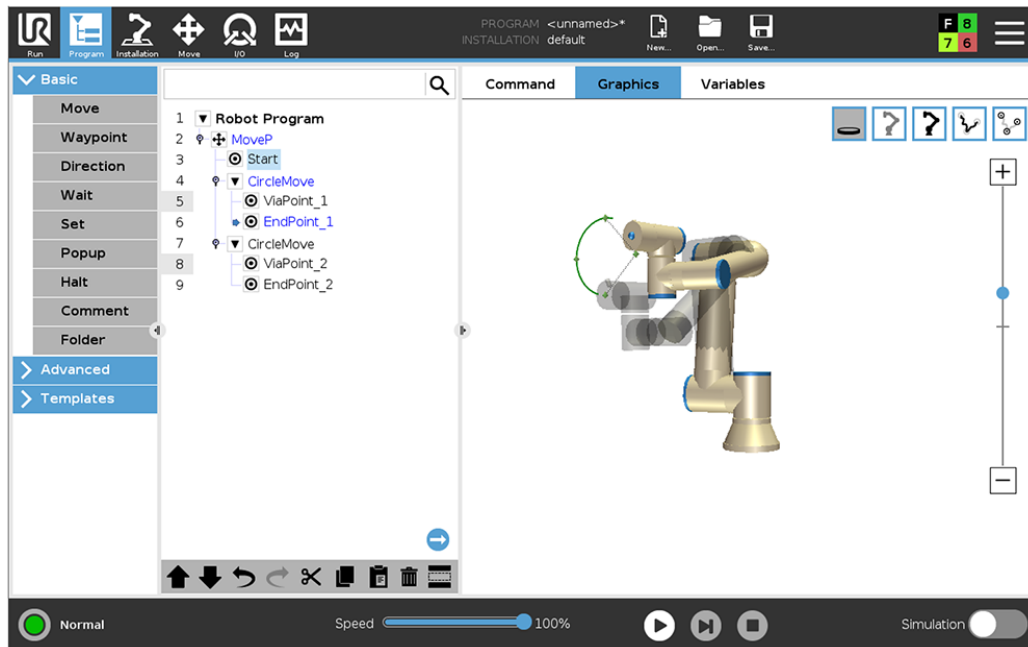


Figure 6.11 – Completing a Circle

12. Add another **MoveP** using:
 - 50 mm/s speed.
 - Link **Waypoint 1** to the **EndPoint 1**.
 - Link **Endpoint 2** to **Start Position**.
 - Set **ViaPoint 2** by moving to **ViaPoint 1** and moving to the left 130 mm by decreasing the value in the Y direction.
13. Add a **MoveJ** at the end of the program and link to the **Start Position**. Notice how it resets the rotation of the wrist axis.

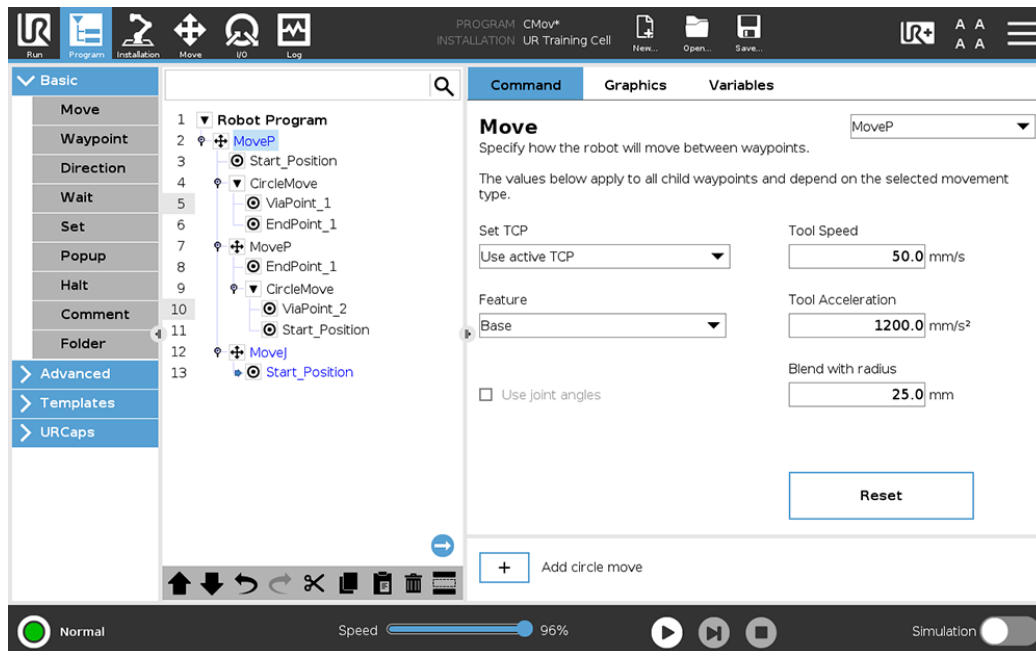


Figure 6.12 – CMov Program 3

14. Save program.

15. Run program.

Hands-On Activity 5: Optimizing Motion Speed

1. Speed and Acceleration of a Move command.

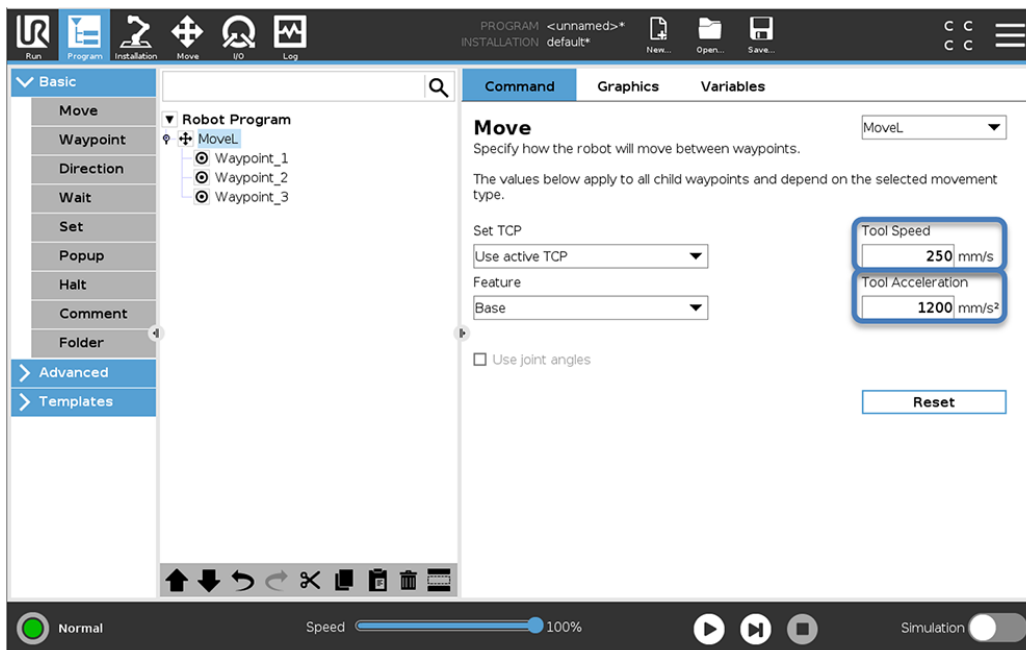


Figure 6.13 – Joint Move Settings

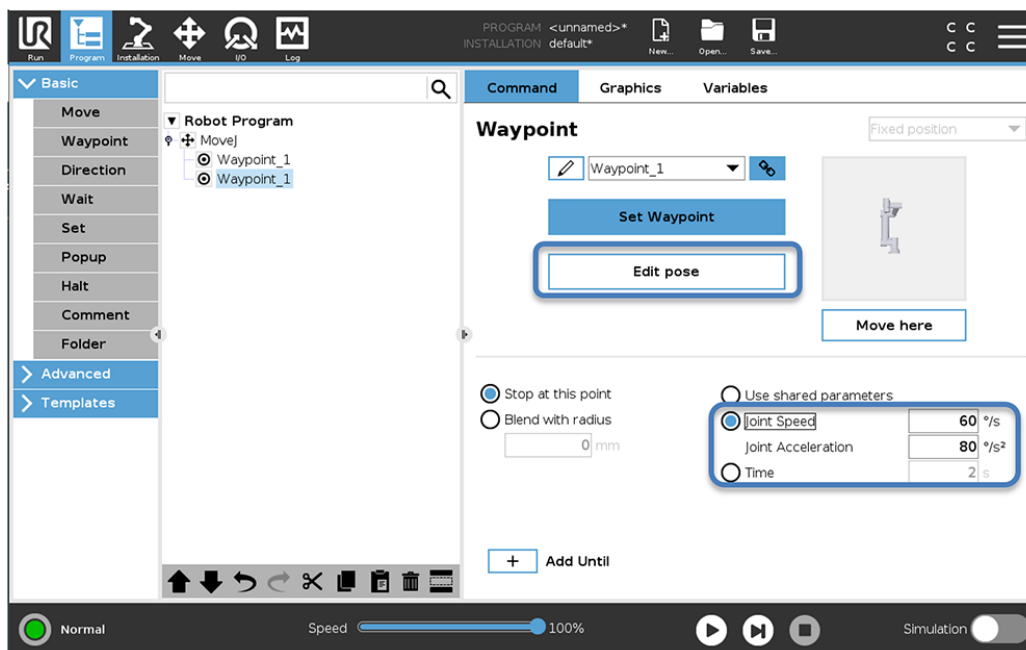


Figure 6.14 – Linear Move Settings

2. Speed, Acceleration and Time of a single point.

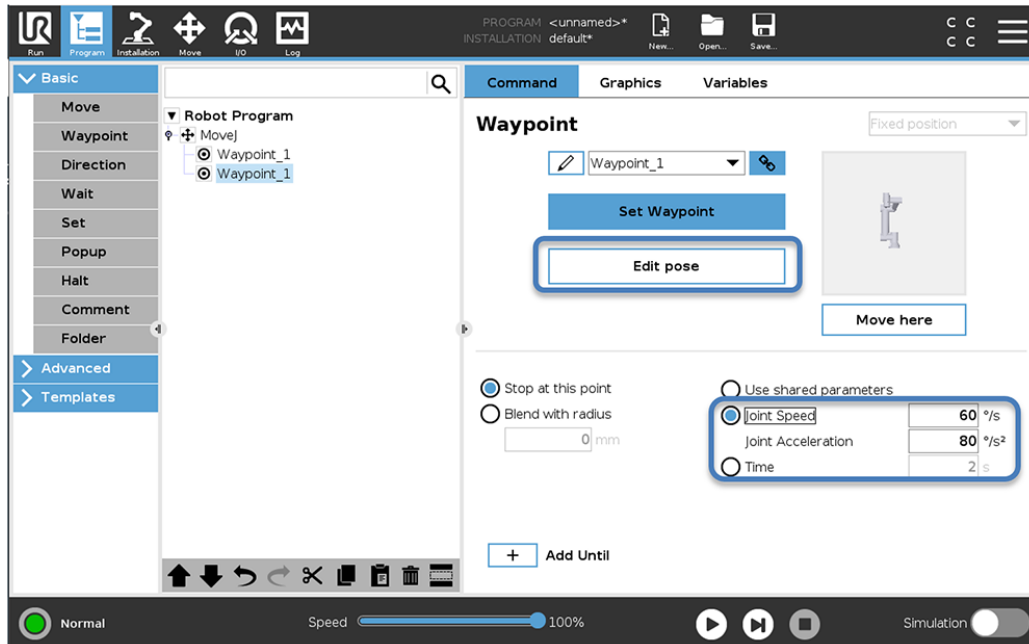


Figure 6.15 – Optimizing a Single Point

3. Open JTrace and insert the **Timer** commands.

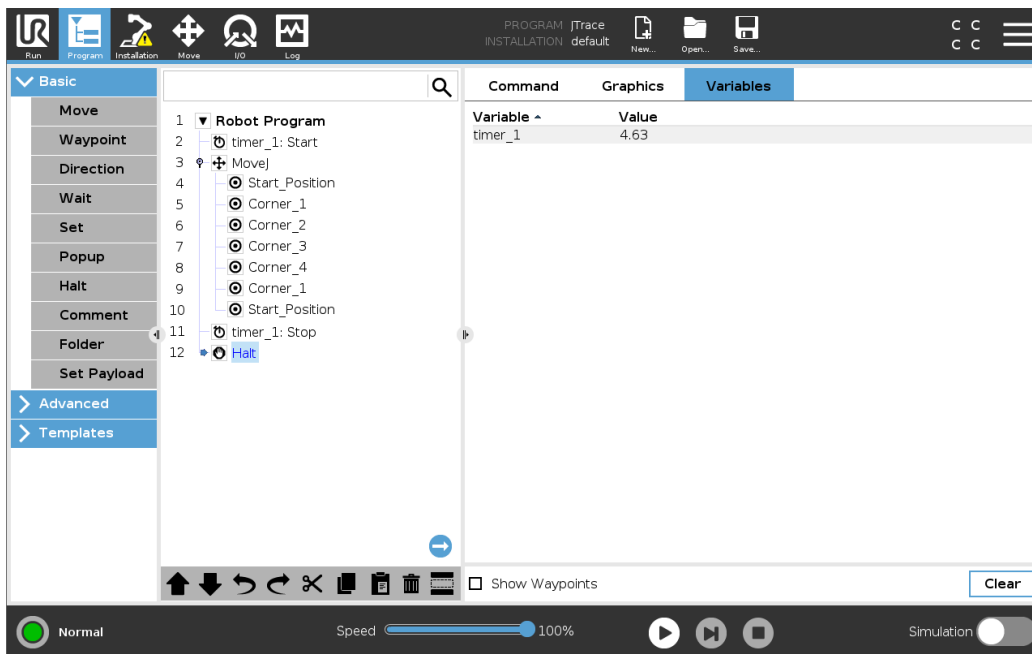


Figure 6.16 – JTrace Timer Code

- Run the program once and write down the initial cycle time.
- Blend the four Corners with a 25 mm radius and run again.
- Compare the two cycle times.
- How much did the cycle time improve?
- Click on the **variable** tab to see the results in **timer_1**.

4. In **JTrace** increase the joint speed on the **MoveJ** (default is 60 °/s).
 - Change to 120 °/s and run the program.
 - Compare this cycle time with your previous cycle time. Is your speed limited by your safety settings?
5. In **JTrace** increase the joint acceleration on the **MoveJ** (default is 80 °/s²).
 - Change to 150 °/s² and run the program.
 - Compare the two cycle times. Is your speed limited by your safety settings?
 - Did velocity or acceleration have a bigger impact?
6. Now do the same with **LTrace**. Open **LTrace** and insert the **Timer commands**.

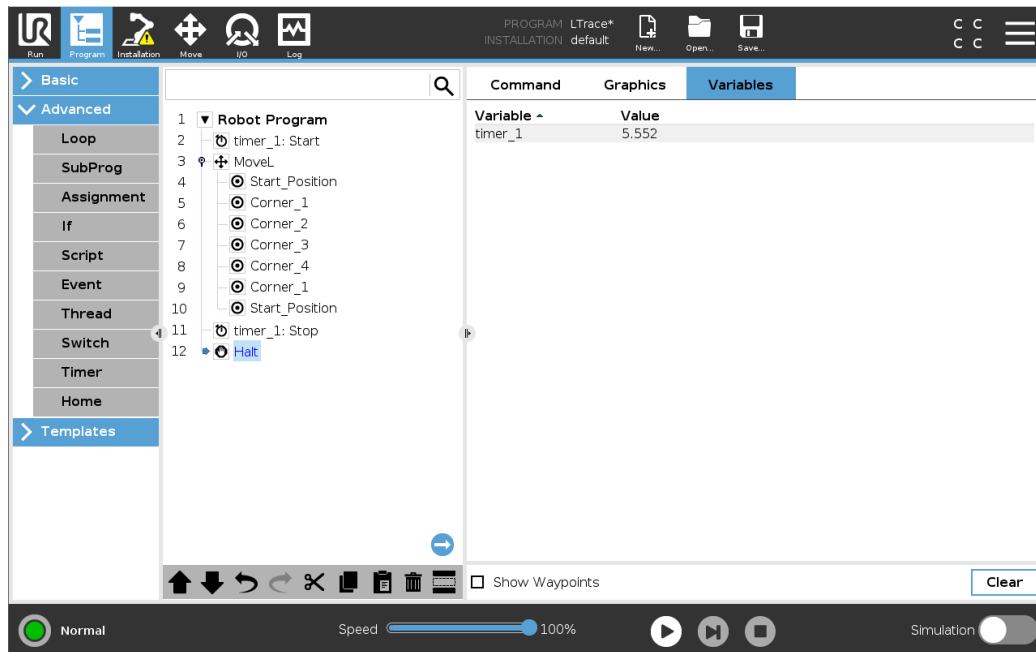


Figure 6.17 – LTrace Timer Code

- Run the program once and write down the initial cycle time.
 - Blend the four Corners with a 25 mm radius and run again.
 - Compare the two cycle times.
 - How much did the cycle time improve?
7. In **LTrace** increase the tool speed on the **MoveL** (default speed is 250 mm/s).
 - Change to 350 mm/s and run the program.
 - Compare this cycle time with your previous cycle time. Is your speed limited by your safety settings?
 8. In **LTrace** increase the tool acceleration on the **MoveL** (default acceleration is 1200 mm/s²).
 - Change to 1500 mm/s² and run the program.
 - Compare the two cycle times. Is your speed limited by your safety settings?
 - Did velocity or acceleration have a bigger impact?

Hands-On Activity 6: Understanding Singularities

1. Outer Workspace Limits.
 - Go to the **Corner 2 Waypoint**.
 - Move robot at 50% speed towards you.

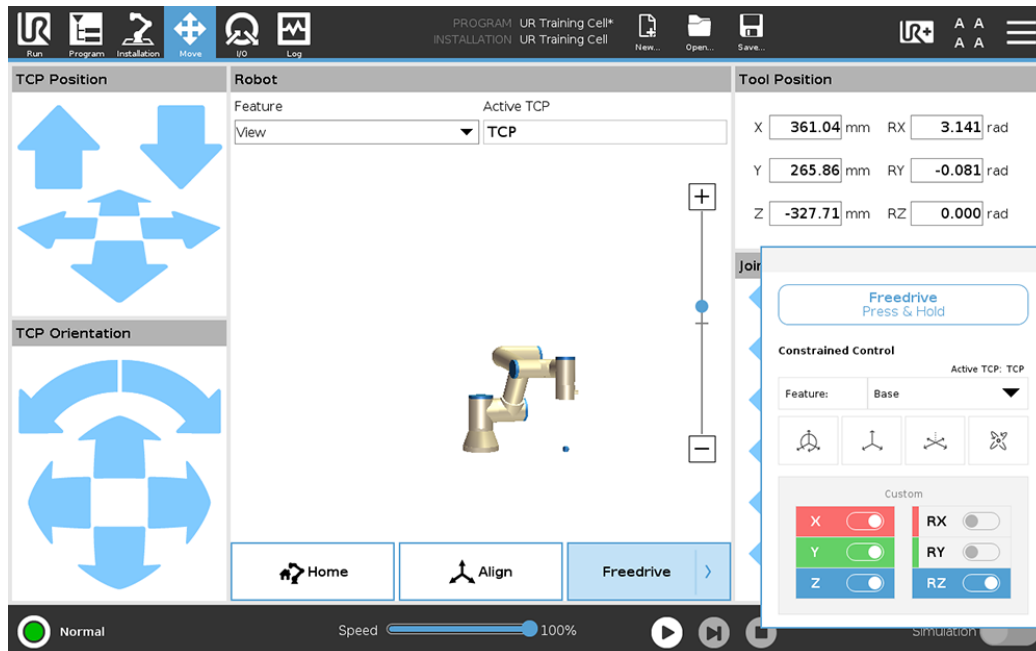


Figure 6.18 – Robot Pose at Outer Limits

- Once it errors use the joint arrows to get the robot back into a usable position.
- **Shoulder Joint** positive.
- **Elbow Joint** negative.
- Return to **Start Position**.

Hands-On Activity 7: Create a Pick-and-Place Program (Waypoints Only)

1. Save a new program file. Name it **PPP1**.
2. When programming the next activity, use the following parameters.
 - Set the **Speed Slider** in the footer at 50%.
 - Use the default speed for the **Joint Moves** of 60 degrees/sec.
 - Set speed for **Linear Moves** at 125 mm/sec.
 - No blending radius in **Waypoints**.
3. Place a block in the nest on the right side of the conveyor.



Figure 6.19 – Pick Point

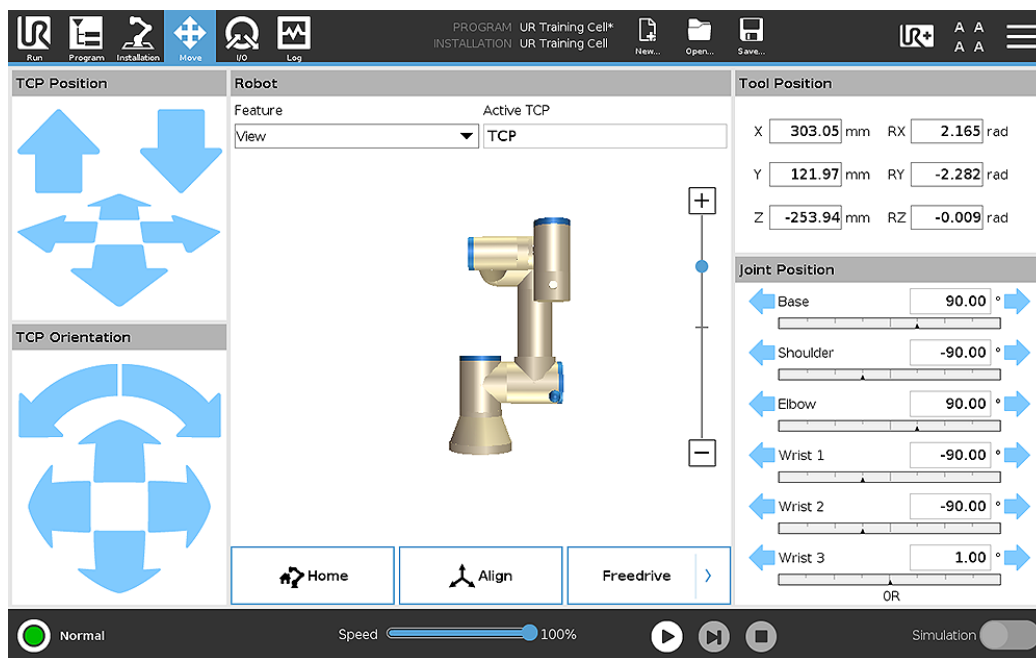


Figure 6.20 – Start Position

- Create a **Waypoint** and name it **Start Position**.
 - Note: The robot must be at this first point to run the program.
4. Activate and Open gripper.
 - This action will vary based on the gripper installed on the robot.
 5. Add a new **Waypoint** halfway to the right end of the conveyor; name it **Transit 1**.
 6. Add a new **Waypoint** about 50 mm above the block in the nest on the conveyor.
 - The gripper should be centered to the block in an orientation to grip.
 - Name it **Approach 1**.
 7. Add a **Gripper Close command** to close the gripper and grab the block.
 8. Add a new **Waypoint** after **Pick 1**.
 - Move the robot vertically 50 mm. Link this **Waypoint** to **Approach 1**.
 - Set **Approach 1** to this new location to refine the approach motion.
 9. Add a new **Waypoint** and link it to **Transit 1**. Move to this position.
 10. Add a **Waypoint** and link to **Start Position**. Move to this position.
 11. Add a **Waypoint** on the way to the left end of the conveyor. Name it **Transit 2**.
 12. Program a place position in the first location on the provided pallet. Name it **Place 1**.

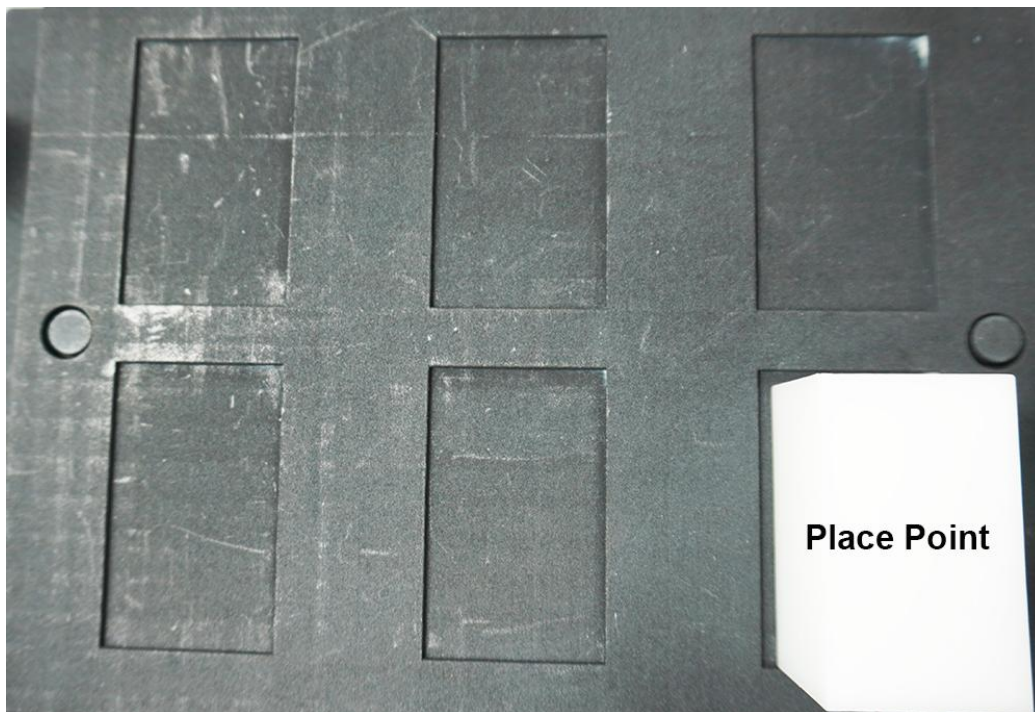


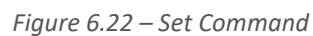
Figure 6.21 – Block at Place Point

13. Add a **Gripper Open command**.
14. Program a retract position by moving the robot vertically for 50 mm. Name it **Approach 2**.
15. Add a new **Waypoint** before the place position. Link this position to **Approach 2**.
16. Add a new **Waypoint** and link it to **Transit 2**. Move to this position.

17. Add a new **Waypoint** and link to **Start Position**. Move to this position.

- Step through the program one line at a time. Press **Play** and then **Pause**. Use **Next** to step through each point.
- Run the program for one cycle with the **Speed Slider** at 50%. Be prepared to hit the **E-Stop** if the robot does not move as desired. Remove the block from the nest and place at the pick position on the conveyor. Run the program for one cycle with the **Speed Slider** at 100%.

1. Set payload at 1.1 kg when picking up a block.



3. Wait for **DigitalIn (0)** to be ON.



4. Run program.
 - To start the program, you must position the **Digital IN (0)** switch on the **IO Simulation Test Box** to ON.
5. Save program.

(Optional) Hands-On Activity 9: Tracing Plates Exercise

1. For additional practice, program the robot using what was learned in this module to trace the two provided plates that are double-sided.
 - You can use the double pointer attachment that is provided. Another option is to take one pointer from the double pointer fixture and place it between the gripper fingers as shown in Figure 6.24.



Figure 6.24 – Bonus Tracing Exercises

Module 7: Pick and Place

Hands-On Activities (UR Academy Student Module)

Source: Universal Robots Academy, Student Module 7 “Pick and Place”. Activities and figures reproduced in original order; figure numbers follow the source.

Hands-On Activity 1: Inputs and Outputs (I/O)

1. Review **I/O Test Box**.



Figure 7.1 – I/O Test Box

2. Press the **I/O icon** located in the **Header** to show the **I/O screen**.

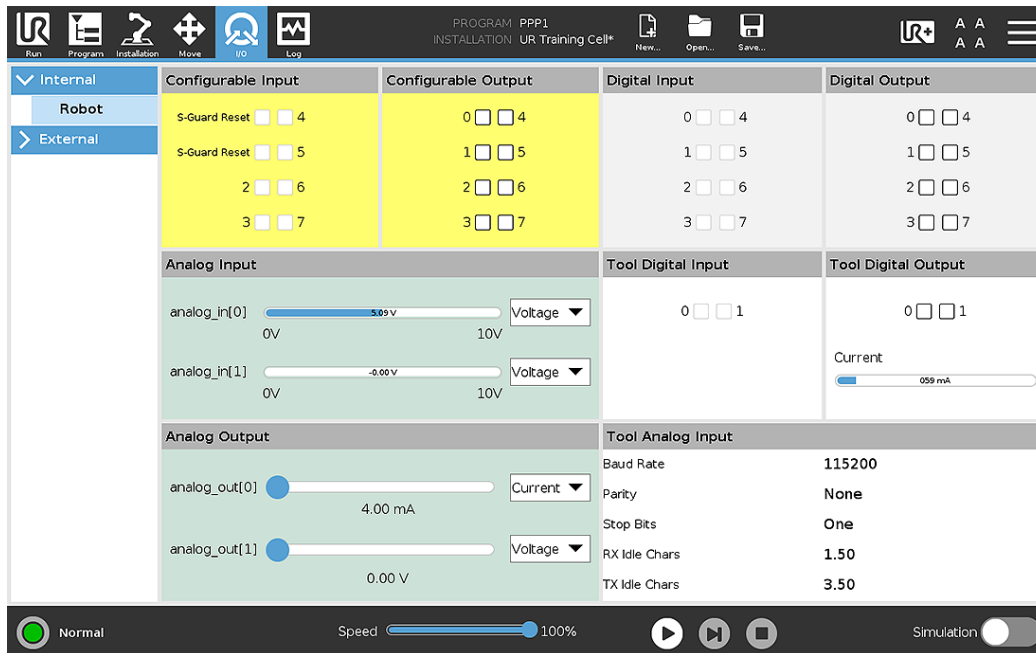


Figure 7.2 – I/O Screen

3. Review I/O for all components in the system.

- Toggle the **Digital IN (0)** and **IN (1)** switches located in the lower left corner of the **I/O Test Box**. Watch the **Digital Inputs** change state on the pendant.
- Place a block in front of the conveyor's sensors and watch the **Digital Inputs** change state on the pendant.
- **Digital Inputs**
 - **DI (0)** – Toggle Switch 1 on **I/O Test Box**
 - **DI (1)** – Toggle Switch 2 on **I/O Test Box**
 - **DI (2)** – Sensor on **Conveyor**
 - **DI (3)** – Sensor on **Conveyor**
- Press the buttons located in the **I/O Screen** on the pendant. Watch the LED lights **Digital OUT (0)** and **OUT (1)** change state on the **I/O Test Box**.
- **Digital Outputs**
 - **DO (0)** – Check LED status on **I/O Test Box**.
 - **DO (1)** – Check LED status on **I/O Test Box**.

4. Setup I/O

- Press the **Installation icon** in the **Header** and select the **I/O Setup Tab**.
- Rename the **Digital Inputs** and **Digital Outputs** identified in the previous section.

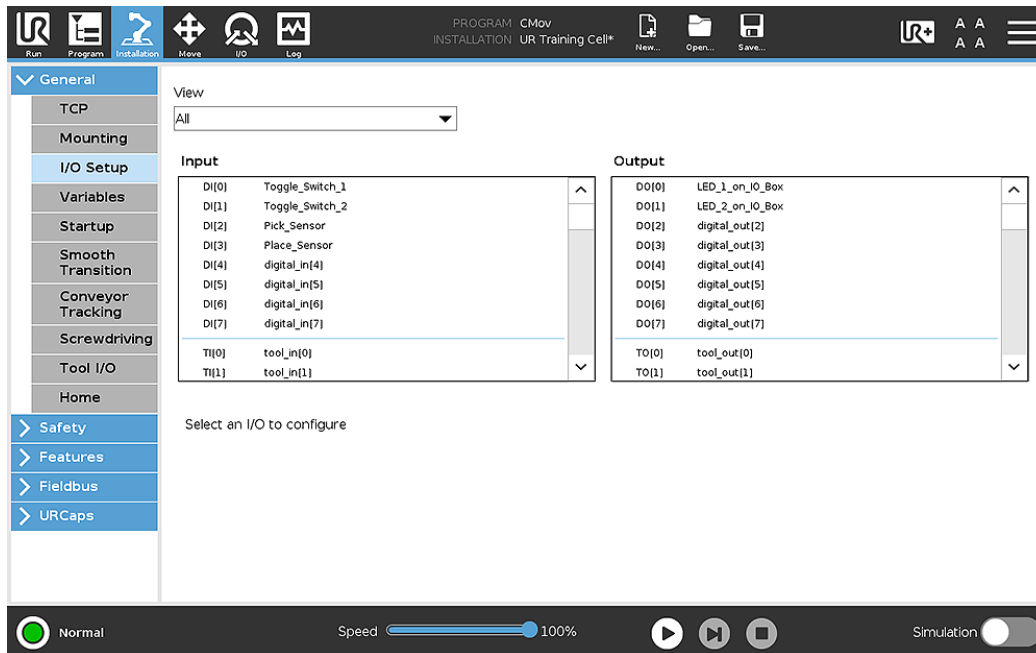


Figure 7.3 – I/O Setup

5. Save the **Installation File** named **UR Training Cell** used in Module 6.

Hands-On Activity 2: Create a Pick-and-Place Program

1. Save a new program file. Name your program.
 - As an example: **Mod7_PickAndPlace**
2. When programming the next activity, use the following parameters.
 - Set the **Speed Slider** in the footer at 50%.
 - Use the default speed for the **Joint Moves** of 60 degrees/sec.
 - Set speed for **Linear Moves** at 125 mm/sec.
 - No blending radius in **Waypoints**.
3. Place a block in the nest on the right side of the conveyor.



Figure 7.4 – Pick Location

4. Create a **Waypoint** above the **Conveyor** and name it **Transit**.

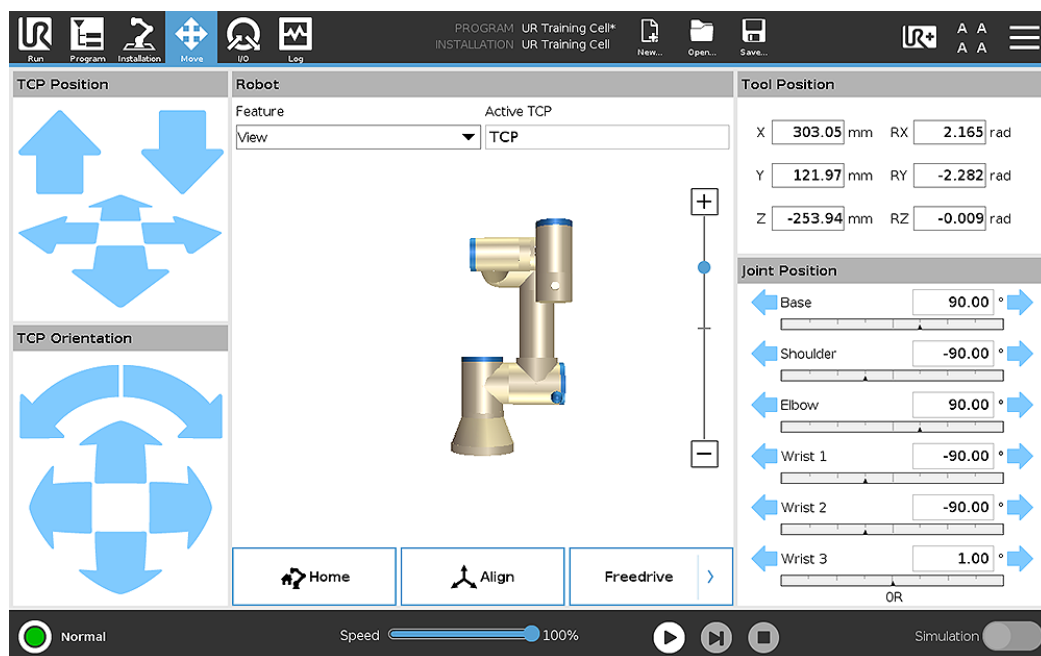


Figure 7.5 – Initial Position/Transit Location

5. Activate the gripper and use a command to open its fingers.
 - Note: Necessary steps will vary based on gripper/equipment.
6. Move the robot to a position above the block. Add a **Waypoint** and name it **Pick_App**.
 - The gripper should be centered to the block and have an orientation to grip.
7. Move the robot down to the block where the gripper can grab it. Add a **Waypoint** and name it **Pick_Pos**.
8. Add a command to close the gripper and grab the block.

- Depending on the equipment we may need to add a **Wait Command** for the gripper to close.
9. Add a **Waypoint** after picking up the block. From here either:
 - Move the robot up and set this as a new **Waypoint** (labelled **Pick_Exit**), or
 - Link this new waypoint to **Pick_App**.
 10. Add a **Waypoint** and link it to **Transit**.
 11. Move the robot to a position above one of the spots on the provided tray (as seen in Figure 7.6).
 12. Add a **Waypoint** at this point and name it **Place_App**.
 13. Move the robot down where the block touches the tray. Add a **Waypoint** and name this **Place_Pos**.

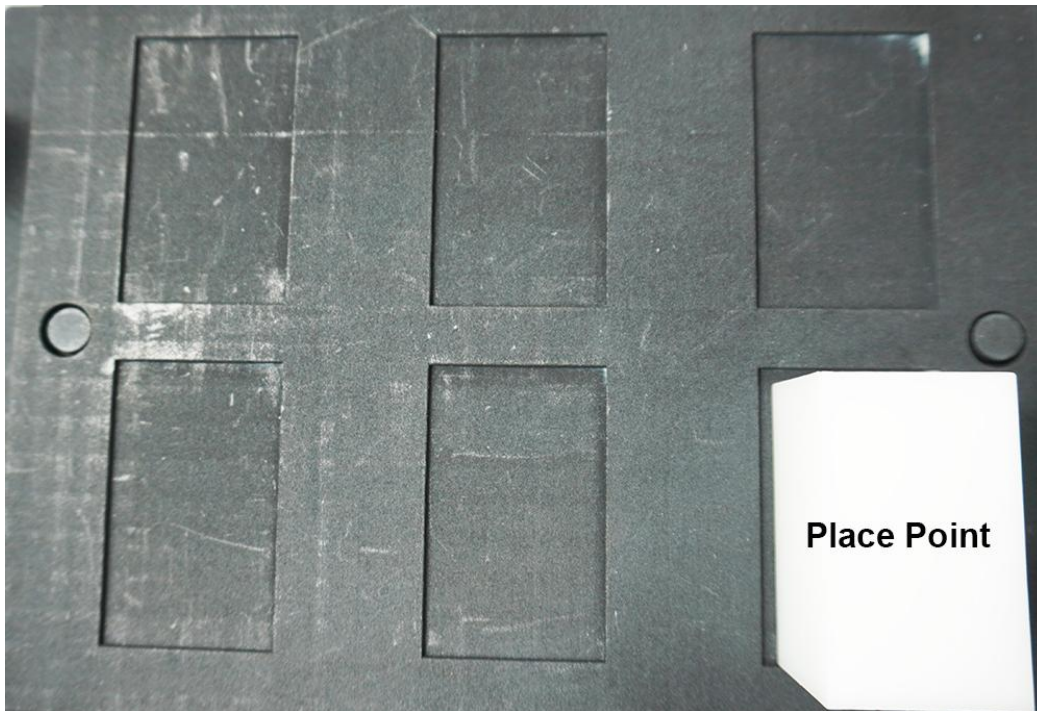


Figure 7.6 – Place Location

14. Add a command to open the gripper and release the block.
15. Add a **Waypoint** after dropping the block. From here either:
 - Move the robot up and set this as a new **Waypoint** (labelled **Place_Exit**), or
 - Link this new waypoint to **Place_App**.
16. Run the program.

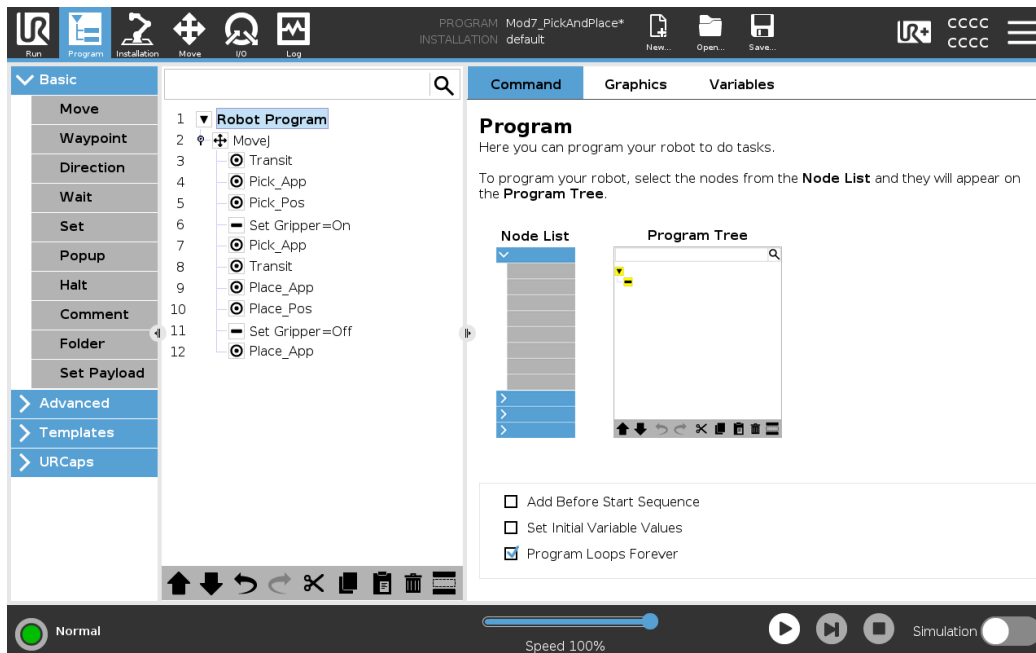


Figure 7.7 – Example of Pick-and-Place Program

Hands-On Activity 3: Improvements to Pick-and-Place

1. Add a **Set Payload Command** after picking up the block. Change the payload to equal the weight of the **Gripper** and the **Part**.

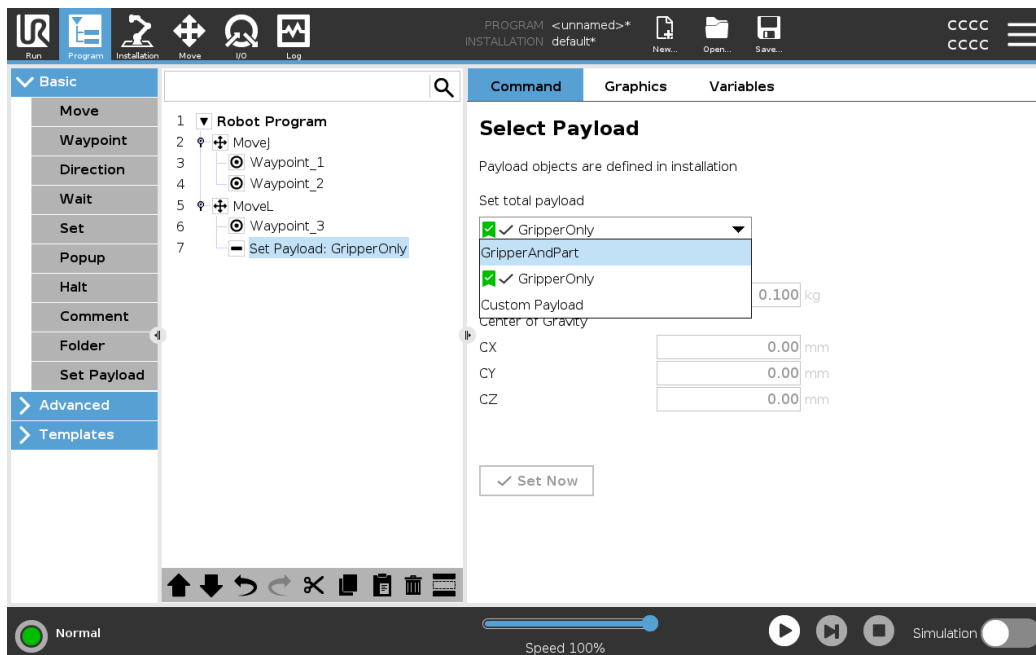


Figure 7.8 – Set Payload Command

2. Add a **Set Payload Command** after releasing the block. Change the payload to the weight of the **Gripper**.

3. Add a **Wait Command** after the robot is at **Pick_App**. Set it to wait until the **Conveyor Sensor** is High.

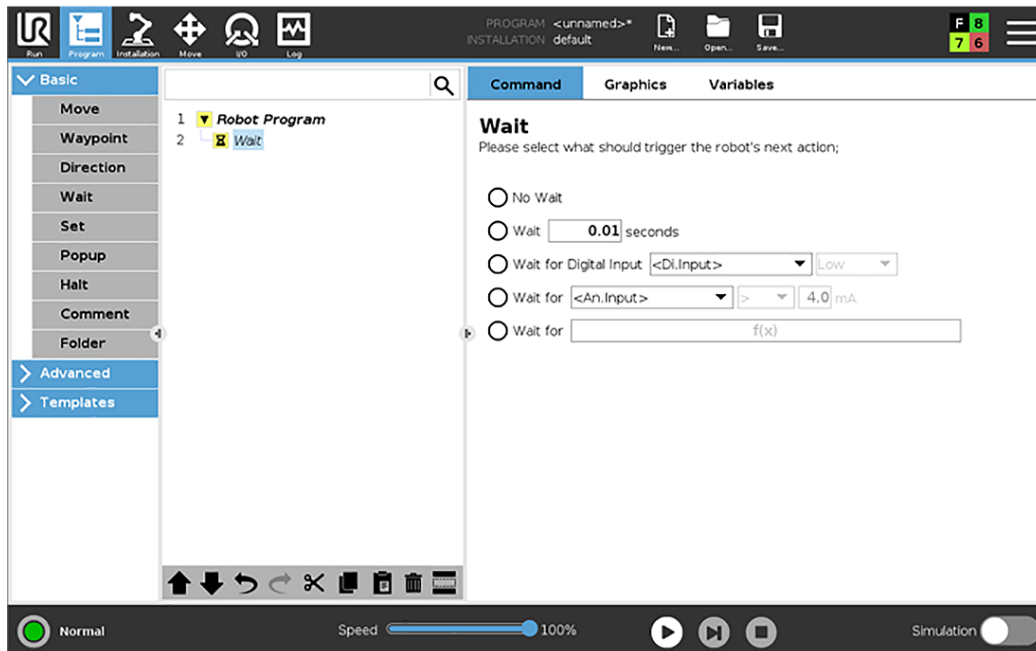


Figure 7.9 – Wait Command

4. Add a **Set Command** at the beginning of the program. Set it to turn the **Conveyor On/High**.

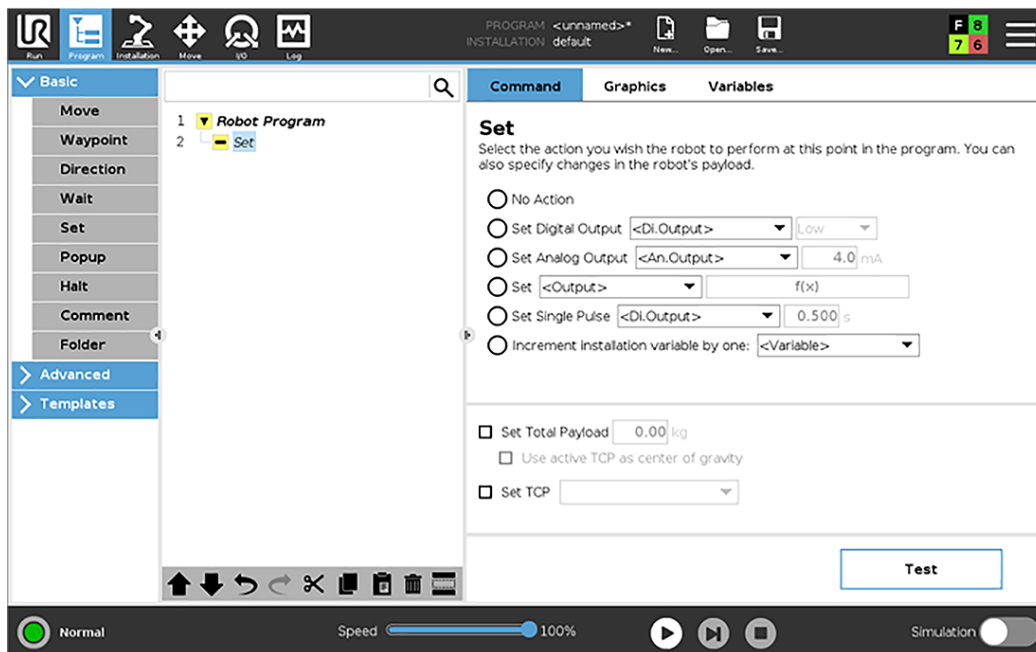


Figure 7.10 – Set Command

5. Run the program.

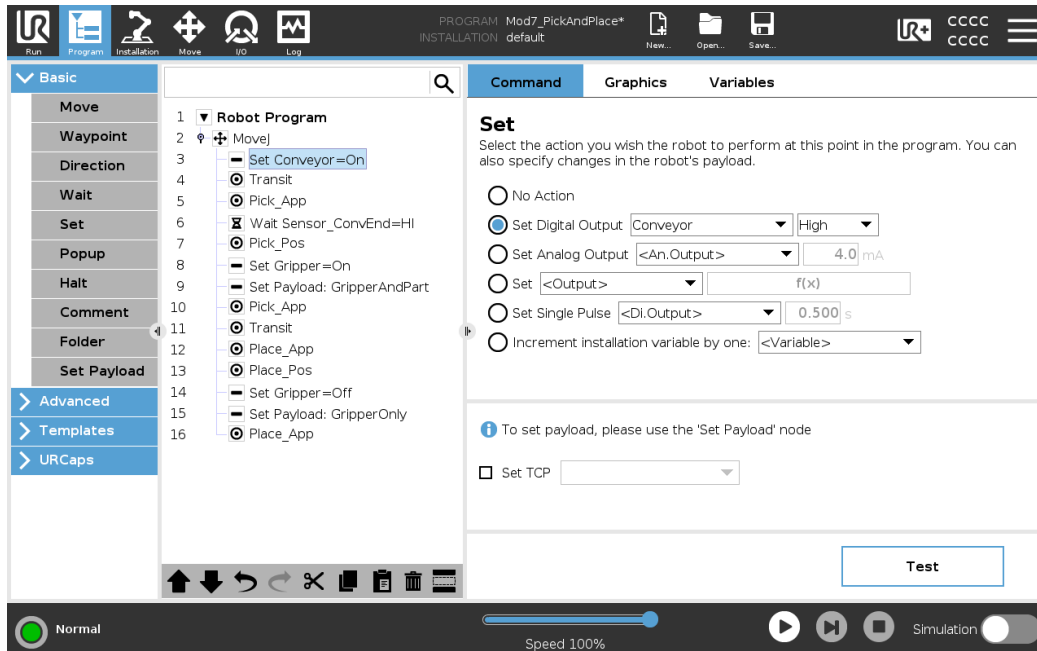


Figure 7.11 – Updated Example of Pick-and-Place Program

6. Optimize the program by moving the **Pick/Place** and **Exit Locations** into **MoveL Commands**.
 - If you used **Linked Waypoints**, **Exit Locations** would be the second approach points after the **Pick/Place Locations**.
7. Run the program.

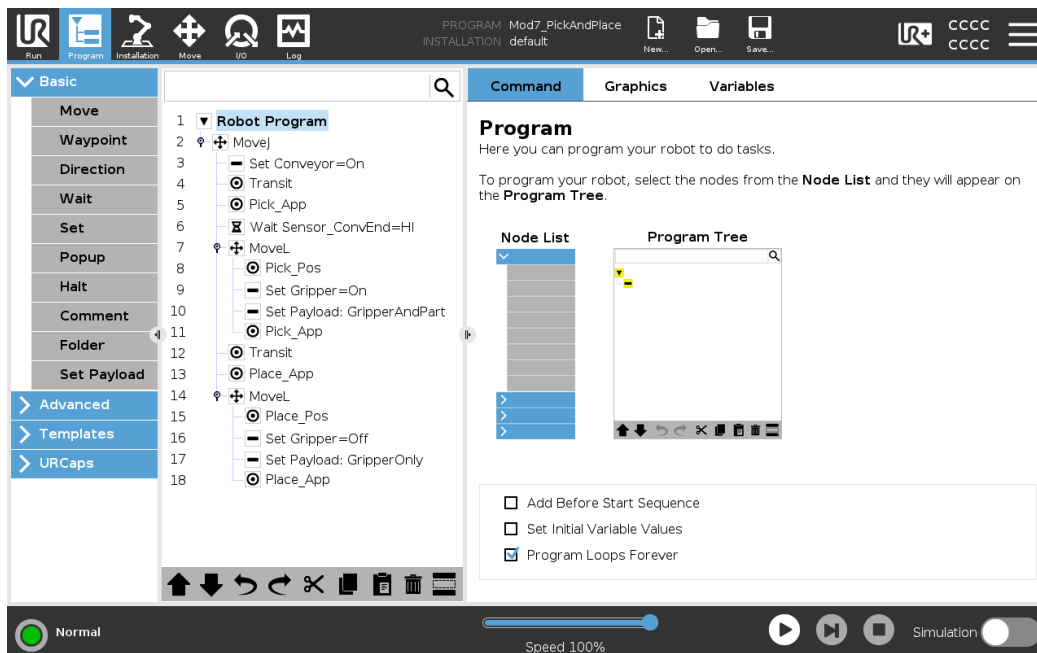


Figure 7.12 – Optimized Example of Pick-and-Place Program

8. Save the program; either as a new program or overwrite your existing file.

Module 8: Programming

Hands-On Activities (UR Academy Student Module)

Source: Universal Robots Academy, Student Module 8 “Programming”. Activities and figures reproduced in original order; figure numbers follow the source.

Hands-On Activity 1: Using Variables

1. Add a counter to the program.
2. Open program **PPP1** and Save As **PPP2**.
 - Delete the **Wait** command for the **Toggle Switch**.

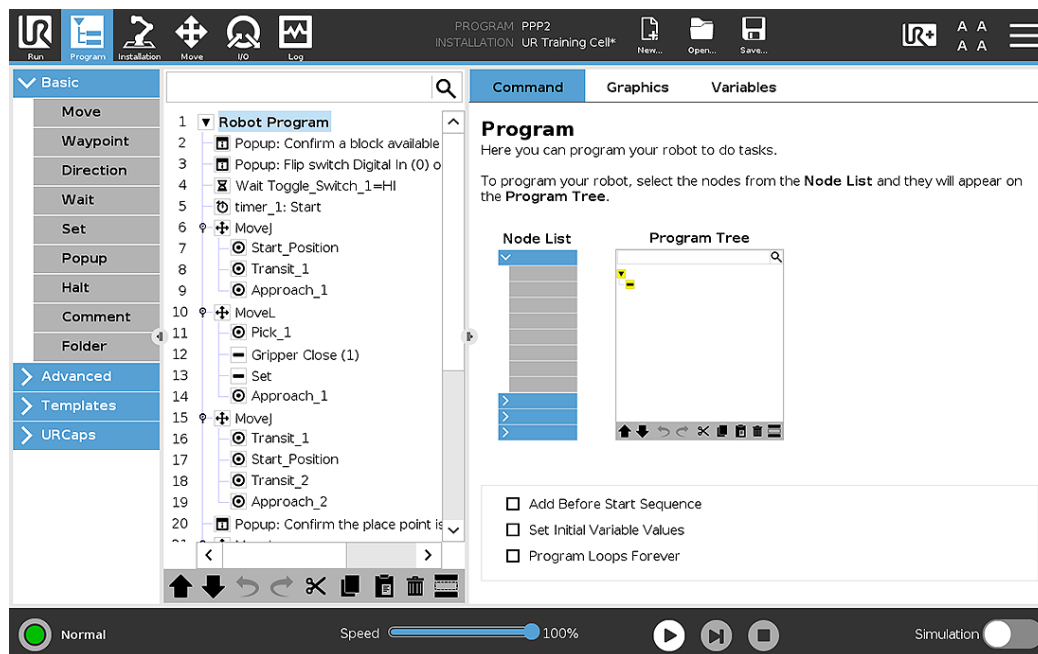


Figure 8.1 – PPP2 Program Edits

- Insert and Modify a **Variable**.

Step	Action						
1.	In the Node list, click the Advanced tab and then the Assignment button to enter a new variable into the Program Tree  var_1 := ...						
2.	In the Command tab <table border="1"> <thead> <tr> <th>If</th><th>Then</th></tr> </thead> <tbody> <tr> <td>You are creating a new variable</td><td>Rename the variable to an appropriate name</td></tr> <tr> <td>You are assigning a new value to an existing variable</td><td> Select the relevant variable in the dropdown menu Variable  var_1 ▼ Create New var_1 </td></tr> </tbody> </table>	If	Then	You are creating a new variable	Rename the variable to an appropriate name	You are assigning a new value to an existing variable	Select the relevant variable in the dropdown menu Variable  var_1 ▼ Create New var_1
If	Then						
You are creating a new variable	Rename the variable to an appropriate name						
You are assigning a new value to an existing variable	Select the relevant variable in the dropdown menu Variable  var_1 ▼ Create New var_1						
3.	Type in the value or expression you want to assign to the variable <table border="1"> <thead> <tr> <th>Variable</th><th>Expression</th></tr> </thead> <tbody> <tr> <td>  var_1 ▼ </td><td> := 10 </td></tr> </tbody> </table>	Variable	Expression	 var_1 ▼	:= 10		
Variable	Expression						
 var_1 ▼	:= 10						

Figure 8.2 – Variable Instructions

- At the end of the program insert a variable by going to the **Advanced** tab and then **Assignment** button.
- Rename **var_1** to **Counter**.
- Click in the **Expression Field** to pull up the **Expression Editor**. Under the **Variable drop-down menu**, select **Counter**. Then add +1 so that the expression reads **Counter + 1**. Click **Submit** to complete the assignment.

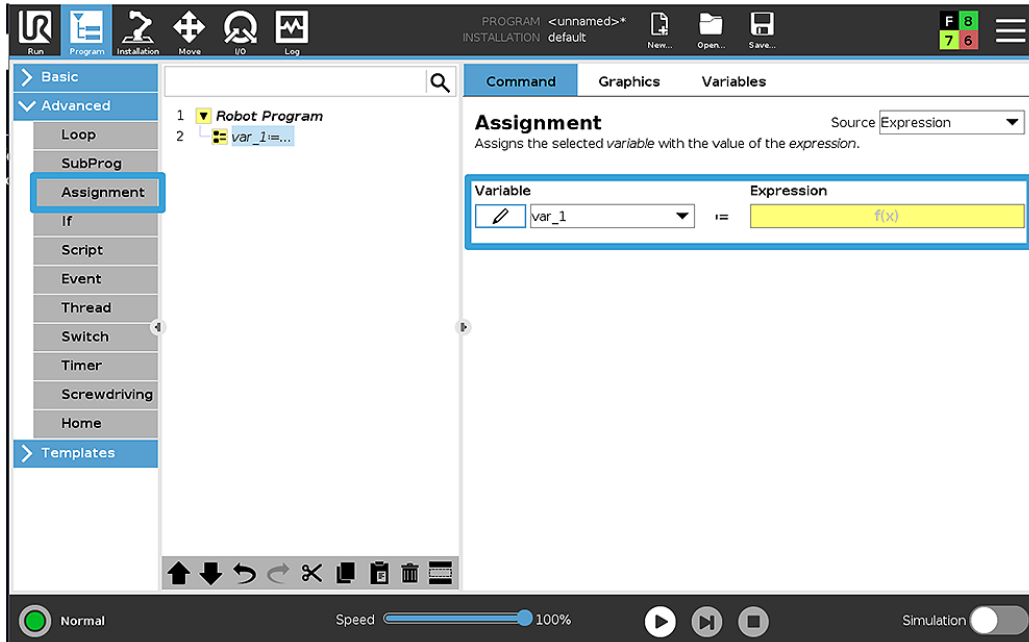


Figure 8.3 – Assigning a Variable

6. Initialize Counter.

- Press the **Robot Program** node at the top of the **Program Tree**.
- From the **Command** tab check the **Set Initial Variable Values** checkbox.
- Select **Counter** from the **Variable** pull-down menu, check the **Keep value from previous run** checkbox, and enter a value of **"1"** for the expression.

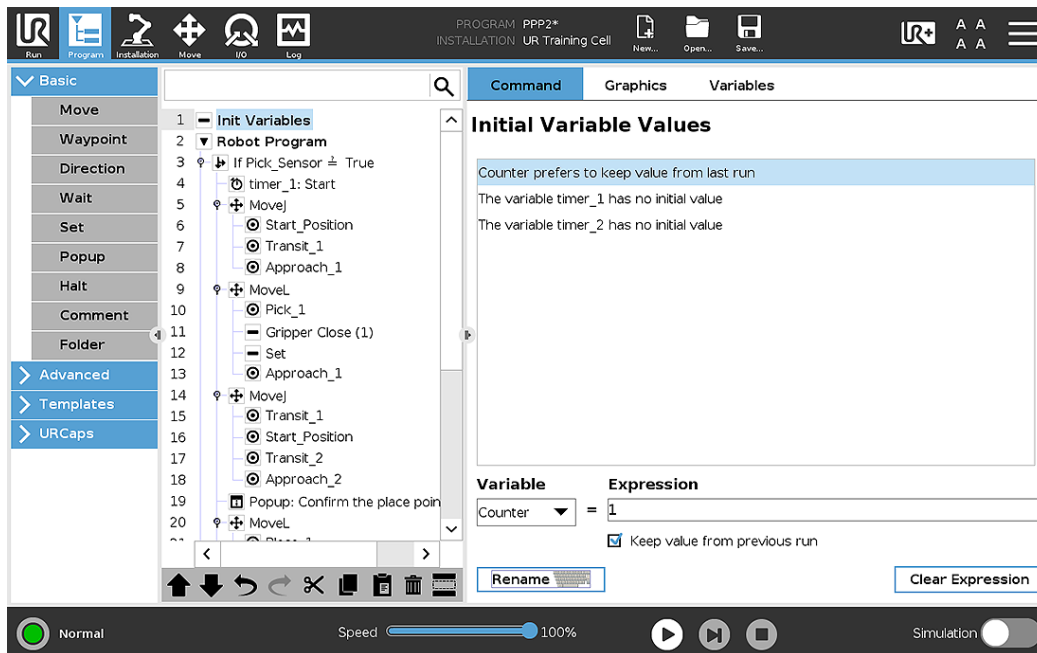


Figure 8.4 – Initializing Counter Variable

Hands-On Activity 2: Using If/Else Logic

1. Add logic to check if a block is present in the conveyor nest for the robot to pick.

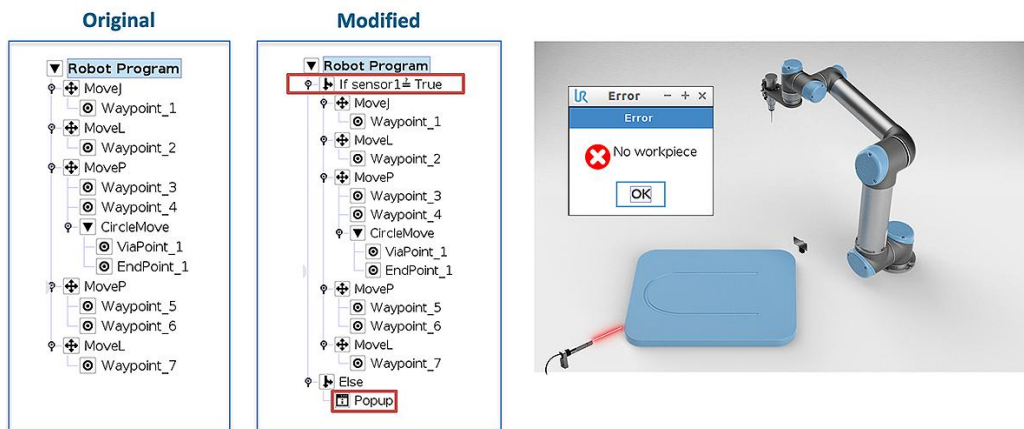


Figure 8.5 – Program Logic Example

2. Insert an **If command** at the beginning of the program.
 - Insert **If**.
 - **Pick Sensor = True**.
 - Use cut and paste to relocate the existing move nodes into the **If statement**.

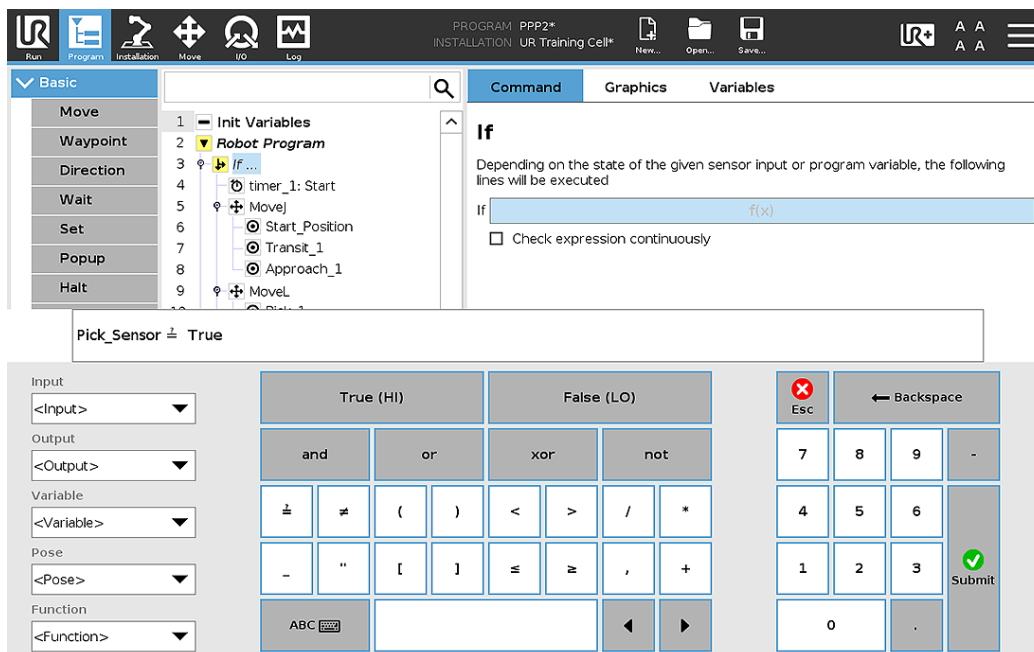


Figure 8.6 – If Pick Sensor Equals True

- Insert an **Else command** at the end of the program.
- Insert **Else**.
- Insert a **Popup** with a message **“No block at pick point.”**

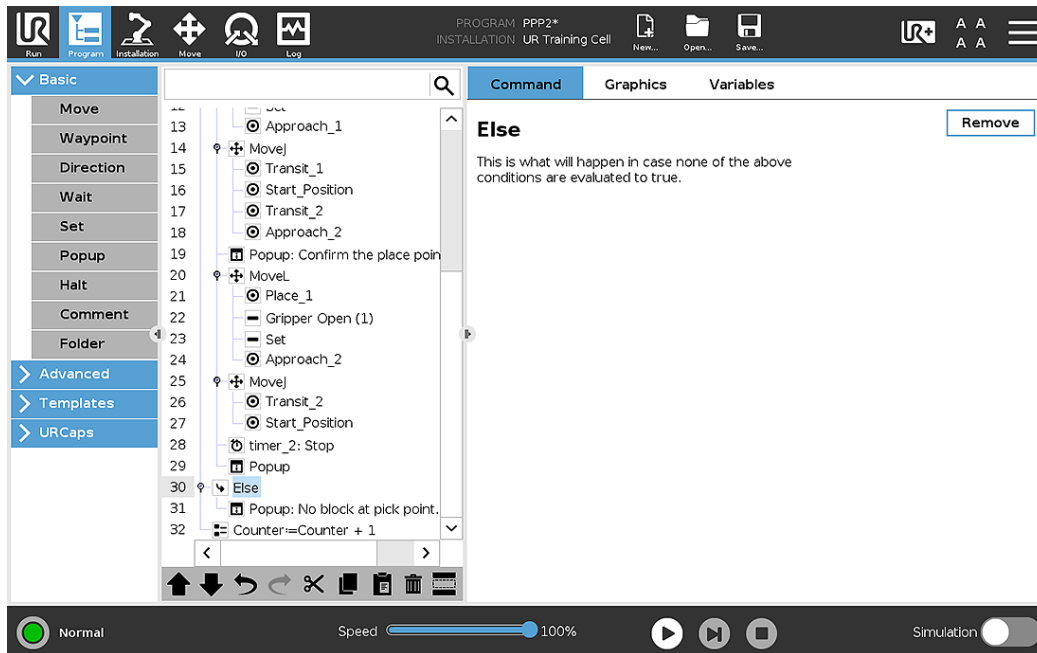


Figure 8.7 – No block at pick point

3. Run program.
4. Press the **Variables** tab to see the values of the variables during operation.
5. Save program.

Hands-On Activity 3: Using Switch/Case Logic

1. Prompt the operator to select from three possible place points.
2. Create a **Folder** after the **Popup** confirming the **Pick Point** is clear.

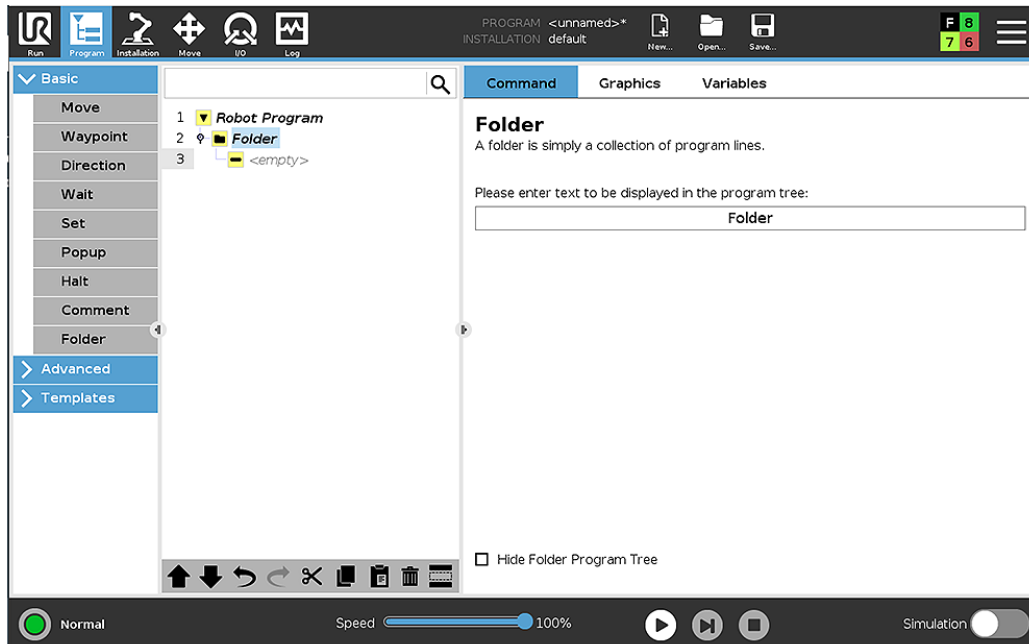


Figure 8.8 – Creating a Folder Example

- The new **Folder** will contain items associated with the place point section of the robot program.
- 3. Rename the **Folder** “**Select Place Point**”.
- 4. Assignments by the operator.

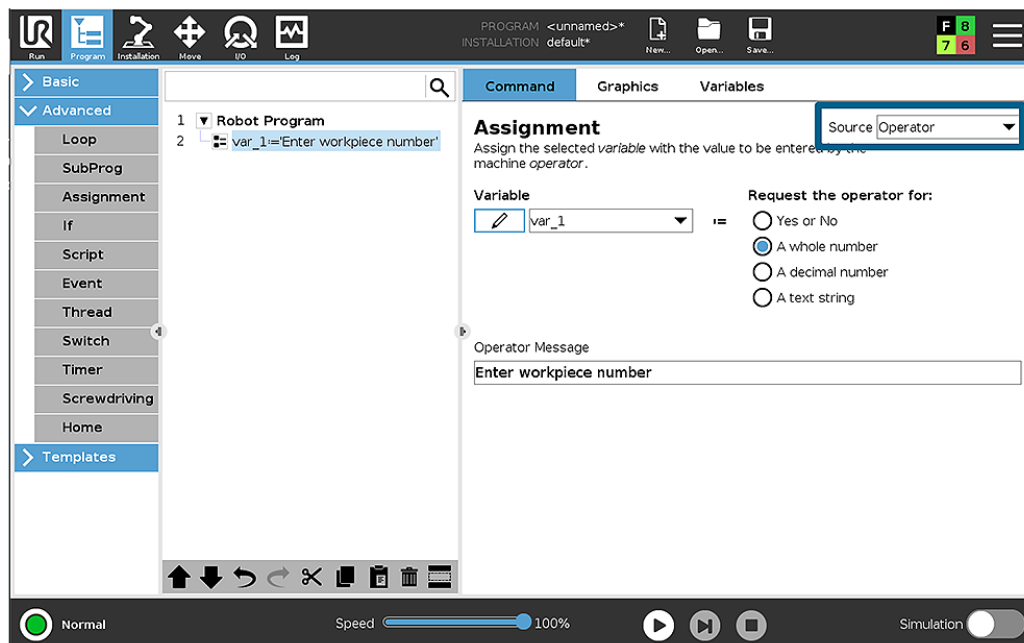


Figure 8.9 – Variable Assignment by Operator

- 5. Press the **Assignment** command to insert a new variable.
- 6. Rename the variable **Place Point**.

7. From the **Source** pull-down menu select **Operator**.
8. Check the appropriate circle to configure the request the operator for: **A whole number**.
9. Configure the **Operator Message** field to display “**Select Place Position: 1 or 2 or 3.**”

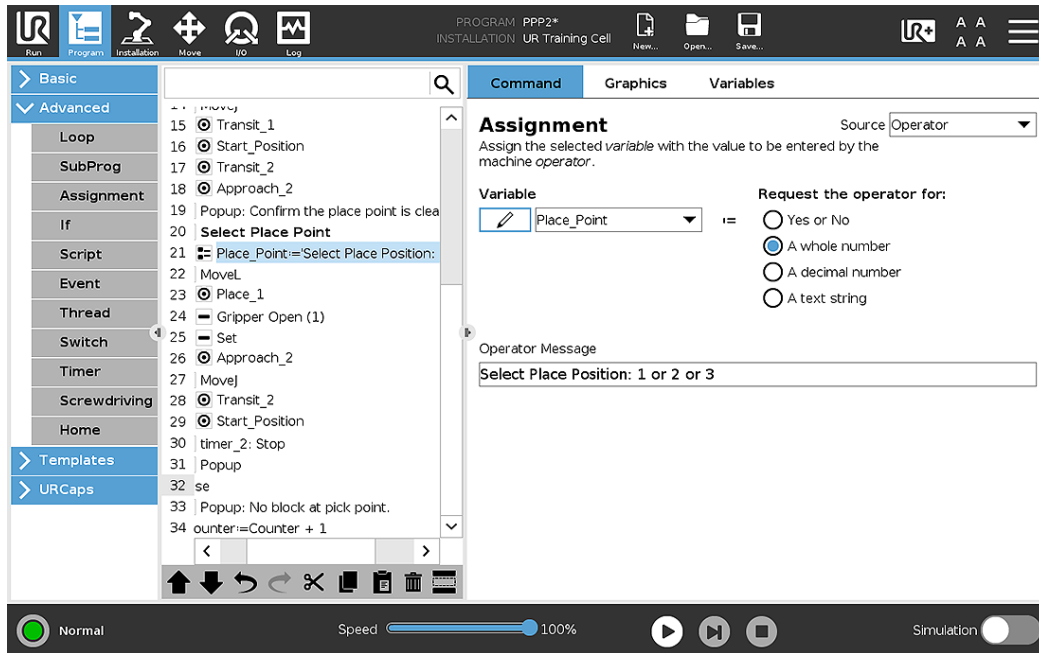


Figure 8.10 – Place Position Program

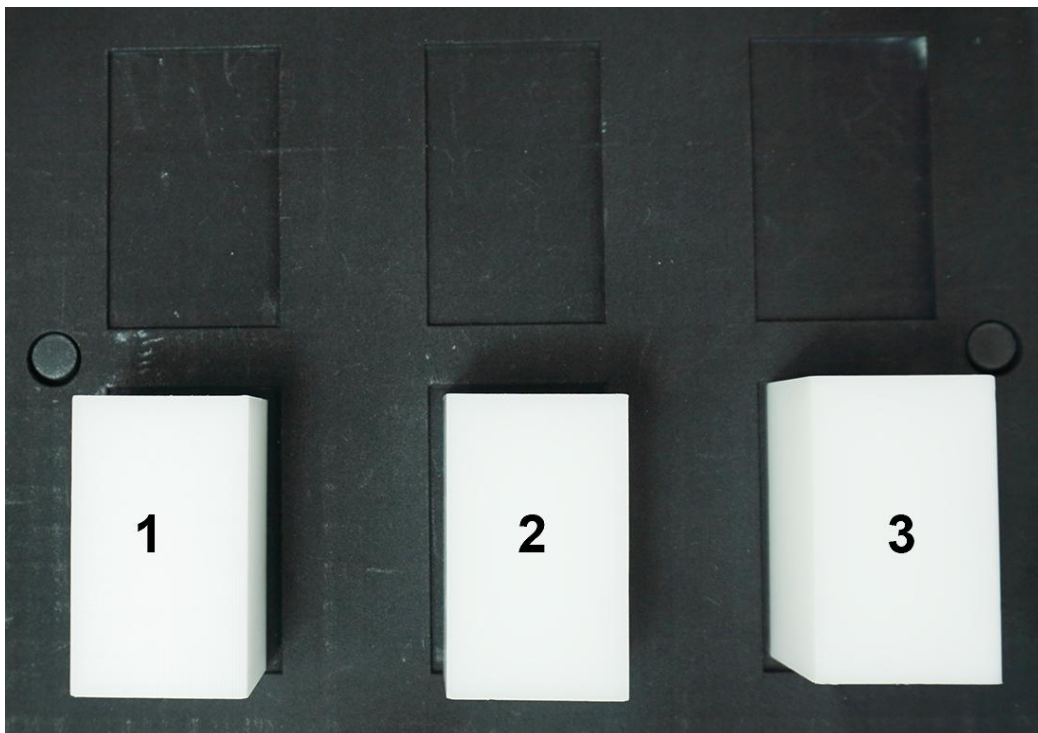


Figure 8.11 – Place Position Locations

10. **Switch Case:** Add a **Switch** to place the product in different locations based on operator input:

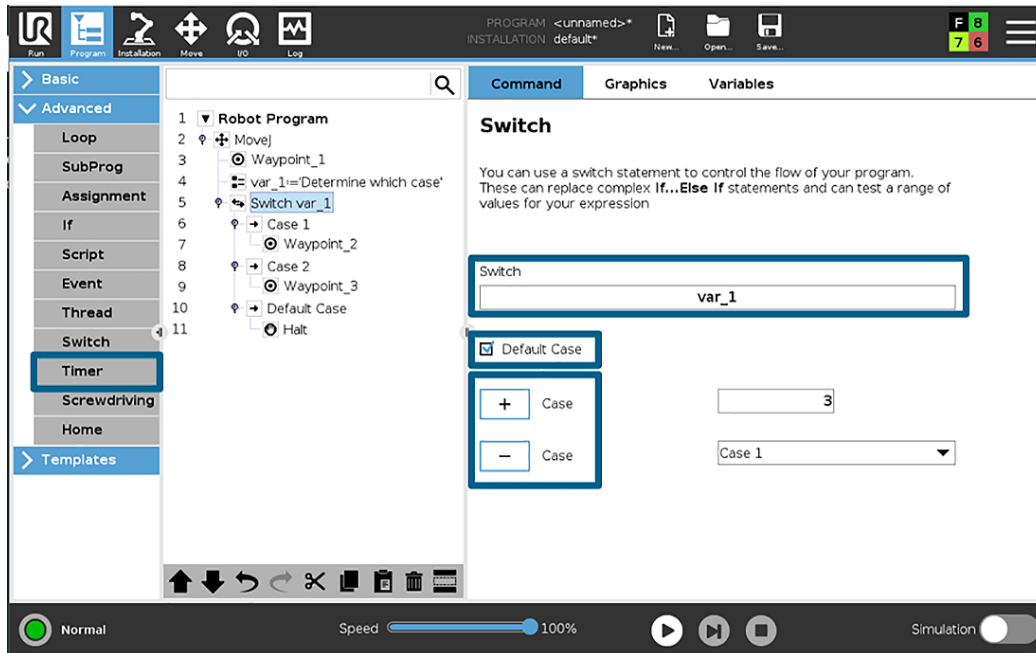


Figure 8.12 – Switch/Case Command Logic Example

11. Select the **Place_Point** Variable to **Switch**.
12. For **Case_1**, cut and paste the **MoveJ** command using the **Place_1** Waypoint. Current **Waypoint Place_1** is used for the first position. Add the additional **Waypoints** and **Gripper** command.

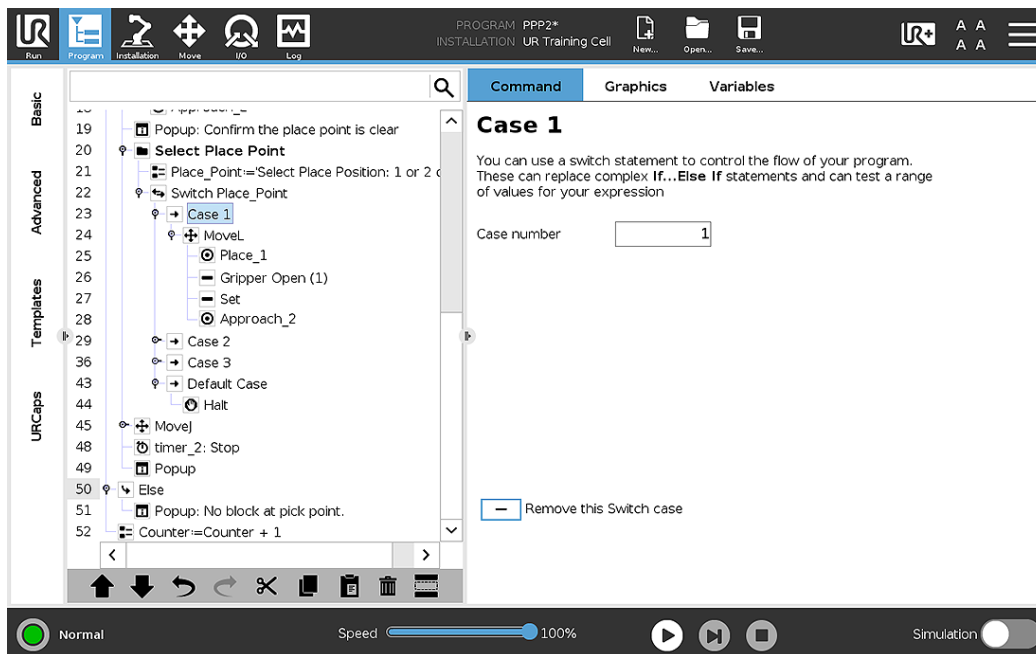


Figure 8.13 – Case Action Detail

13. For **Case_2** copy the **MoveJ** command using the **Place_1** Waypoint. Replace **Place_1** with **Waypoint Place_2** located left of the **Place_1** on the block pallet.
14. Add a new **Waypoint** above **Place_2** and name it **Approach_3**. It should be 50 mm above **Place_2**.

15. For **Case_3** copy the **MoveJ** command using the **Place_2 Waypoint**. Replace **Place_2** with **Waypoint Place_3** located left of **Place_2** on the block pallet.
16. Add a new **Waypoint** above **Place_3** and name it **Approach_4**. It should be 50 mm above **Place_3**.
17. Check the **Default Case** and add a **Halt** command.

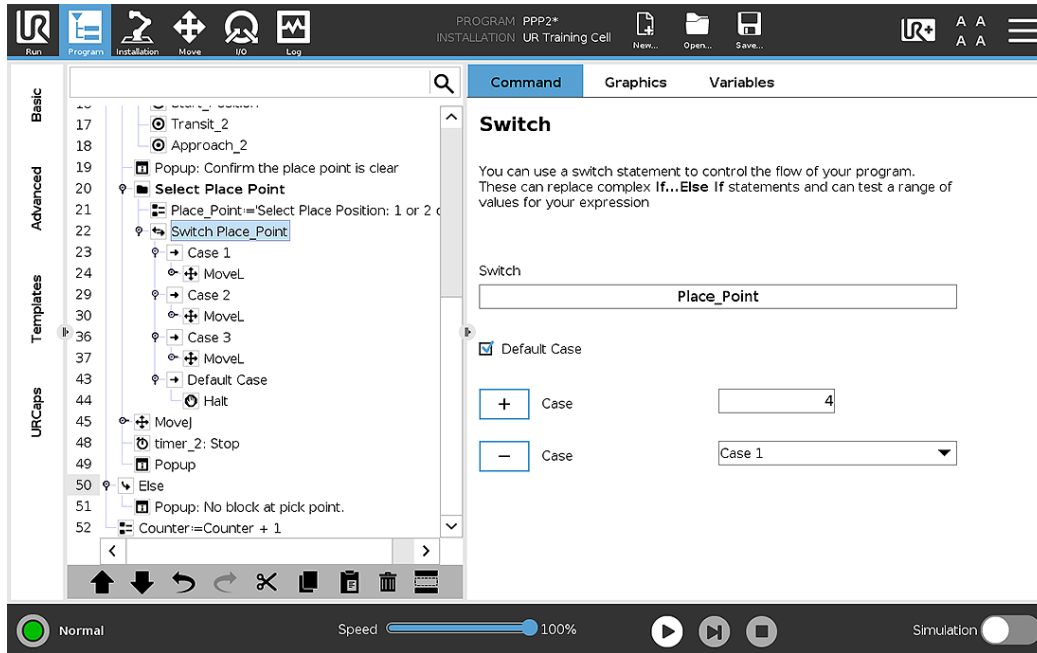


Figure 8.14 – Switch/Case Program

18. Run program three times selecting a different place position each time.
19. Make sure each place option works properly.
20. Save ALL Programs.

Hands-On Activity 4: Using a Sub-Program

1. Create a program to inspect a part with a frequency selected by the operator.
2. Open **PPP1** and Save As **Inspect1**.
3. Delete **Popup flip switch**.
4. Delete **Wait Toggle**.
5. Delete **Timer** start and stop.
6. Delete timer popup.
7. Delete **Move until**.
8. Create **Counter_1 Variable**. Set to a value of 1 in initialization. Increase by a value of 1 each time the robot reaches the end of the program.
9. Create **Counter_2 Variable**. Set to 1 in initialization. Increase by 1 at the end of the program. See above.
10. Before starting program create a **Variable Inspect_1** that is defined by the operator. Use the text **“Run inspection routine after how many cycles?”**

- Add a **Popup** “Is a block available for pickup?”.

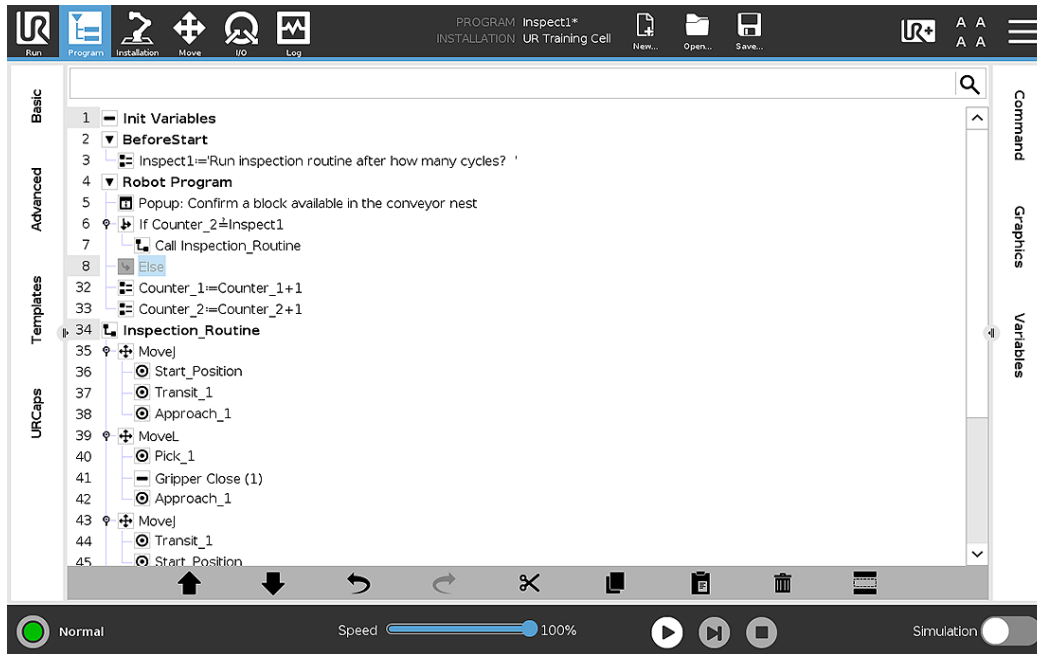


Figure 8.15 – Inspect1 Program Edits

11. If **Counter_2 = Inspect_1**
12. Select **SubProg**.
13. Rename the **SubProg** “**Inspection Routine**”.
14. In the **SubProg** keep the **Move** commands for the picking of the block.
15. After the robot is back to the **Start Position** add a **MoveJ** command with the **Waypoint InspectTransit**.
16. Insert a **MoveL** command with the **Waypoint InspectPlace**.
17. Add a **Gripper Open** command.
18. Insert a **MoveJ** command with the **Waypoints** linked to **InspectTransit** and **Start Position**.
19. Set **Counter_2** to value 0.

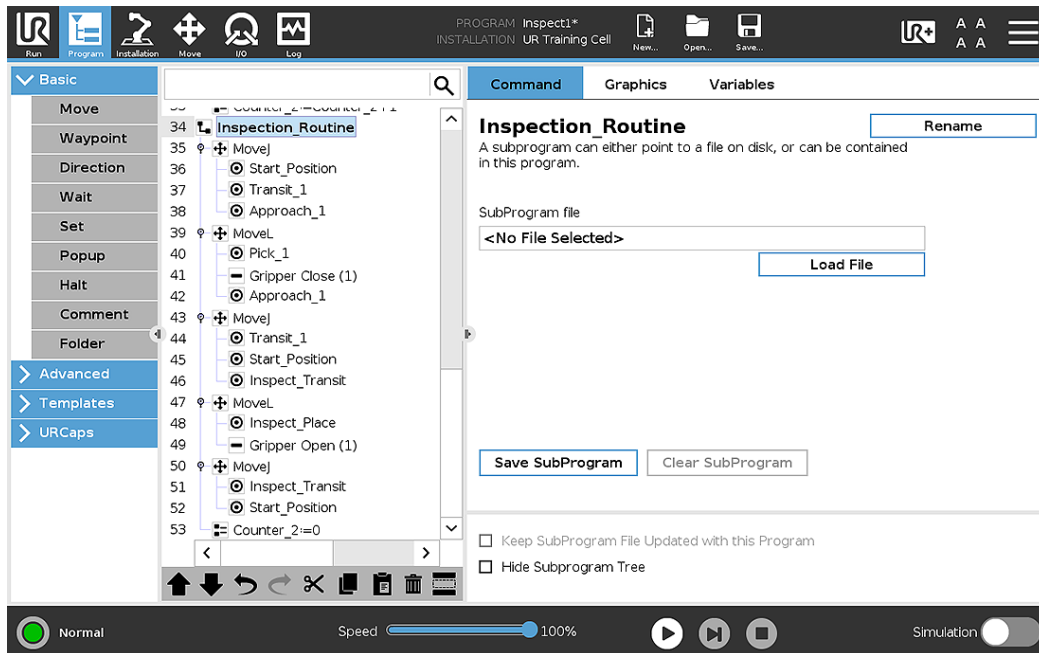


Figure 8.16 – Inspection Routine

20. Go to the **If Command** and select an **Else statement**.
21. Put the pick and place program here with **Counter_1** and **Counter_2** at the bottom.
22. Run program. Select different frequency of block inspections. Loop program. Confirm it works.
23. Save program.

Module 9: Palletizing

Hands-On Activities (UR Academy Student Module)

Source: Universal Robots Academy, Student Module 9 “Palletizing”. Activities and figures reproduced in original order; figures renumbered 9.x (the source labels them 8.x).

Hands-On Activity 1: Palletizing Wizard

1. Open program **PPP2** and Save As program **PPP3**.
2. Delete the folder named **Select Place Point**.
3. Delete **Popup**: Is the pallet clear?
4. Delete the **Approach 2 Waypoint** from the last **MoveJ**.
5. After the **MoveJ** node ending at the **Approach_2** waypoint, Insert a **Palletizing Template** by going to **Program, Templates, Palletizing**.

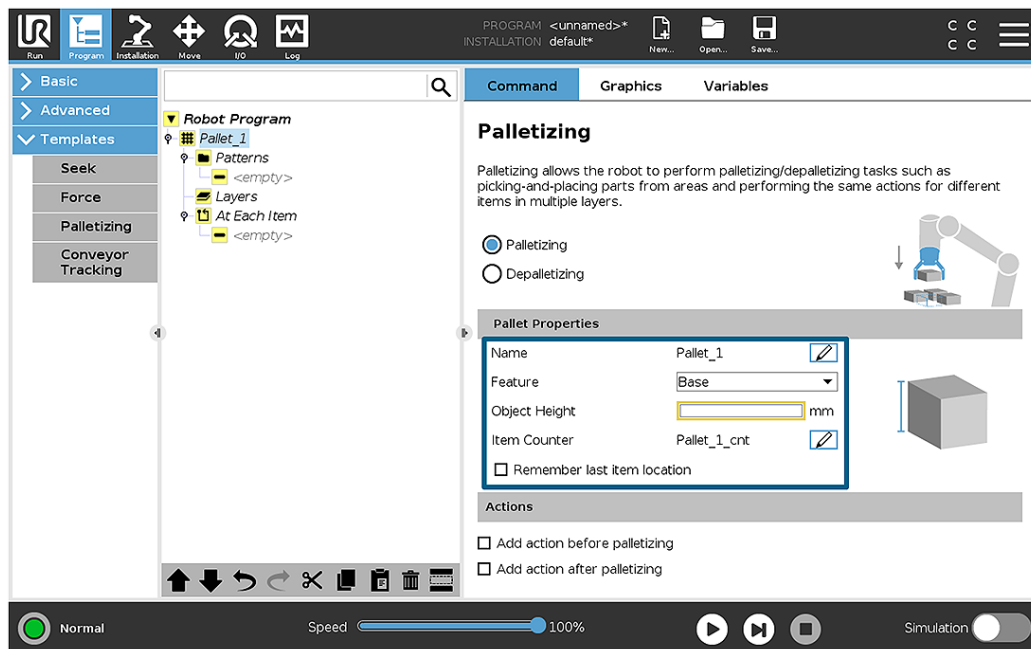


Figure 9.1 – Pallet Template

6. On the **Command tab**, enter an **Object Height** of 19 mm in the appropriate field.

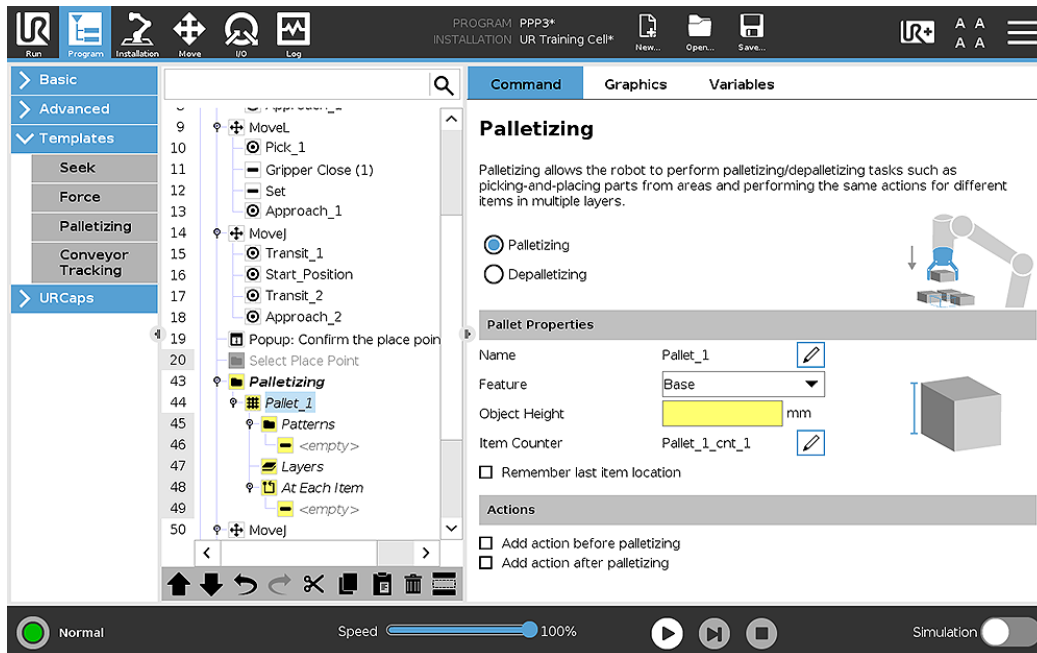


Figure 9.2 – PPP3 Program Edits

7. From the **Patterns** folder select **Grid** as the pattern to match the plate supplied with the training cell.

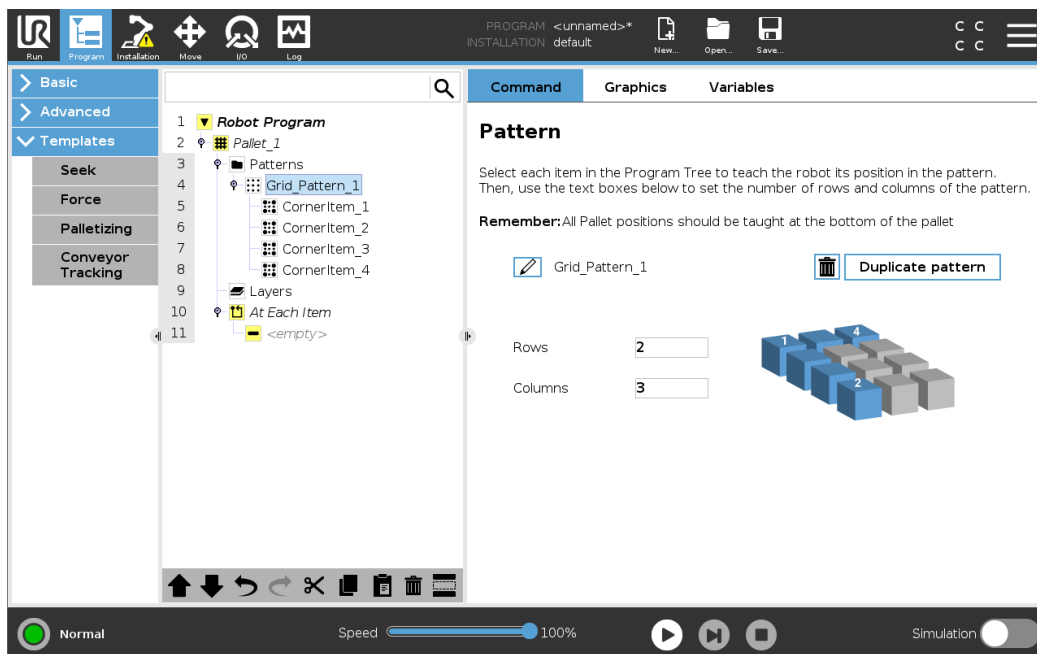


Figure 9.3 – Grid Command

8. Select the **Grid_Pattern_1** node and define the number of rows and columns. Two (2) rows and three (3) columns.
9. Teaching the **Grid Pattern**.
 - Before teaching the four corners required for the palletizing routine, jog the robot to the **Pick_1 Waypoint** and open the gripper if it is closed.

- Place a block in the nest and close the gripper. Leave the block in the gripper while teaching the corner points.
- **CornerItem_1** will be the furthest point from the robot on the grid. Teach the other corner items in the clockwise direction as shown in the images below. The four corners help the robot determine the overall size of the pallet.

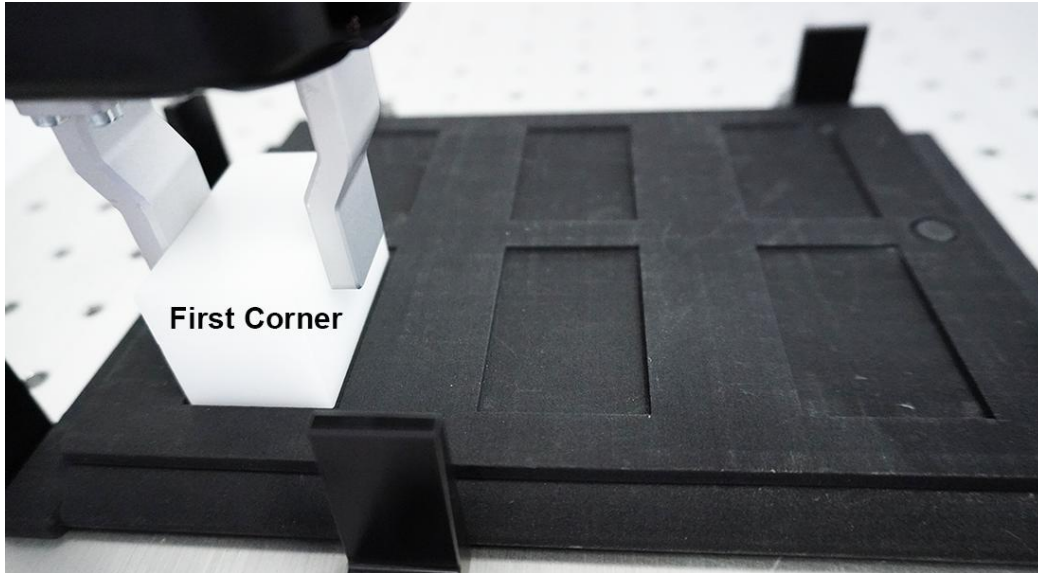


Figure 9.4A – First Corner Location

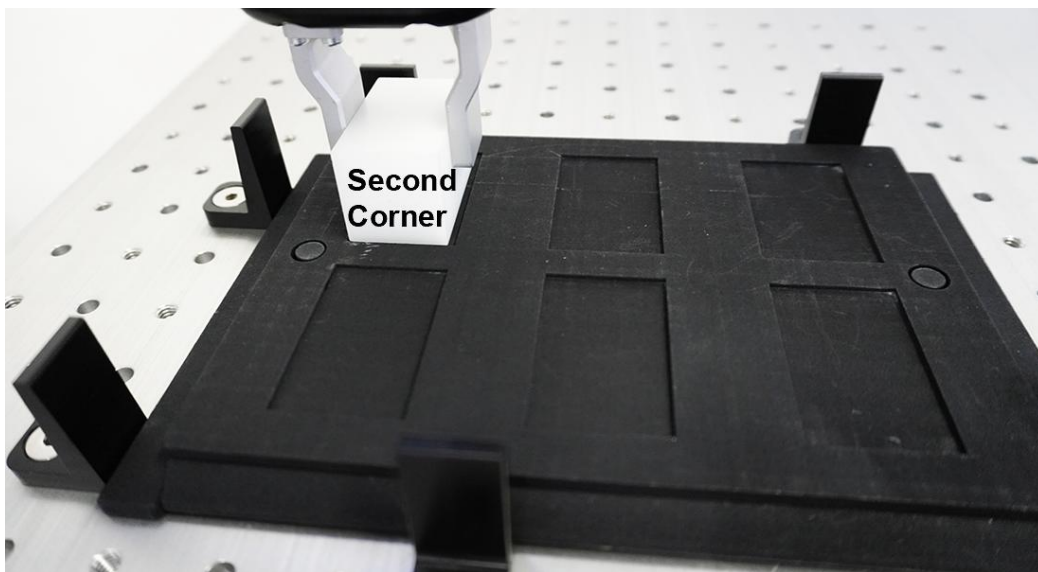


Figure 9.4B – Second Corner Location

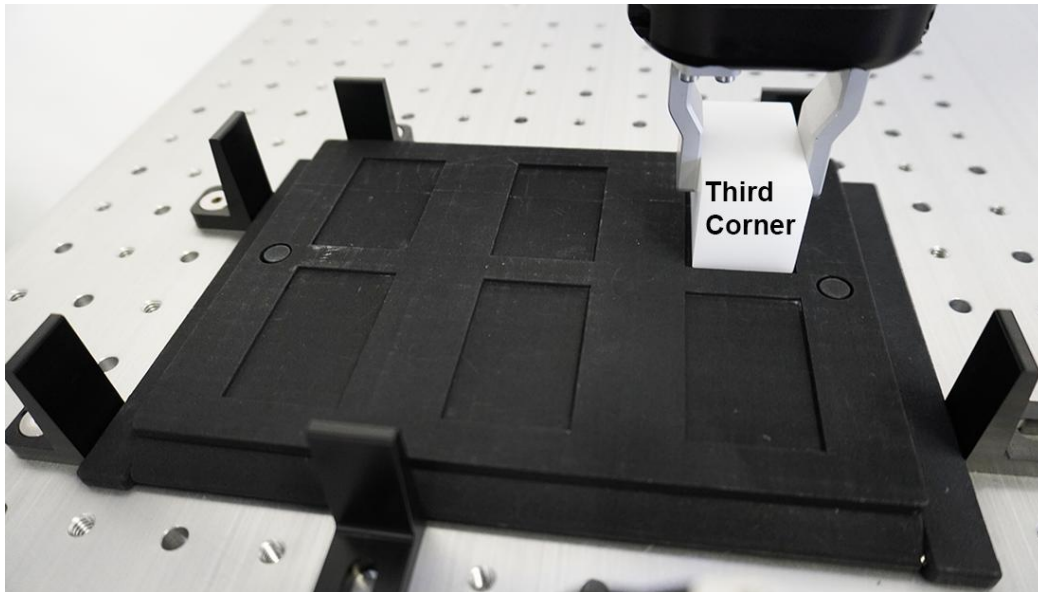


Figure 9.4C – Third Corner Location

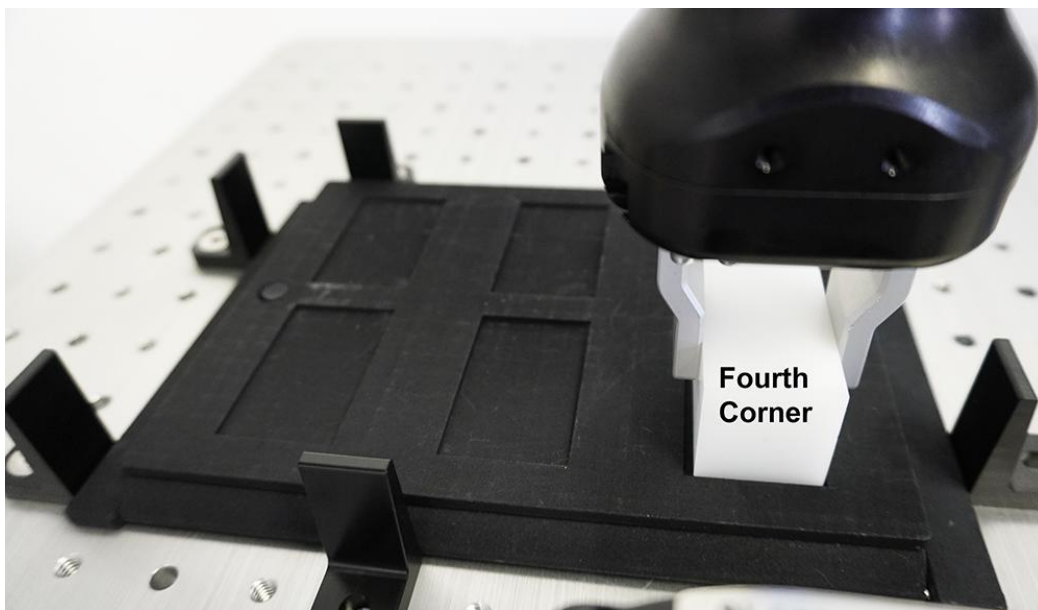


Figure 9.4D – Fourth Corner Location

10. Select the **Layers node** and choose **Grid_Pattern_1** from the **Choose Pattern drop-down menu** on the **Command tab**.
11. The layer count for this exercise is one (1). No additional layers should be added.

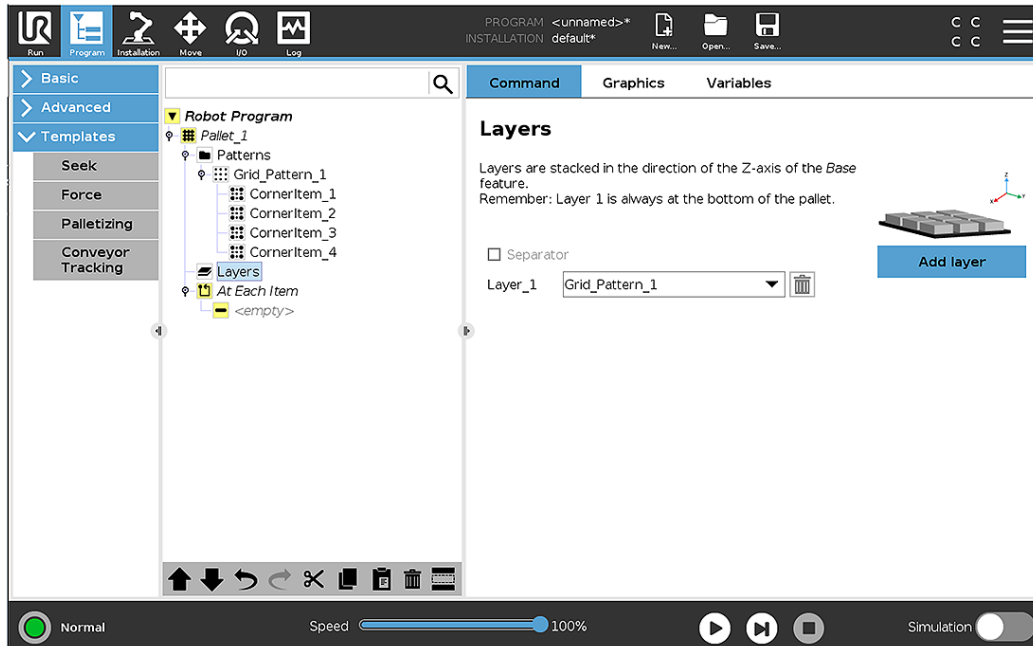


Figure 9.5 – PPP3 Program Edits

12. Define the actions performed **At Each Item** location on the pallet by completing the six steps in the **At Each Item** Wizard.

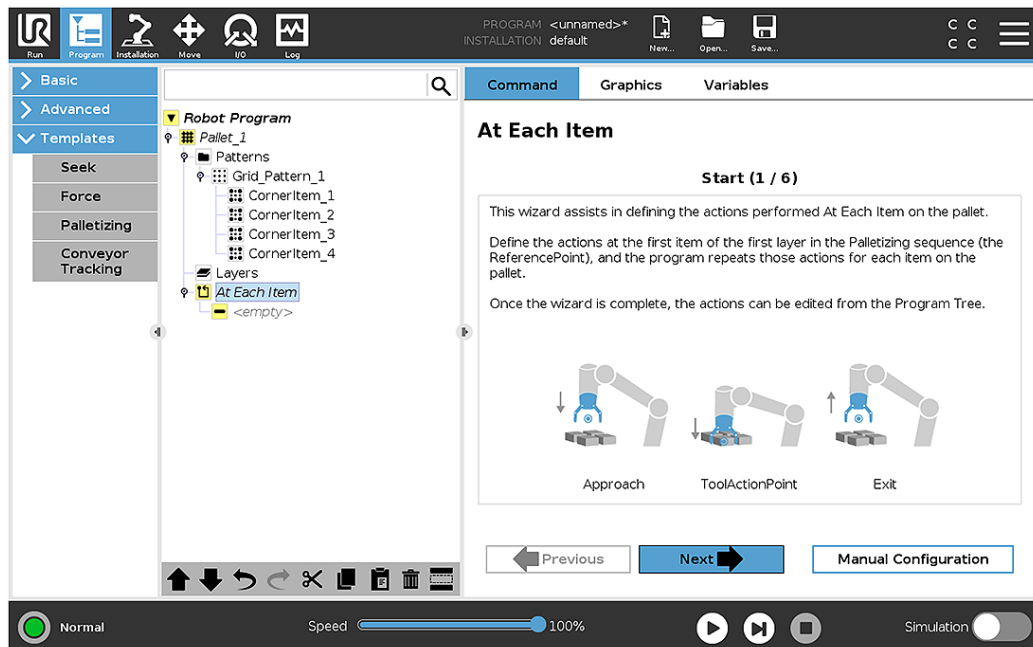


Figure 9.6 – At Each Item Wizard

13. Define **Approach Point**. This is typically a Z+ move to a safe clearance point above the **Tool Action Point**.
14. Define **Tool Action Point** (this is the same as **CornerItem_1**).
15. Define **Exit Point**.

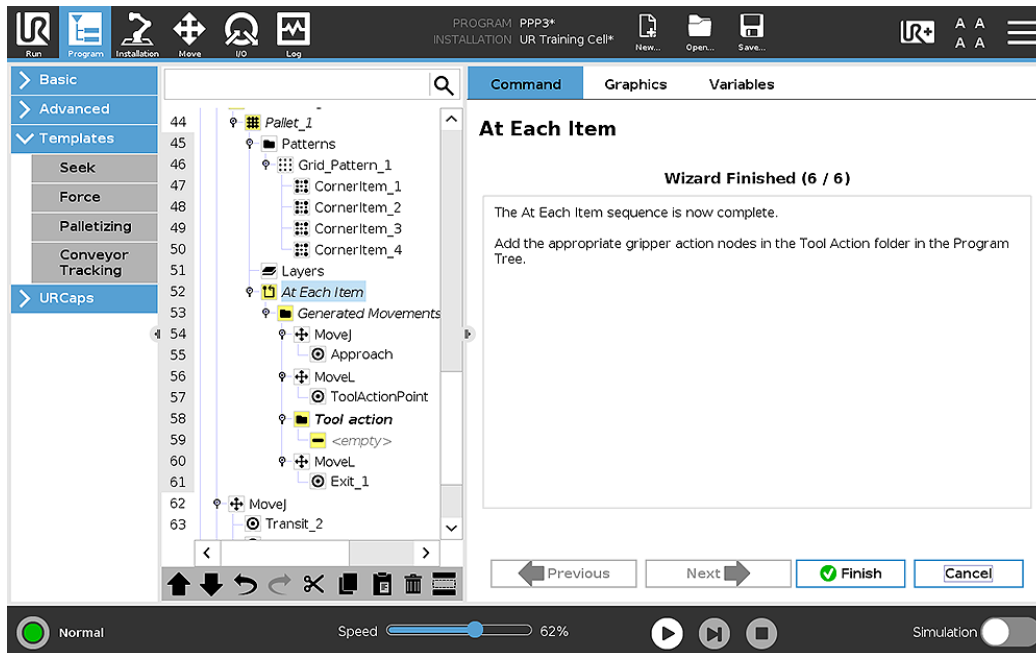


Figure 9.7 – At Each Item Wizard

16. Define the **Tool Action**.
17. Insert a **Gripper Open** command.

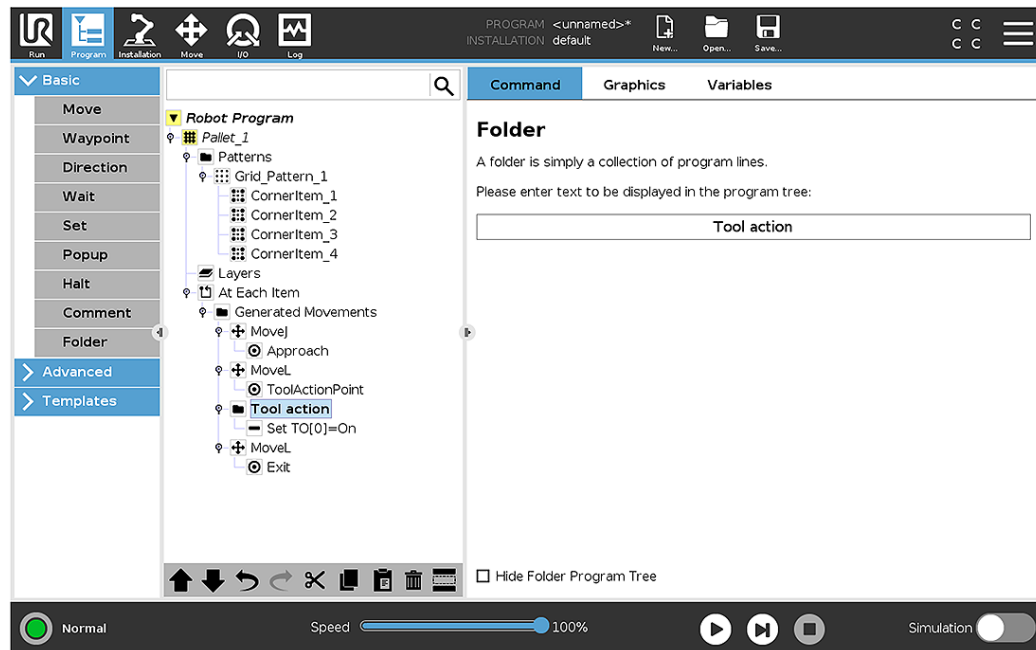


Figure 9.8 – Tool Action

18. Use default speed settings for **MoveJ** commands. **Joint Speed** 60.0°/s, **Joint Acceleration** 80.0°/s².
19. Set speed of **MoveL** to 50 mm/s.
20. Press the **Robot Program** node at the top of the tree. Check the **loops forever** checkbox.

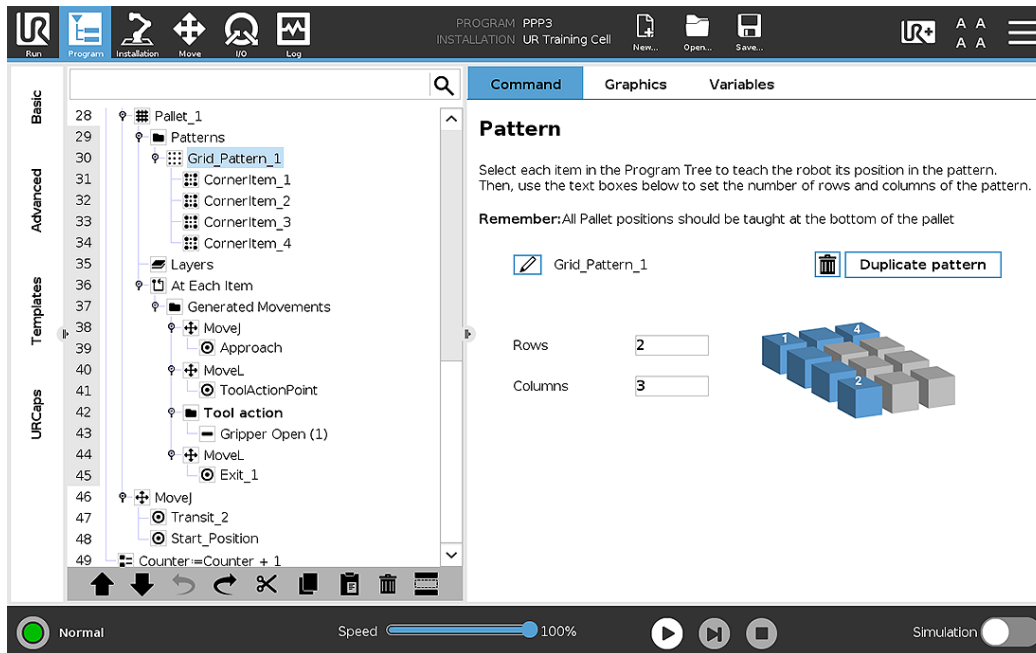


Figure 9.9 – Palletizing Template

21. Save the program.

Hands-On Activity 2: Before Start Commands

1. Go to **Installation tab** in the header. Select **General** and **Setup IO**. Rename **digital_out[2]** to **Conveyor Power**.
2. Go to **Installation tab** in the header. Select **General** and **Setup IO**. Rename **analog_out[1]** to **Conveyor Speed**.
3. Go to **IO tab** in the header. For the analog output **Conveyor Speed** select **Voltage** in the pull-down menu.
4. Go to **Program tab** in header.
5. Add a **Counter Variable**.
6. Press the **Robot Program node** and check the **Initialize checkbox**.
7. Initialize **Counter** to a value of 1.
8. Press the **Robot Program node** and check the **Add Before Start Sequence checkbox**.
 - Have the program wait for an **Operator** input at the start of the program.
9. Insert a **Popup** with text: **"Confirm an Empty Pallet is Ready"**
10. Set **Digital Output Conveyor Power** to On (High).
11. Set **Analog Output** to 4.0 Volts.
 - Insert **timer_1** and select **Start**.
 - Insert the following logic immediately after the **Robot Program node**:
 - **If statement** with the following Expression **Counter $\geq$ 7**
12. Add the following logic under the **IF statement**.

- Insert **timer_1** and select **Stop**.
 - Insert a **Popup** with **Cycle Time Value (timer_1)**.
 - Set **Digital Output digital_out[2]** to **DO2 Off (Low)**.
13. Insert a **Popup** with text: **“Empty Pallet”** and **Halt** program.
14. After **If** command insert a **Wait** command for **Pallet Sensor** to be On.
- Initialize all variables prior to the start of the program.

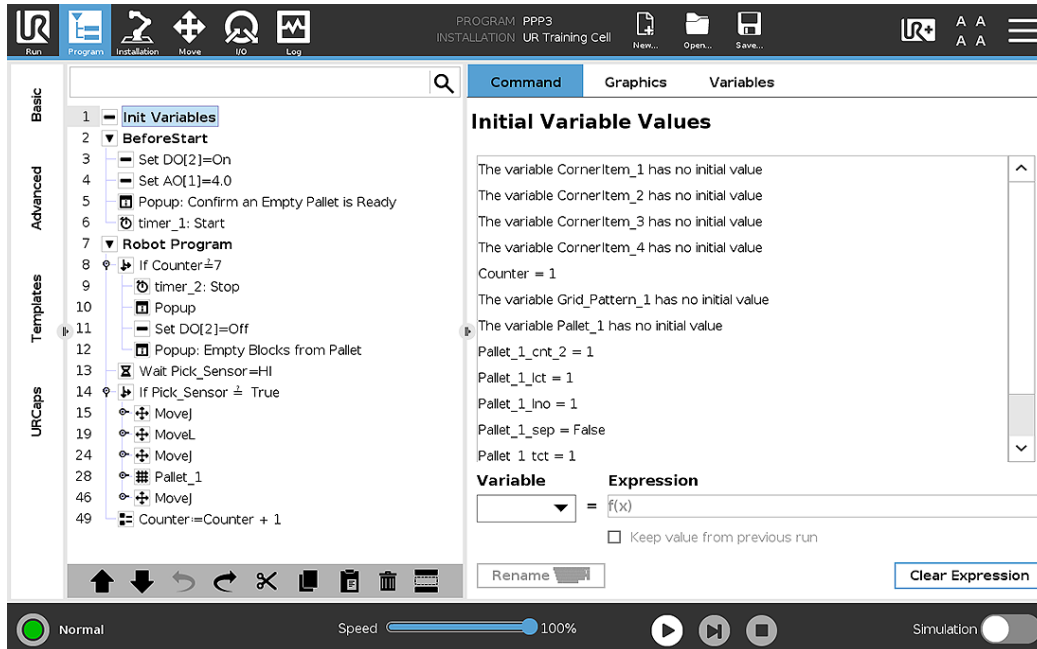


Figure 9.10 – Before Start Program

Hands-On Activity 3: Run and Optimize the Palletizing Program

1. Place a block on the infeed conveyor in the guide when the robot picks a block.
2. Run program.
 - Did the program run successfully?
 - What is the cycle time?
 - With our testing setup it was approximately 117 seconds.
3. Optimize your **Move** commands.
 - Change speed to 100°/s and acceleration to 120°/s² on **MoveJ**.
 - Change speed to 250 mm/s on **MoveL**.
4. Run program.
 - What is the cycle time?
 - With our testing setup it was approximately 89 seconds.
5. Save program.

Module 10: Features/Planes

Hands-On Activities (UR Academy Student Module)

Source: Universal Robots Academy, Student Module 10 “Features/Planes”. Activities and figures reproduced in original order; figures renumbered 10.x (the source labels them 9.x).

Hands-On Activity 1: Creating a Plane Feature

1. Open the **PPP3** program and Save As **PPP4**.
2. Create a plane feature for the palletizing grid.
3. Press the **Installation Icon** and press the **Features menu**.
4. Select **Plane** to add the first plane to the **Features List**.
5. Press **Teach this plane** to open the **Plane Wizard**.

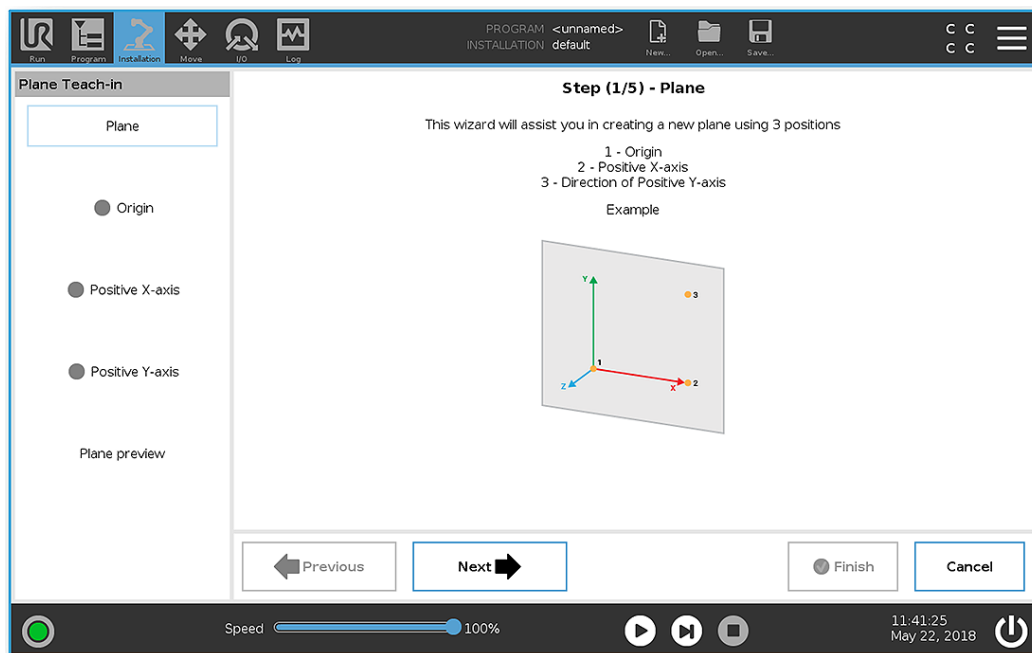


Figure 10.1 – Plane Feature Wizard

- Complete the 5 steps of the **Plane Wizard**. Use the **Corner Location** from the **Pallet Template** for the **Origin, Position 1**.

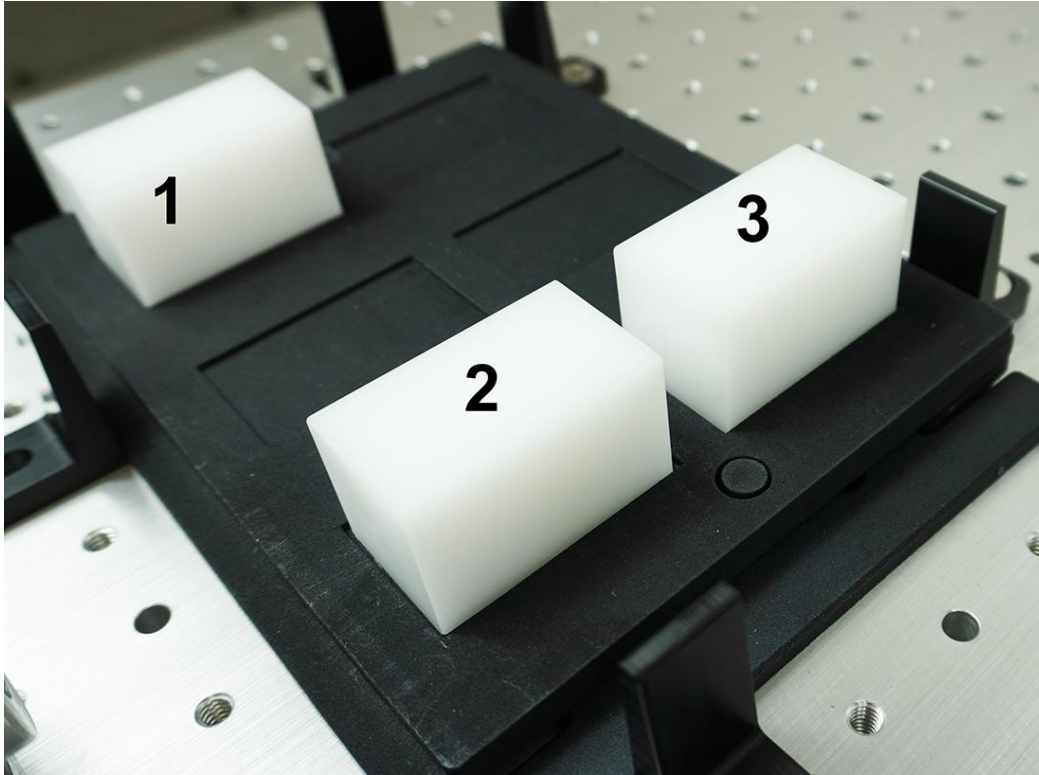


Figure 10.2 – Flat Pallet Wizard Points

6. Go to first corner. Place a block in position. Manually trigger the gripper closed.
7. Set **Origin, Position 1** at the first corner.
8. Raise the robot slightly in Z and move in the positive X direction to the second corner of the pallet. Move to the place position on the pallet by lowering the robot slightly in the Z direction. Set **Positive X-Axis Position 2**.
9. Raise the robot slightly in Z and move in the positive Y direction to the corner of the pallet. Move to the place position on the pallet by lowering the robot slightly in the Z direction. Drop in Z direction to be in place position on pallet. Set **Positive Y-Axis Position 3**.
10. Rename **Plane_1** as **Flat Pallet**.
11. Press the **Program Icon** and select the **Pallet_1** node. In the command window, press the **Feature drop-down menu** and select the **Flat_Pallet Feature**.

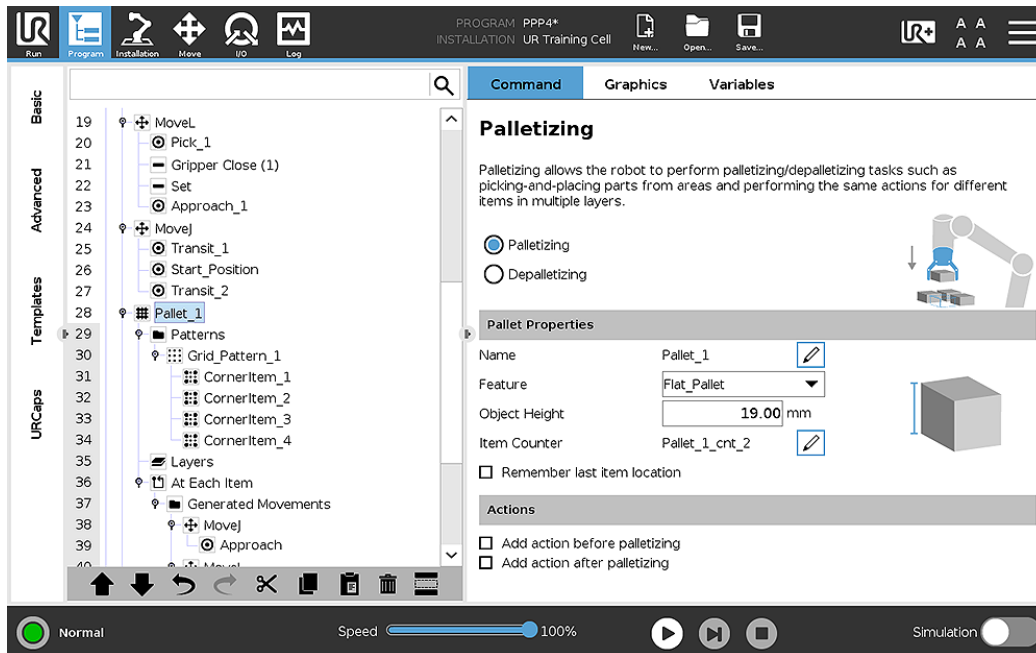


Figure 10.3 – Flat Pallet Plane Feature

12. Navigate to the **Pallet_1** node in the **PPP4** program tree. Press the **At Each Item** node and press the **Redo Wizard** button located in the command window. Use the wizard to update all **Waypoints** and **Gripper Action**.
 - Note: Any **Waypoints** taught with a **Wizard** must be retaught with the **Wizard**!
13. Run the palletizing routine to confirm it is still functional.
14. Save As **PPP4Flat**.

Hands-On Activity 2: Create the Tilted Palletizing Grid

1. Create a plane feature for the tilted palletizing grid.
2. Add a second plane feature and complete the 5 steps of the **Plane Wizard**.
 - Use the **Corner Location** from the **Pallet Template** for the **Origin, Position 1**.

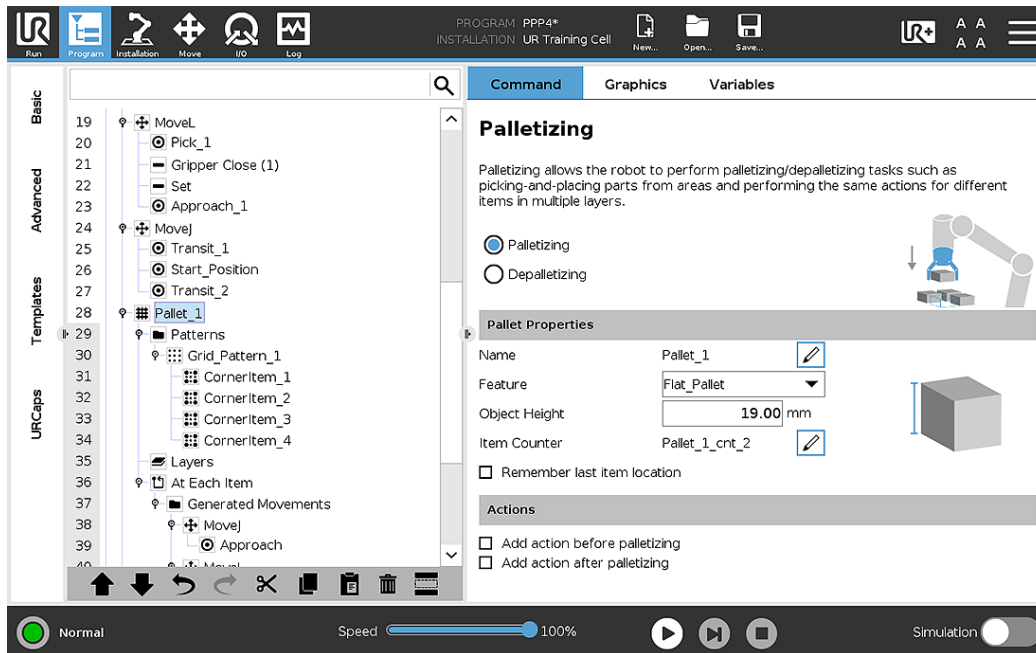


Figure 10.4 – Tilted Pallet Wizard Points

3. Go to first corner. Place a block in position. Manually trigger the gripper close.
4. Press the **Installation Tab**. Select **Features** then **Plane**. Complete **Wizard**.
5. Jog robot to the **Origin** Position of the tilted pallet.
 - Consider selecting the **Tool Feature** to Jog the robot for this exercise.
 - Use RX- to adjust the block angle to match the angle of the tilted pallet.
 - Now you use the X, Y and Z directions to move the robot in position.
 - Place a block in position. Manually trigger the gripper close.
6. Set **Origin, Position 1** at the first corner.
7. Raise the robot slightly in Z. Switch to the **Plane Feature** by selecting it in the pull-down menu. Move in the positive X direction to that corner of the pallet. Switch to the **Tool Feature**. Drop in Z+ direction to be in place position on pallet. Set **Positive X-Axis Position 2**.
8. Raise the robot slightly in Z. Switch to the **Plane Feature** (same as above) and move in the positive Y direction to that corner of the pallet. Switch to the **Tool Feature**. Drop in Z+ direction to be in place position on pallet. Set **Positive Y-Axis point Position 3**.
 - Complete **Wizard** for **Plane_1**. Rename as **Tilted_Pallet**.

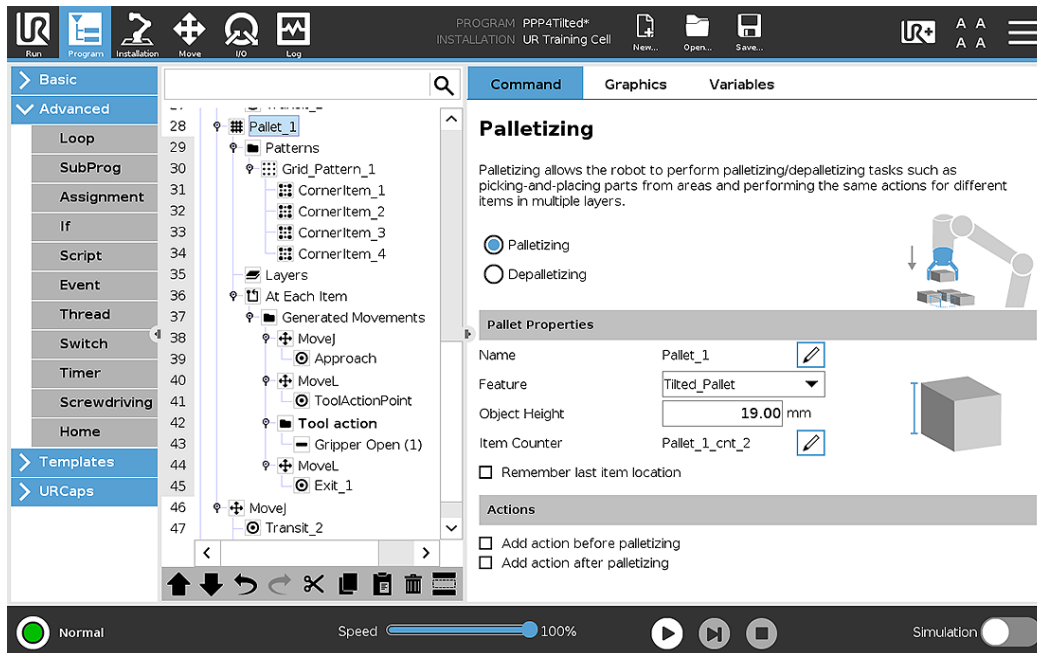


Figure 10.5 – Tilted Pallet Plane Feature

9. Run the palletizing routine to confirm it is still functional.
10. Save As **PPP4Tilted**.

Hands-On Activity 3: Jogging Relative to a Feature

1. Press/Tap the **Move Icon** and press the **Feature** pull-down menu.
2. The **Features Flat_Pallet** and **Tilted_Pallet** have now been added to the feature list.
3. Move the robot above the pallet location.
4. Jog the robot around the pallet location using the 5 different features.
 - **View**
 - **Base**
 - **Tool**
 - **Flat Pallet**
 - **Tilted Pallet**

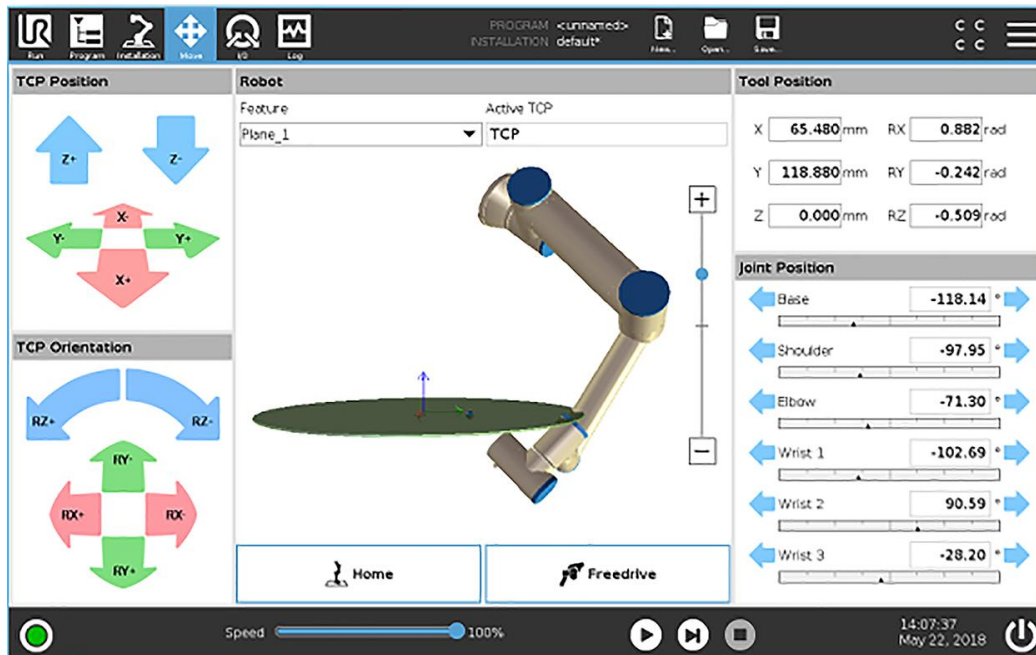


Figure 10.6 – Jog Relative to a Feature Example

Hands-On Activity 4: Selecting a Variable Feature to the Palletizing Program

1. Have the operator select if the feature plane is flat or at an angle.
2. In the **Before Start Program** have the operator assign a **Variable Pallet_Tilt** to 1 for flat and 2 for tilted.

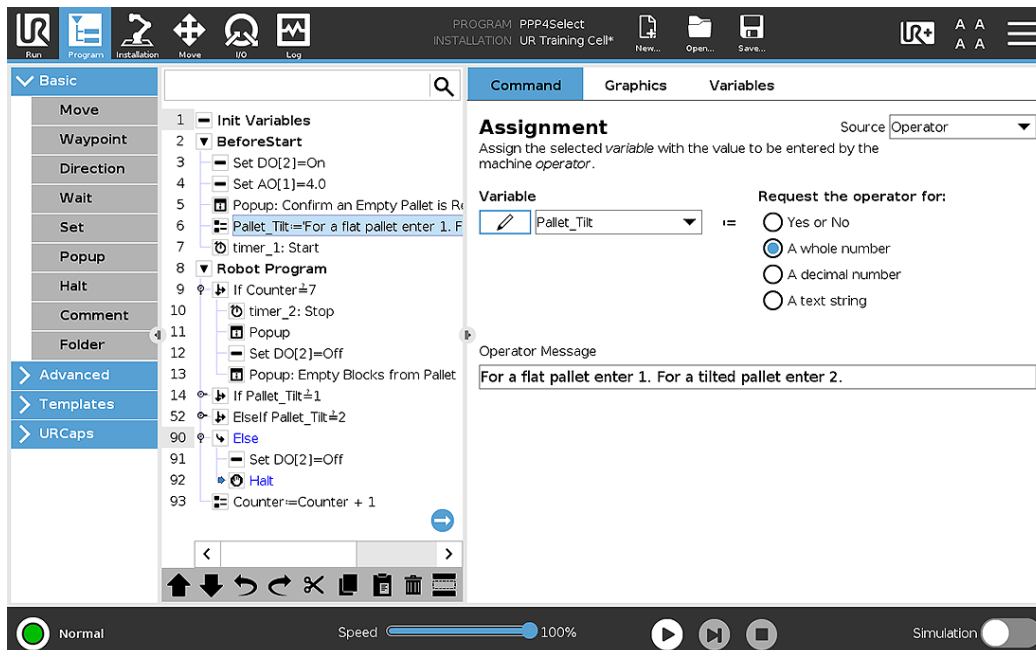


Figure 10.7 – Plane Feature Selection

3. Add an **If** command for when **Pallet_Tilt $\neq$ 1**

4. Create the **Folder** named **Pallet_Tilt Flat**.
5. Move the main program into the folder.
6. Make sure the **Feature** used for Palletizing has been changed to **Pallet_Flat**.

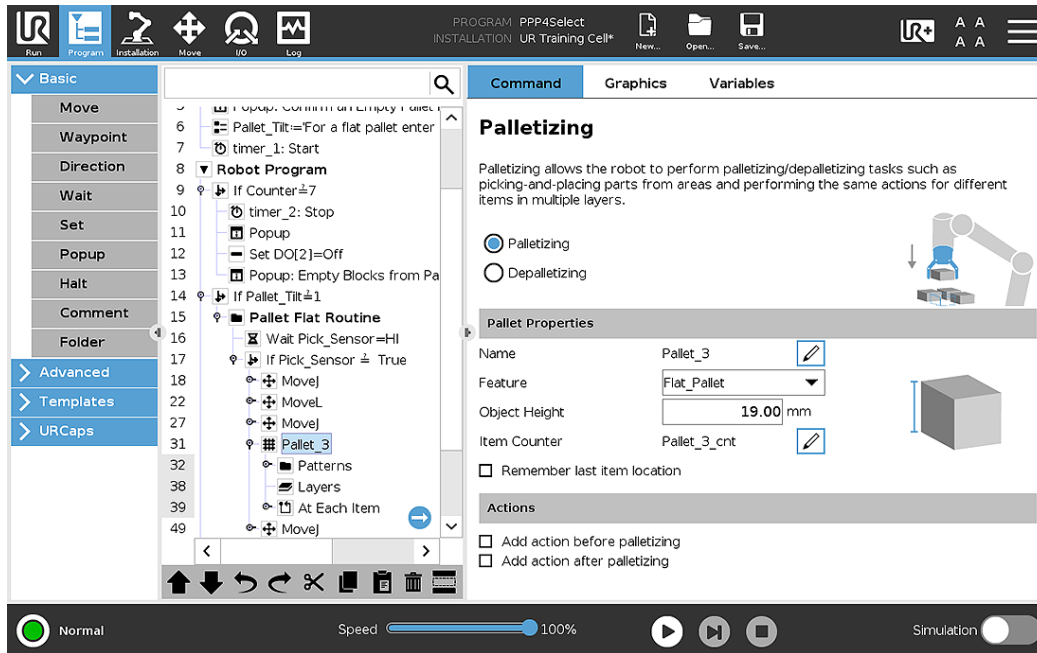


Figure 10.8 – Pallet Flat Folder

7. Add the **Elseif** command for **Pallet_Tilt = 2**
8. Copy the **Folder** named **Pallet Tilt Flat** and paste it under the **Elseif**.
9. Rename the folder to **Pallet_Tilted**.
10. Make sure the **Feature** used for Palletizing has been changed to **Pallet_Tilted**.

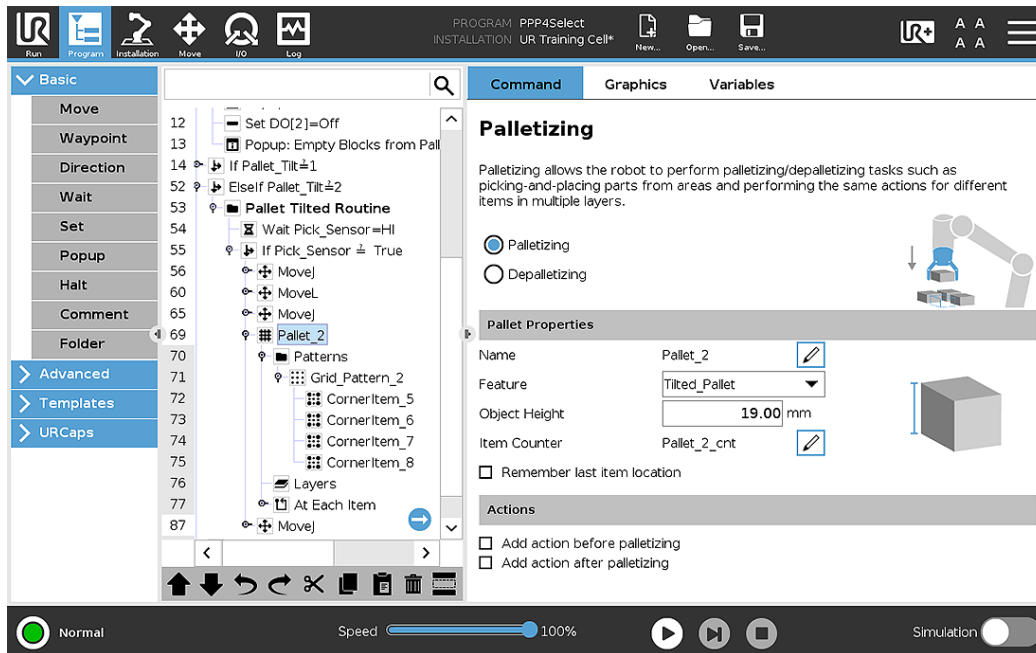


Figure 10.9 – Pallet Tilted Folder

11. Add an **Else** statement.
12. Set **DO(2)** to Off.
13. **Halt** program.

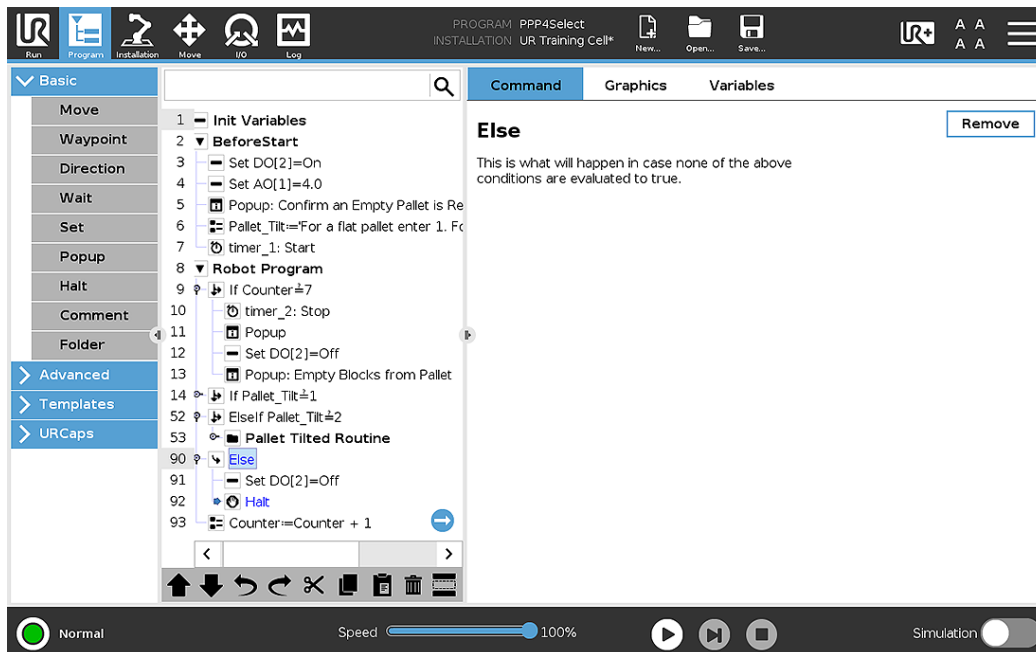


Figure 10.10 – Else Statement

14. Run program. Verify the program will run for both operator selections.
15. Save As **PPP4Select**.

Module 11: Safety Settings

Hands-On Activities (UR Academy Student Module)

Source: Universal Robots Academy, Student Module 11 “Safety Settings”. Activities and figures reproduced in original order; figures renumbered 11.x (the source labels them 10.x).

Hands-On Activity 1: Factory Presets

1. Open program **PPP3**.
2. Place a block on the infeed conveyor in the guide when the robot picks a block.
3. Confirm settings in your **Move commands**.
 - Speed of $100^\circ/\text{s}$ and acceleration of $120^\circ/\text{s}^2$ on **MoveJ**.
 - Speed of 250 mm/s on **MoveL**.
4. Run program **PPP3**.
 - What is your cycle time in seconds? Approximately 90 seconds?
5. Go to the **Installation tab** in the header. Select **Safety** and **Robot Limits**. Enter the safety password provided by instructor.
6. Change the **Safety Preset** from: **Most Restrictive** to **Restrictive**.

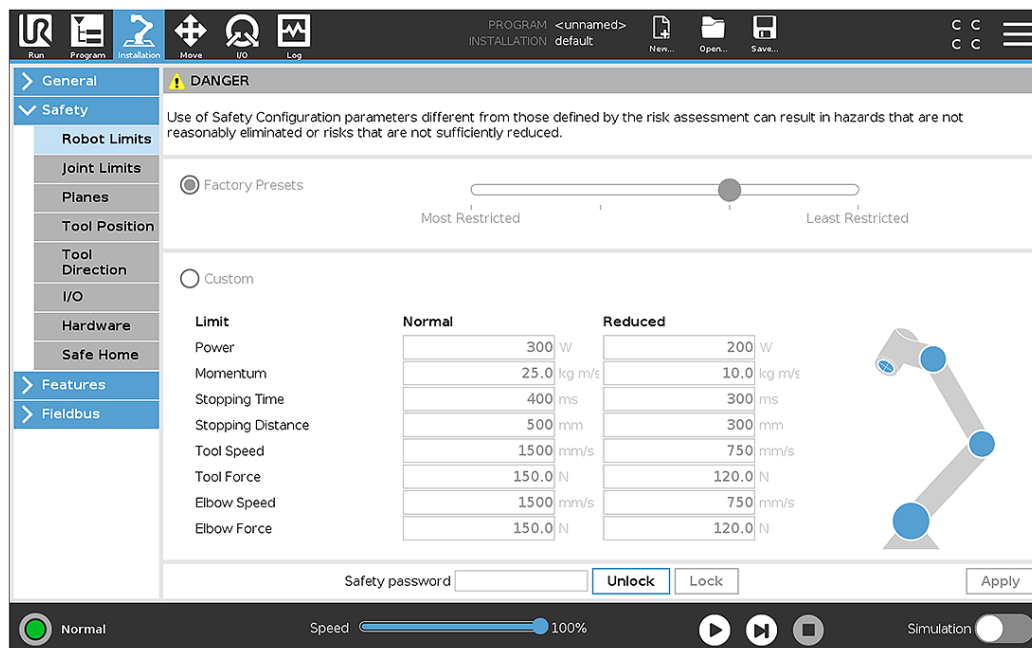


Figure 11.1 – Safety Presets

7. Apply and Restart the robot (not the controller).
8. Confirm **Safety Configuration** and Initialize robot.
9. Run the program. What is your cycle time in seconds? Approximately 75 seconds?
 - Note the speeds that were being limited are affected.

- Changing a safety setting to increase speed requires validating the change in the **Risk Assessment**. Additional safety measures may be required.

10. Change the **Safety Preset** back to **Most Restrictive**.

11. Save All Programs.

Hands-On Activity 2: Joint Limits

1. Go to the **Installation tab** in the header. Select **Safety** and **Joint Limits**. Enter the safety password provided by instructor.
2. Restrict the following axis as defined below:

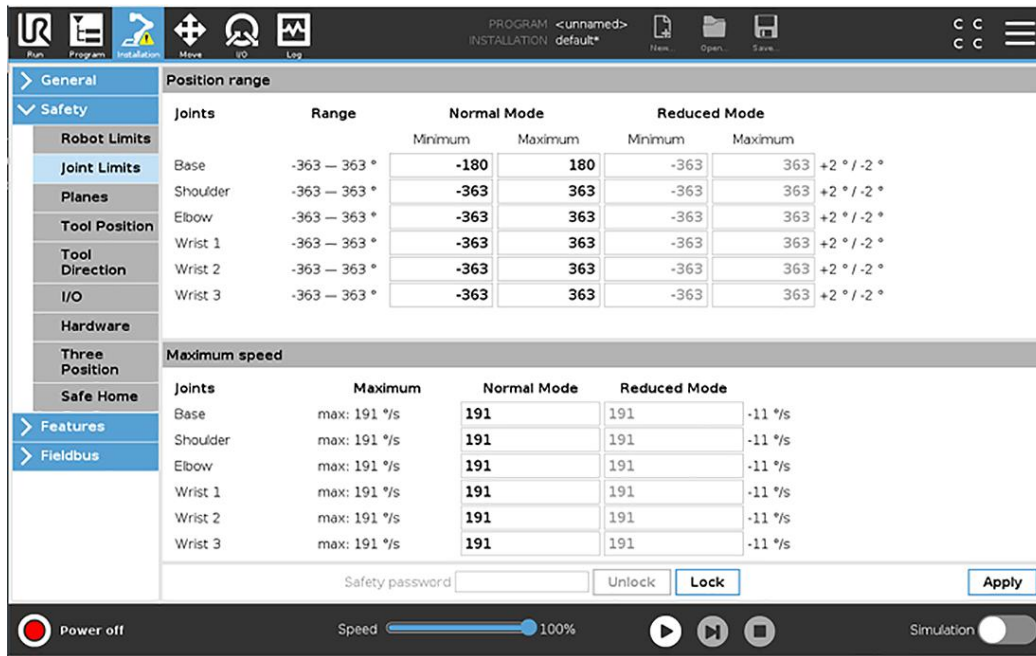


Figure 11.2 – Joint Limits

- **Base (Axis 1):** -30, +180.
 - **Shoulder (Axis 2):** -180, -45.
 - **Elbow (Axis 3):** -45.
 - **Wrist 1 (Axis 4):**
 - **Wrist 2 (Axis 5):**
 - **Wrist 3 (Axis 6):**
3. Apply and Restart the robot.
 4. Confirm **Safety Configuration** and Initialize robot.
 - Note the **Safety Reference Block** in the **Header** changes each time the **Safety Settings** are modified.
 5. Go to **Move Screen** and see the limit restrictions in the **Joint Position Window**.

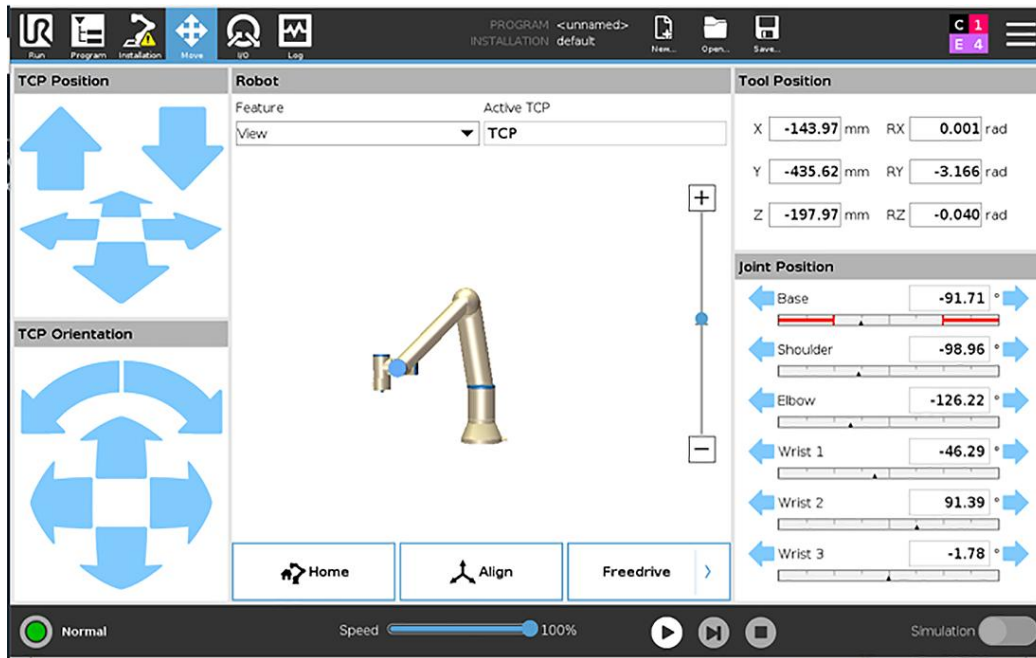


Figure 11.3 – Joint Limits in Move Window

6. Run the program and confirm it functions within **Joint Limits**.

Hands-On Activity 3: Create a Safety Plane

1. Go to the **Installation** tab in the header. Select **Safety** and **Planes**. Enter the safety password provided by instructor.
2. Create a **Safety Plane** for the tabletop base.

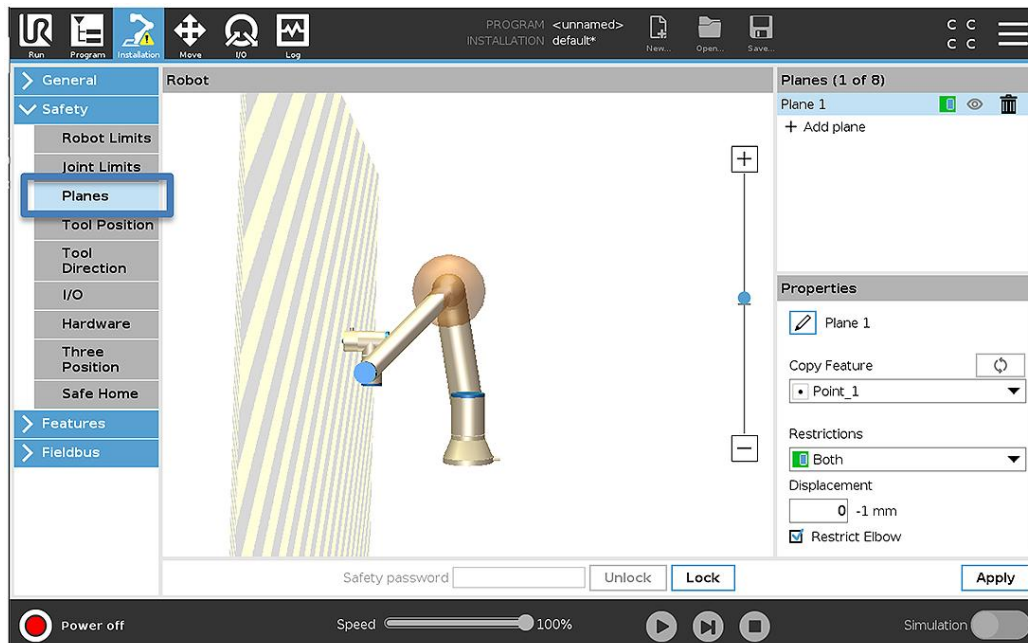


Figure 11.4 – Safety Plane

3. Apply and Restart the robot.
4. Confirm **Safety Configuration** and Initialize robot.
5. Add **Plane_1**.
6. Copy **Base Plane**.
7. Select **Normal** in pull-down menu.
8. Enter a 50 mm **Displacement Value**.
 - Do not check **Restrict Elbow checkbox**.
9. Run the program and confirm it functions within **Safety Plane Limits**.

Hands-On Activity 4: Create a Tool Flange Safety Sphere with Direction

1. Go to the **Installation** tab in the header. Select **Safety** and **Tool Position**. Enter the safety password provided by instructor.
2. Copy **TCP Point Feature** and enter a value of 75 mm for the radius.

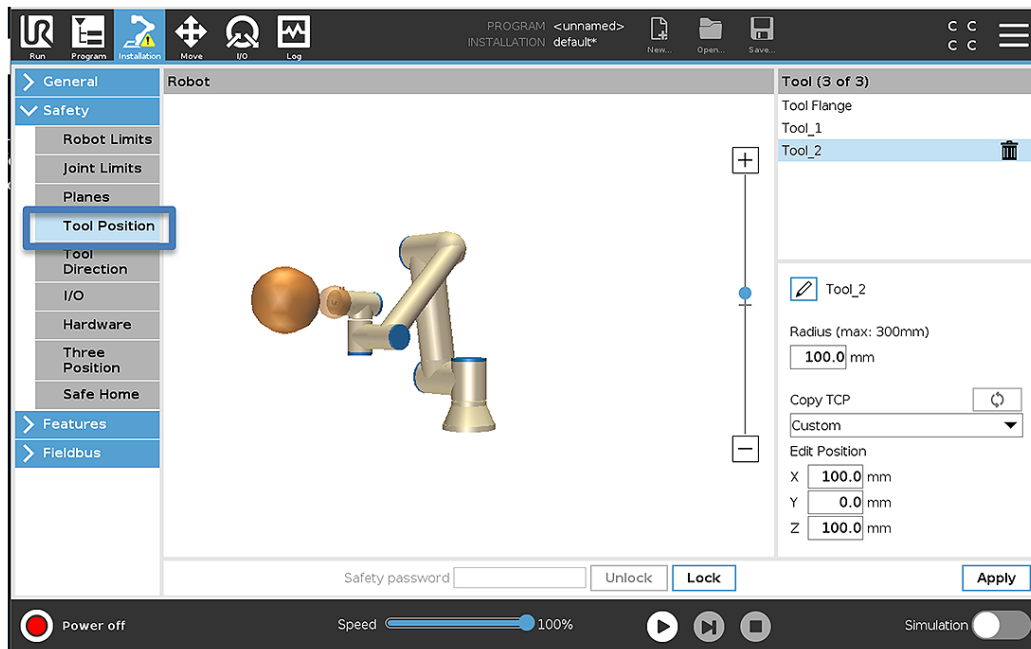


Figure 11.5 – Tool Position Limits

3. Select **Normal** in pull-down menu.
4. Apply and Restart the robot.
5. Confirm **Safety Configuration** and Initialize robot.
6. Go to the **Installation** tab in the header. Select **Safety** and **Tool Direction**. Enter the safety password provided by instructor.
7. Copy **TCP At Start Feature** and enter a value of 30 for the deviation.

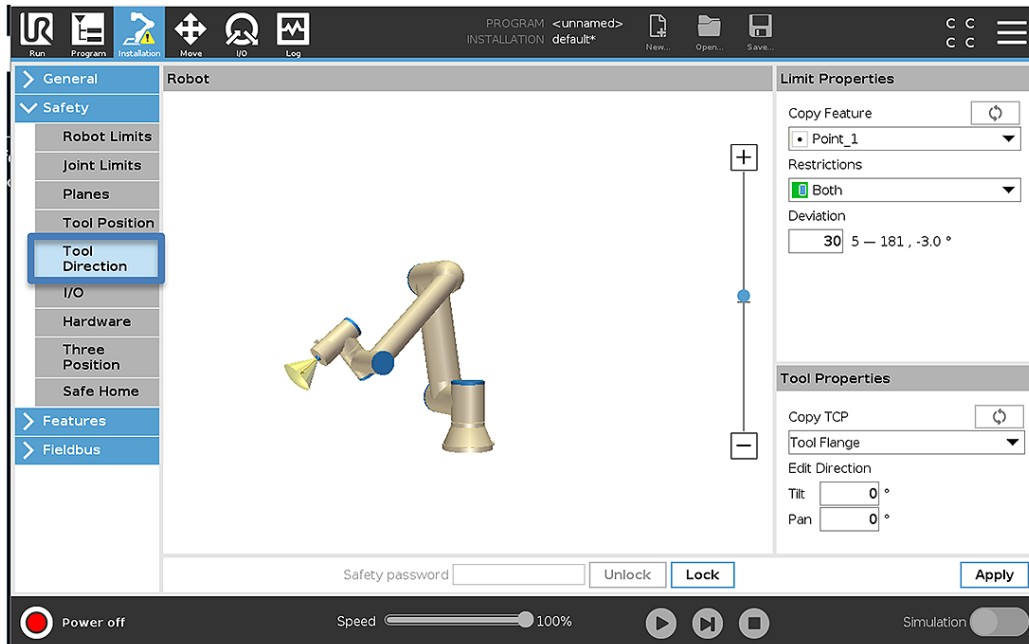


Figure 11.6 – Tool Direction Limits

8. Select **Normal** in pull-down menu.
9. Apply and Restart the robot.
10. Confirm **Safety Configuration** and Initialize robot.
11. Run the program and confirm it functions within **Tool Position and Direction Limits**.
12. Save program.

Hands-On Activity 5: Fault Recovery Modes

The example of Figure 11.7 shows what to expect when recovering from a safety violation. This would include any external safety devices added to a robot cell.

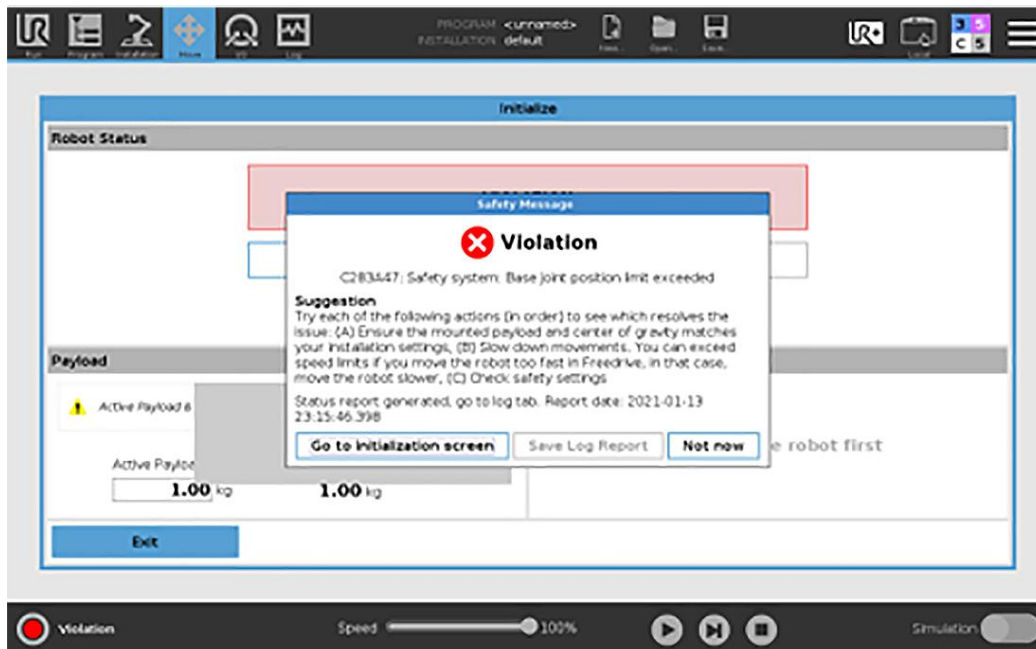


Figure 11.7 – Fault Recovery

Recovery Mode. After encountering a violation manually jog robot away from obstruction to continue.

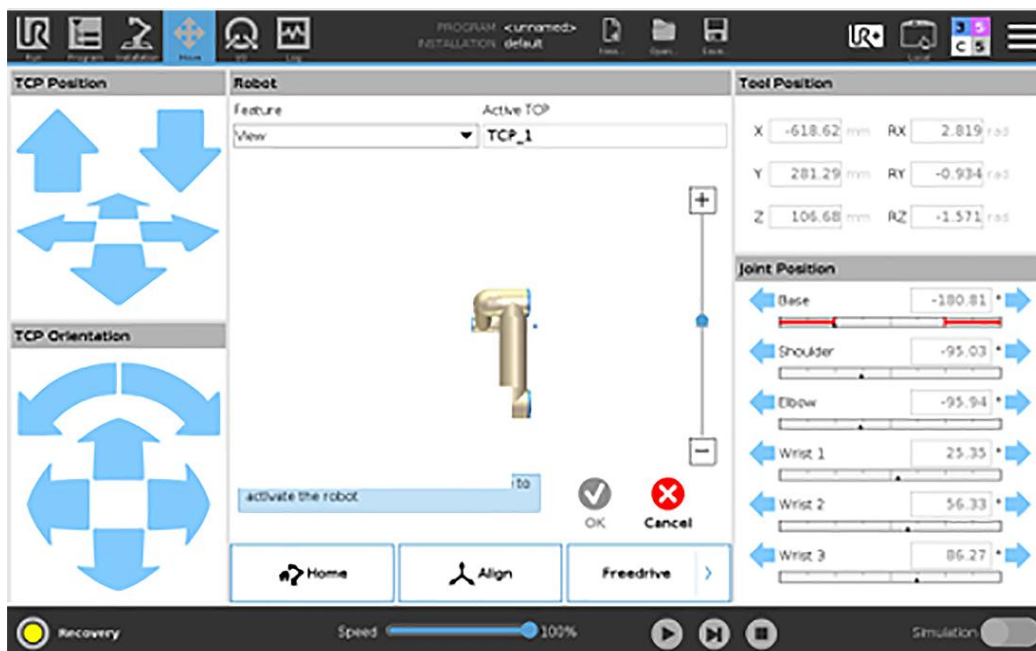


Figure 11.8 – Fault Recovery

1. Go to the **Start Position Waypoint**. Go to the **Move** tab and rotate the **Base** axis in the positive direction. You will see a **Joint Limit Violation** for the **Base** axis when it reaches 120 degrees.



Figure 11.9 – Joint Limit Violation

2. Manually jog robot away from obstruction to continue.
3. Go to the **Start Position Waypoint**. Go to the **Move** tab and rotate the **Base** axis in the positive direction. You will see a **Plane Limit Violation** for the **Safety Plane** when the gripper limit sphere reaches the plane limit.
 - **Safety Plane Violation.**



Figure 11.10 – Safety Plane Error Message

4. Press the **Enable** button to activate robot.

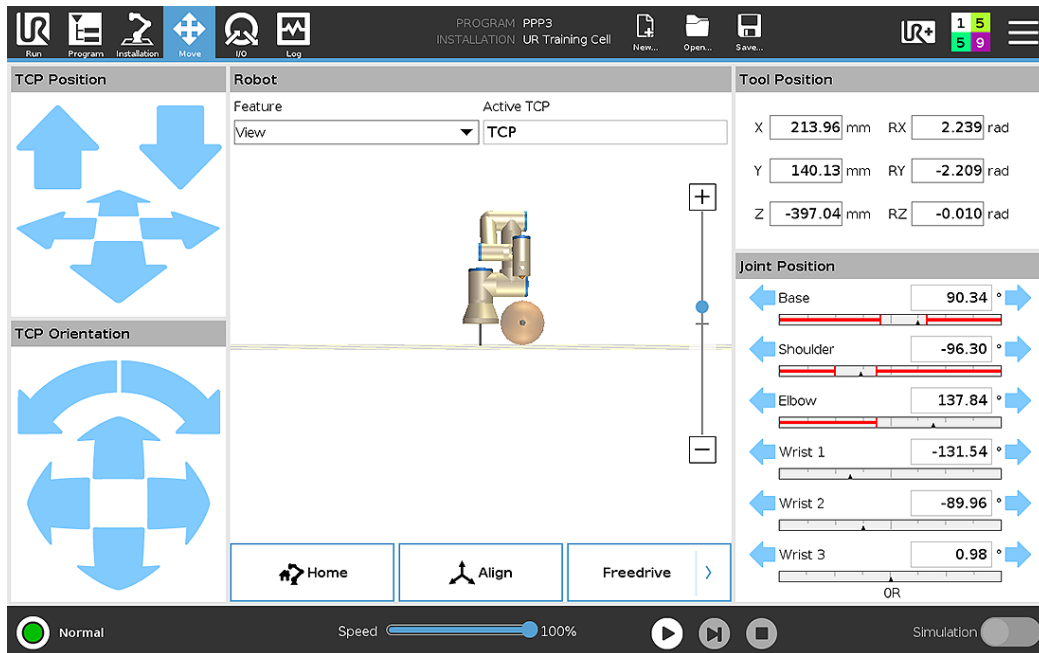


Figure 11.11 – Safety Plane Robot Pose

5. Manually jog robot away from obstruction to continue.